

Network Layer

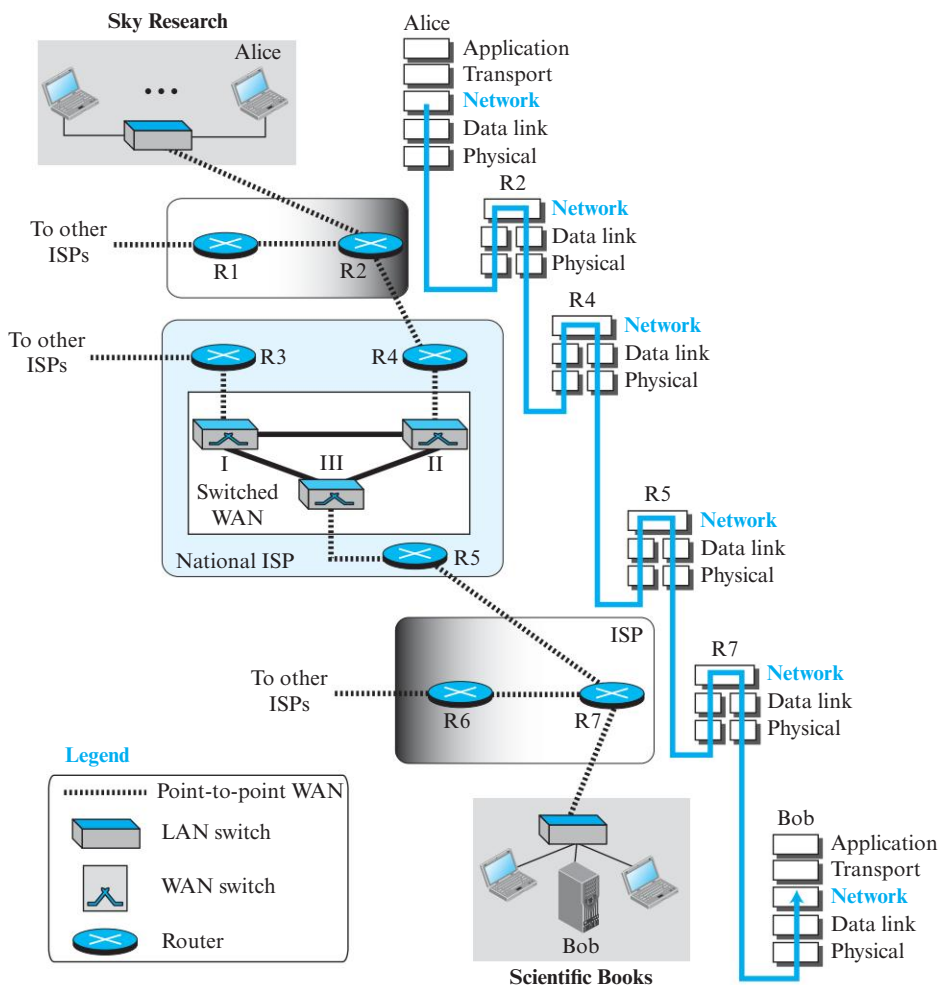
The network layer in the TCP/IP protocol suite is responsible for the host-to-host delivery of messages. This delivery includes many issues such as the design of logical addresses to uniquely define any host connected to the Internet. It also requires some routing protocols that help the network-layer packets to find their way from the source to the destination. In this chapter, we address all these issues and connect them to make sense of the network layer. The network layer in this chapter is discussed in five sections:

- ❑ In the first section, we introduce the general concepts and issues in the network layer. We first discuss services that can be provided at the network layer, including packetizing, routing, and forwarding. We then introduce packet switching used in the network layer. We also discuss network-layer performance, network-layer congestion, and structures of routers.
- ❑ In the second section, we concentrate on the network layer at the TCP/IP suite and discuss some of the protocols used in version 4, including Internet Protocol version 4 (IPv4) and Internet Control Message Protocol version 4 (ICMPv4). We also discuss IPv4 addressing and related issues.
- ❑ In the third section, we concentrate on the unicast routing and unicast routing protocols. We show how intradomain and interdomain routing protocols are used in the Internet today.
- ❑ In the fourth section, we move to multicasting and multicast routing protocols and show how some intradomain unicast routing protocols can be extended to be used as multicast routing protocols. We also discuss one new independent multicast protocol and briefly introduce the interdomain multicast routing protocol.
- ❑ In the fifth section, we introduce the new generation of network-layer protocols, IPv6 and ICMPv6 and the addressing used in this generation. We put the discussion of these protocols in the last section to allow the reader to skip them without affecting the rest of the material. We believe the full implementation of these protocols is still a few years away.

4.1 INTRODUCTION

Figure 4.1 shows the communication between Alice and Bob at the network layer. This is the same scenario we used in Chapters 2 and 3 to show the communication at the application and the transport layers, respectively.

Figure 4.1 *Communication at the network layer*



The figure shows that the Internet is made of many networks (or links) connected through the connecting devices. In other words, the Internet is an internetwork, a combination of LANs and WANs. To better understand the role of the network layer (or the internetwork layer), we need to think about the connecting devices (routers or switches) that connect the LANs and WANs.

As the figure shows, the network layer is involved at the source host, destination host, and all routers in the path (R2, R4, R5, and R7). At the source host (Alice), the network layer accepts a packet from a transport layer, encapsulates the packet in a datagram, and delivers the packet to the data-link layer. At the destination host (Bob), the datagram is decapsulated, the packet is extracted and delivered to the corresponding transport layer. Although the source and destination hosts are involved in all five layers of the TCP/IP suite, the routers use three layers if they are routing packets only; however, they may need the transport and application layers for control purposes. A router in the path is normally shown with two data-link layers and two physical layers, because it receives a packet from one network and delivers it to another network.

4.1.1 Network-Layer Services

Before discussing the network layer in the Internet today, let's briefly discuss the network-layer services that, in general, are expected from a network-layer protocol.

Packetizing

The first duty of the network layer is definitely **packetizing**: encapsulating the payload (data received from upper layer) in a network-layer packet at the source and decapsulating the payload from the network-layer packet at the destination. In other words, one duty of the network layer is to carry a payload from the source to the destination without changing it or using it. The network layer is doing the service of a carrier such as the postal office, which is responsible for delivery of packages from a sender to a receiver without changing or using the contents.

The source host receives the payload from an upper-layer protocol, adds a header that contains the source and destination addresses and some other information that is required by the network-layer protocol (as discussed later) and delivers the packet to the data-link layer. The source is not allowed to change the content of the payload unless it is too large for delivery and needs to be fragmented.

The destination host receives the network-layer packet from its data-link layer, decapsulates the packet, and delivers the payload to the corresponding upper-layer protocol. If the packet is fragmented at the source or at routers along the path, the network layer is responsible for waiting until all fragments arrive, reassembling them, and delivering them to the upper-layer protocol.

The routers in the path are not allowed to decapsulate the packets they received unless the packets need to be fragmented. The routers are not allowed to change source and destination addresses either. They just inspect the addresses for the purpose of forwarding the packet to the next network on the path. However, if a packet is fragmented, the header needs to be copied to all fragments and some changes are needed, as we discuss in detail later.

Routing

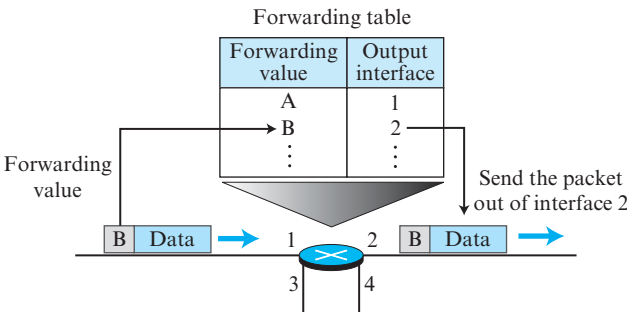
Another duty of the network layer, which is as important as the first, is routing. The network layer is responsible for routing the packet from its source to the destination. A physical network is a combination of networks (LANs and WANs) and routers that connect them. This means that there is more than one route from the source to the destination. The network layer is responsible for finding the *best* one among these possible

routes. The network layer needs to have some specific strategies for defining the best route. In the Internet today, this is done by running some *routing protocols* to help the routers coordinate their knowledge about the neighborhood and to come up with consistent tables to be used when a packet arrives. The routing protocols, which we discuss later in the chapter, should be run before any communication occurs.

Forwarding

If routing is applying strategies and running some routing protocols to create the decision-making tables for each router, *forwarding* can be defined as the action applied by each router when a packet arrives at one of its interfaces. The decision-making table a router normally uses for applying this action is sometimes called the *forwarding table* and sometimes the *routing table*. When a router receives a packet from one of its attached networks, it needs to forward the packet to another attached network (in unicast routing) or to some attached networks (in multicast routing). To make this decision, the router uses a piece of information in the packet header, which can be the destination address or a label, to find the corresponding output interface number in the forwarding table. Figure 4.2 shows the idea of the forwarding process in a router.

Figure 4.2 Forwarding process



Error Control

In Chapter 3, we discussed error control mechanisms for the transport layer. In Chapter 5, we'll discuss error control mechanisms for the data-link layer. Although error control also can be implemented in the network layer, the designers of the network layer in the Internet ignored this issue for the data being carried by the network layer. One reason for this decision is the fact that the packet in the network layer may be fragmented at each router, which makes error checking at this layer inefficient.

The designers of the network layer, however, have added a checksum field to the datagram to control any corruption in the header, but not the whole datagram. This checksum may prevent any changes or corruptions in the header of the datagram between two hops and from end to end.

We need to mention that although the network layer in the Internet does not directly provide error control, the Internet uses an auxiliary protocol, ICMP, that provides some kind of error control if the datagram is discarded or has some unknown information in the header. We discuss ICMP later in the chapter.

Flow Control

Flow control regulates the amount of data a source can send without overwhelming the receiver. If the upper layer at the source computer produces data faster than the upper layer at the destination computer can consume it, the receiver will be overwhelmed with data. To control the flow of data, the receiver needs to send some feedback to the sender to inform the latter that it is overwhelmed with data.

The network layer in the Internet, however, does not directly provide any flow control. The datagrams are sent by the sender when they are ready, without any attention to the readiness of the receiver.

A few reasons for the lack of flow control in the design of the network layer can be mentioned. First, since there is no error control in this layer, the job of the network layer at the receiver is so simple that it may rarely be overwhelmed. Second, the upper layers that use the service of the network layer can implement buffers to receive data from the network layer as they are ready and do not have to consume the data as fast as it is received. Third, flow control is provided for most of the upper-layer protocols that use the services of the network layer, so another level of flow control makes the network layer more complicated and the whole system less efficient.

Congestion Control

Another issue in a network-layer protocol is congestion control. Congestion in the network layer is a situation in which too many datagrams are present in an area of the Internet. Congestion may occur if the number of datagrams sent by source computers is beyond the capacity of the network or routers. In this situation, some routers may drop some of the datagrams. However, as more datagrams are dropped, the situation may become worse because, due to the error control mechanism at the upper layers, the sender may send duplicates of the lost packets. If the congestion continues, sometimes a situation may reach a point where the system collapses and no datagrams are delivered. We discuss congestion control at the network layer later in the chapter although it is not implemented in the Internet.

Quality of Service

As the Internet has allowed new applications such as multimedia communication (in particular real-time communication of audio and video), the quality of service (QoS) of the communication has become more and more important. The Internet has thrived by providing better quality of service to support these applications. However, to keep the network layer untouched, these provisions are mostly implemented in the upper layer. Since QoS manifests itself more when we use multimedia communication, we discuss this issue in Chapter 8.

Security

Another issue related to communication at the network layer is security. Security was not a concern when the Internet was originally designed because it was used by a small number of users at universities for research activities; other people had no access to the Internet. The network layer was designed with no security provision. Today, however, security is a big concern. To provide security for a connectionless network layer, we need to have another virtual level that changes the connectionless

service to a connection-oriented service. This virtual layer, called IPSec, is discussed in Chapter 10.

4.1.2 Packet Switching

From the discussion of routing and forwarding in the previous section, we infer that a kind of *switching* occurs at the network layer. A router, in fact, is a switch that creates a connection between an input port and an output port (or a set of output ports), just as an electrical switch connects the input to the output to let electricity flow.

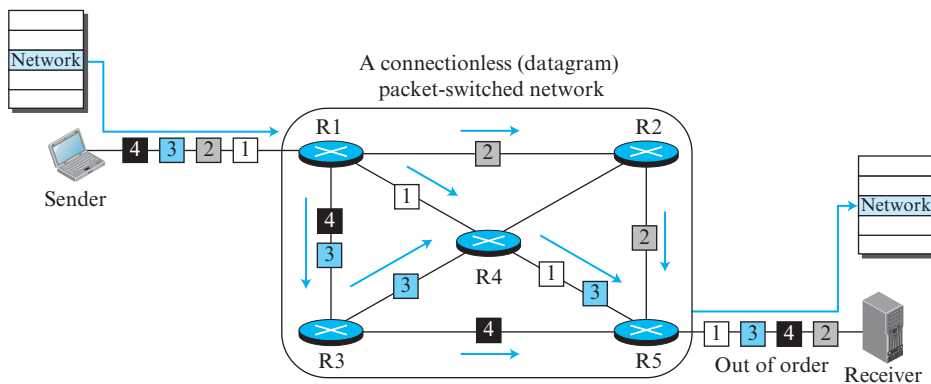
Although in data communication switching techniques are divided into two broad categories, circuit switching and packet switching, only packet switching is used at the network layer because the unit of data at this layer is a packet. Circuit switching is mostly used at the physical layer; the electrical switch mentioned earlier is a kind of circuit switch.

At the network layer, a message from the upper layer is divided into manageable packets and each packet is sent through the network. The source of the message sends the packets one by one; the destination of the message receives the packets one by one. The destination waits for all packets belonging to the same message to arrive before delivering the message to the upper layer. The connecting devices in a packet-switched network still need to decide how to route the packets to the final destination. Today, a packet-switched network can use two different approaches to route the packets: the *datagram approach* and the *virtual circuit approach*. We discuss both approaches in the next section.

Datagram Approach: Connectionless Service

When the Internet started, to make it simple, the network layer was designed to provide a connectionless service in which the network-layer protocol treats each packet independently, with each packet having no relationship to any other packet. The idea was that the network layer is only responsible for delivery of packets from the source to the destination. In this approach, the packets in a message may or may not travel the same path to their destination. Figure 4.3 shows the idea.

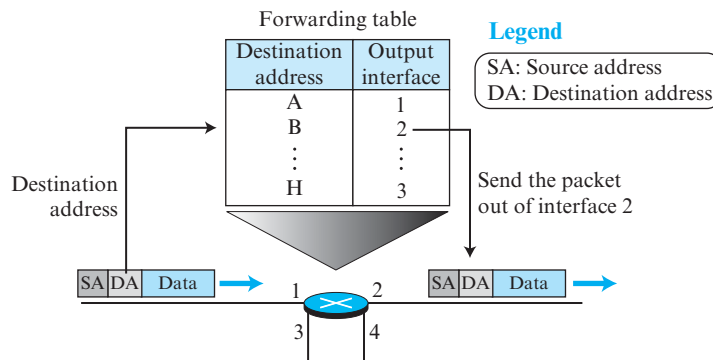
Figure 4.3 A connectionless packet-switched network



When the network layer provides a connectionless service, each packet traveling in the Internet is an independent entity; there is no relationship between packets belonging to the same message. The switches in this type of network are called routers. A packet belonging to a message may be followed by a packet belonging to the same message or to a different message. A packet may be followed by a packet coming from the same or from a different source.

Each packet is routed based on the information contained in its header: source and destination addresses. The destination address defines where it should go; the source address defines where it comes from. The router in this case routes the packet based only on the destination address. The source address may be used to send an error message to the source if the packet is discarded. Figure 4.4 shows the forwarding process in a router in this case. We have used symbolic addresses such as A and B.

Figure 4.4 Forwarding process in a router when used in a connectionless network

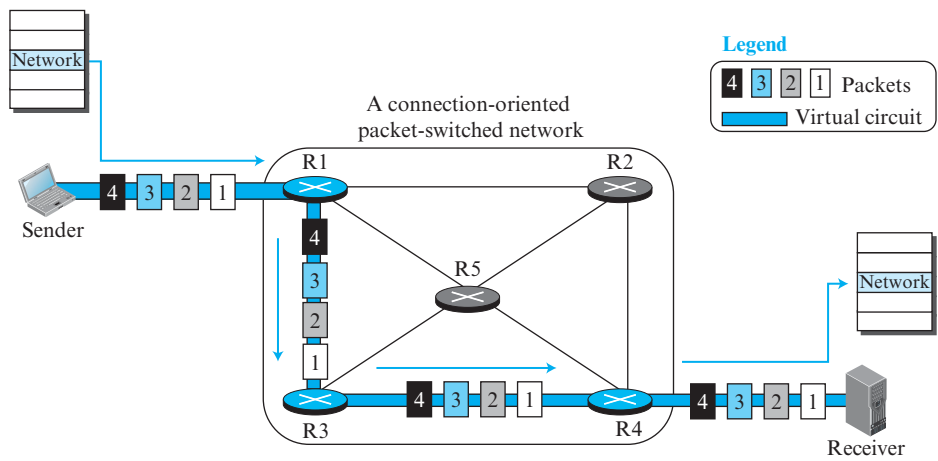


In the datagram approach, the forwarding decision is based on the destination address of the packet.

Virtual-Circuit Approach: Connection-Oriented Service

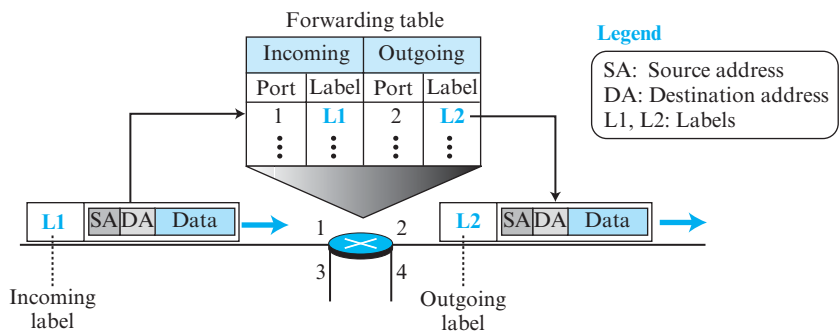
In a connection-oriented service (also called *virtual-circuit approach*), there is a relationship between all packets belonging to a message. Before all datagrams in a message can be sent, a virtual connection should be set up to define the path for the datagrams. After connection setup, the datagrams can all follow the same path. In this type of service, not only must the packet contain the source and destination addresses, it must also contain a flow label, a virtual circuit identifier that defines the virtual path the packet should follow. Shortly, we will show how this flow label is determined, but for the moment, we assume that the packet carries this label. Although it looks as though the use of the label may make the source and destination addresses unnecessary during the data transfer phase, parts of the Internet at the network layer still keep these addresses. One reason is that part of the packet path may still be using the connectionless service. Another reason is that the protocol at the network layer is designed with these addresses, and it may take a while before they can be changed. Figure 4.5 shows the concept of connection-oriented service.

Figure 4.5 A virtual-circuit packet-switched network



Each packet is forwarded based on the label in the packet. To follow the idea of connection-oriented design to be used in the Internet, we assume that the packet has a label when it reaches the router. Figure 4.6 shows the idea. In this case, the forwarding decision is based on the value of the label, or virtual circuit identifier as it is sometimes called.

Figure 4.6 Forwarding process in a router when used in a virtual-circuit network



To create a connection-oriented service, a three-phase process is used: setup, data transfer, and teardown. In the setup phase, the source and destination addresses of the sender and receiver are used to make table entries for the connection-oriented service. In the teardown phase, the source and destination inform the router to delete the corresponding entries. Data transfer occurs between these two phases.

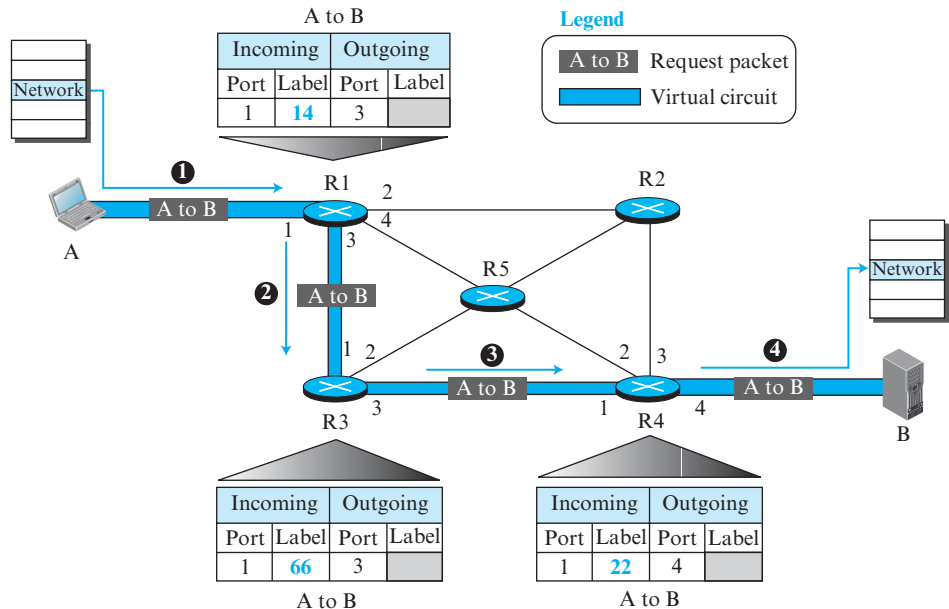
In the virtual-circuit approach, the forwarding decision is based on the label of the packet.

Setup Phase

In the setup phase, a router creates an entry for a virtual circuit. For example, suppose source A needs to create a virtual circuit to destination B. Two auxiliary packets need to be exchanged between the sender and the receiver: the request packet and the acknowledgment packet.

Request packet A request packet is sent from the source to the destination. This auxiliary packet carries the source and destination addresses. Figure 4.7 shows the process.

Figure 4.7 Sending request packet in a virtual-circuit network

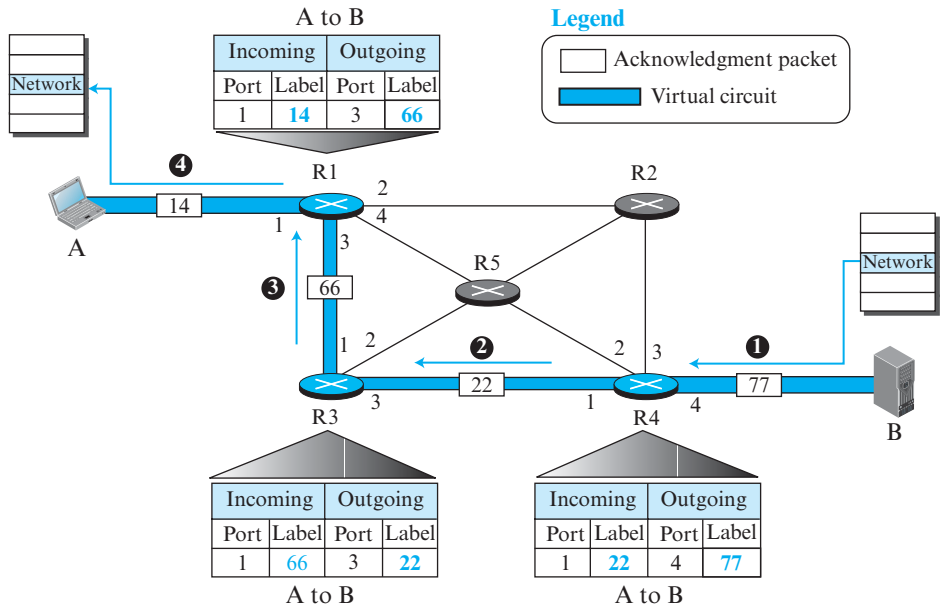


1. Source A sends a request packet to router R1.
2. Router R1 receives the request packet. It knows that a packet going from A to B goes out through port 3. How the router has obtained this information is a point covered later. For the moment, assume that it knows the output port. The router creates an entry in its table for this virtual circuit, but it is only able to fill three of the four columns. The router assigns the incoming port (1) and chooses an available incoming label (14) and the outgoing port (3). It does not yet know the outgoing label, which will be found during the acknowledgment step. The router then forwards the packet through port 3 to router R3.
3. Router R3 receives the setup request packet. The same events happen here as at router R1; three columns of the table are completed: in this case, incoming port (1), incoming label (66), and outgoing port (3).
4. Router R4 receives the setup request packet. Again, three columns are completed: incoming port (1), incoming label (22), and outgoing port (4).

5. Destination B receives the setup packet, and if it is ready to receive packets from A, it assigns a label to the incoming packets that come from A, in this case 77, as shown in Figure 4.8. This label lets the destination know that the packets come from A, and not from other sources.

Acknowledgment Packet A special packet, called the acknowledgment packet, completes the entries in the switching tables. Figure 4.8 shows the process.

Figure 4.8 Sending acknowledgments in a virtual-circuit network

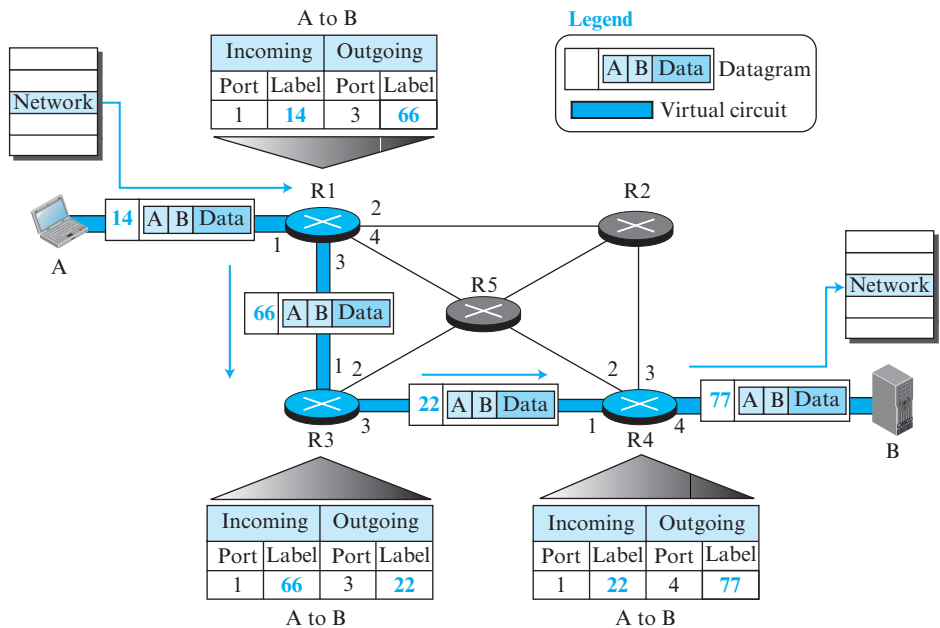


1. The destination sends an acknowledgment to router R4. The acknowledgment carries the global source and destination addresses so the router knows which entry in the table is to be completed. The packet also carries label 77, chosen by the destination as the incoming label for packets from A. Router R4 uses this label to complete the outgoing label column for this entry. Note that 77 is the incoming label for destination B, but the outgoing label for router R4.
2. Router R4 sends an acknowledgment to router R3 that contains its incoming label in the table, chosen in the setup phase. Router R3 uses this as the outgoing label in the table.
3. Router R3 sends an acknowledgment to router R1 that contains its incoming label in the table, chosen in the setup phase. Router R1 uses this as the outgoing label in the table.
4. Finally router R1 sends an acknowledgment to source A that contains its incoming label in the table, chosen in the setup phase.
5. The source uses this as the outgoing label for the data packets to be sent to destination B.

Data-Transfer Phase

The second phase is called the data-transfer phase. After all routers have created their forwarding table for a specific virtual circuit, then the network-layer packets belonging to one message can be sent one after another. In Figure 4.9, we show the flow of one single packet, but the process is the same for 1, 2, or 100 packets. The source computer uses the label 14, which it has received from router R1 in the setup phase. Router R1 forwards the packet to router R3, but changes the label to 66. Router R3 forwards the packet to router R4, but changes the label to 22. Finally, router R4 delivers the packet to its final destination with the label 77. All the packets in the message follow the same sequence of labels, and the packets arrive in order at the destination.

Figure 4.9 Flow of one packet in an established virtual circuit



Teardown Phase

In the teardown phase, source A, after sending all packets to B, sends a special packet called a teardown packet. Destination B responds with a confirmation packet. All routers delete the corresponding entries from their tables.

4.1.3 Network-Layer Performance

The upper-layer protocols that use the service of the network layer expect to receive an ideal service, but the network layer is not perfect. The performance of a network can be measured in terms of *delay*, *throughput*, and *packet loss*. We first define these three terms in a packet-switched network before we discuss their effects on performance.

Delay

All of us expect instantaneous response from a network, but a packet, from its source to its destination, encounters delays. The delays in a network can be divided into four types: transmission delay, propagation delay, processing delay, and queuing delay. Let us first discuss all of these delay types and then show how to calculate a packet delay from the source to the destination.

Transmission Delay

A source host or a router cannot send a packet instantaneously. A sender needs to put the bits in a packet on the line one by one. If the first bit of the packet is put on the line at time t_1 and the last bit is put on the line at time t_2 , transmission delay of the packet is $(t_2 - t_1)$. Definitely, the transmission delay is longer for a longer packet and shorter if the sender can transmit faster. In other words, the transmission delay is

$$\text{Delay}_{\text{tr}} = (\text{Packet length}) / (\text{Transmission rate}).$$

For example, in a Fast Ethernet LAN (see Chapter 5) with the transmission rate of 100 million bits per second and a packet of 10,000 bits, it takes $(10,000)/(100,000,000)$ or 100 microseconds for all bits of the packet to be put on the line.

Propagation Delay

Propagation delay is the time it takes for a bit to travel from point A to point B in the transmission media. The propagation delay for a packet-switched network depends on the propagation delay of each network (LAN or WAN). The propagation delay depends on the propagation speed of the media, which is 3×10^8 meters/second in a vacuum and normally much less in a wired medium; it also depends on the distance of the link. In other words, propagation delay is

$$\text{Delay}_{\text{pg}} = (\text{Distance}) / (\text{Propagation speed}).$$

For example, if the distance of a cable link in a point-to-point WAN is 2000 meters and the propagation speed of the bits in the cable is 2×10^8 meters/second, then the propagation delay is 10 microseconds.

Processing Delay

The processing delay is the time required for a router or a destination host to receive a packet from its input port, remove the header, perform an error detection procedure, and deliver the packet to the output port (in the case of a router) or deliver the packet to the upper-layer protocol (in the case of the destination host). The processing delay may be different for each packet, but normally is calculated as an average.

$$\text{Delay}_{\text{pr}} = \text{Time required to process a packet in a router or a destination host}$$

Queuing Delay

Queuing delay can normally happen in a router. As we discuss in the next section, a router has an input queue connected to each of its input ports to store packets waiting to be processed; the router also has an output queue connected to each of its output ports to store packets waiting to be transmitted. The queuing delay for a packet in a router is measured as the time a packet waits in the input queue and output queue of a router. We can compare the situation with a busy airport. Some planes may need to wait to get the

landing band (input delay); some planes may need to wait to get the departure band (output delay).

Delay_{qu} = The time a packet waits in input and output queues in a router

Total Delay

Assuming equal delays for the sender, routers, and receiver, the total delay (source-to-destination delay) a packet encounters can be calculated if we know the number of routers, n , in the whole path.

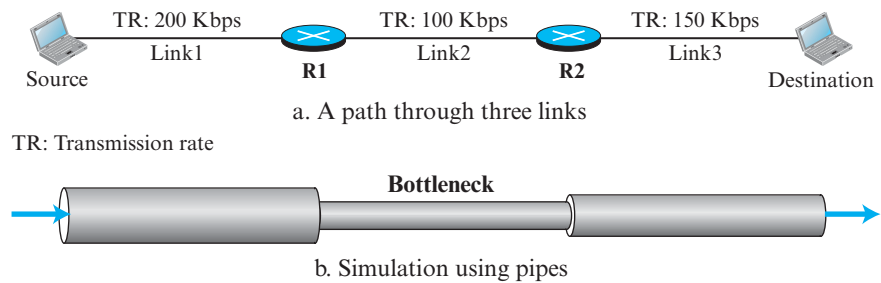
$$\text{Total delay} = (n + 1) (\text{Delay}_{\text{tr}} + \text{Delay}_{\text{pg}} + \text{Delay}_{\text{pr}}) + (n) (\text{Delay}_{\text{qu}})$$

Note that if we have n routers, we have $(n + 1)$ links. Therefore, we have $(n + 1)$ transmission delays related to n routers and the source, $(n + 1)$ propagation delays related to $(n + 1)$ links, $(n + 1)$ processing delays related to n routers and the destination, and only n queuing delays related to n routers.

Throughput

Throughput at any point in a network is defined as the number of bits passing through the point in a second, which is actually the transmission rate of data at that point. In a path from source to destination, a packet may pass through several links (networks), each with a different transmission rate. How, then, can we determine the throughput of the whole path? To see the situation, assume that we have three links, each with a different transmission rate, as shown in Figure 4.10.

Figure 4.10 Throughput in a path with three links in a series

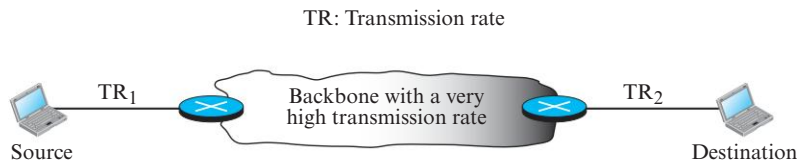


In this figure, the data can flow at the rate of 200 Kbps in Link1. However, when the data arrives at router R1, it cannot pass at this rate. Data needs to be queued at the router and sent at 100 Kbps. When data arrives at router R2, it could be sent at the rate of 150 Kbps, but there is not enough data to be sent. In other words, the average rate of the data flow in Link3 is also 100 Kbps. We can conclude that the average data rate for this path is 100 Kbps, the minimum of the three different data rates. The figure also shows that we can simulate the behavior of each link with pipes of different sizes; the average throughput is determined by the bottleneck, the pipe with the smallest diameter. In general, in a path with n links in series, we have

$$\text{Throughput} = \text{minimum} \{TR_1, TR_2, \dots, TR_n\}.$$

Although the situation in Figure 4.10 shows how to calculate the throughput when the data is passed through several links, the actual situation in the Internet is that the data normally passes through two access networks and the Internet backbone, as shown in Figure 4.11.

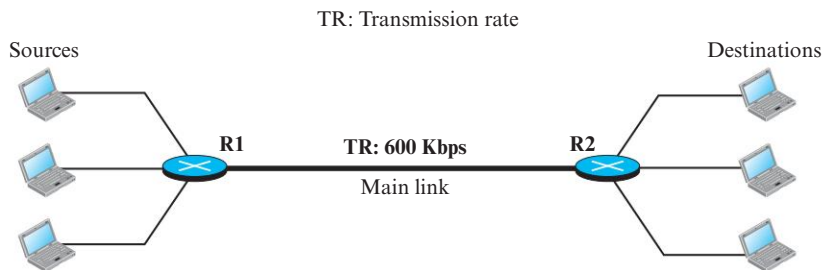
Figure 4.11 *A path through the Internet backbone*



The Internet backbone has a very high transmission rate, in the range of gigabits per second. This means that the throughput is normally defined as the minimum transmission rate of the two access links that connect the source and destination to the backbone. Figure 4.11 shows this situation, in which the throughput is the minimum of TR_1 and TR_2 . For example, if a server connected to the Internet via a Fast Ethernet LAN with the data rate of 100 Mbps, but a user who wants to download a file connected to the Internet via a dial-up telephone line with the data rate of 40 Kbps, the throughput is 40 Kbps. The bottleneck is definitely the dial-up line.

We need to mention another situation in which we think about the throughput. The link between two routers is not always dedicated to one flow. A router may collect the flow from several sources or distribute the flow between several sources. In this case the transmission rate of the link between the two routers is actually shared between the flows and this should be considered when we calculate the throughput. For example, in Figure 4.12 the transmission rate of the main link in the calculation of the throughput is only 200 Kbps because the link is shared between three paths.

Figure 4.12 *Effect of throughput in shared links*



Packet Loss

Another issue that severely affects the performance of communication is the number of packets lost during transmission. When a router receives a packet while processing another packet, the received packet needs to be stored in the input buffer waiting for its

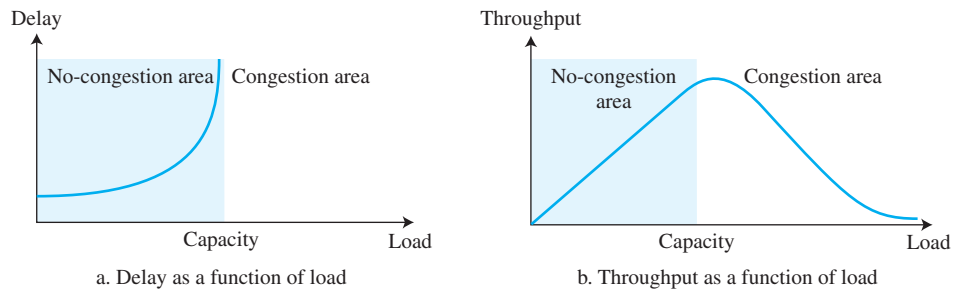
turn. A router, however, has an input buffer with a limited size. A time may come when the buffer is full and the next packet needs to be dropped. The effect of packet loss on the Internet network layer is that the packet needs to be resent, which in turn may create overflow and cause more packet loss. A lot of theoretical studies have been done in queuing theory to prevent the overflow of queues and prevent packet loss.

4.1.4 Network-Layer Congestion

In Chapter 3, we discussed congestion at the transport layer. Although congestion at the network layer is not explicitly addressed in the Internet model, the study of congestion at this layer may help us to better understand the cause of congestion at the transport layer and find possible remedies to be used at the network layer. Congestion at the network layer is related to two issues, throughput and delay, which we discussed in the previous section.

Figure 4.13 shows these two performance measures as functions of load.

Figure 4.13 Packet delay and throughput as functions of load



When the load is much less than the capacity of the network, the **delay** is at a minimum. This minimum delay is composed of propagation delay and processing delay, both of which are negligible. However, when the load reaches the network capacity, the delay increases sharply because we now need to add the queuing delay to the total delay. Note that the delay becomes infinite when the load is greater than the capacity.

When the load is below the capacity of the network, the throughput increases proportionally with the *load*. We expect the throughput to remain constant after the load reaches the capacity, but instead the throughput declines sharply. The reason is the discarding of packets by the routers. When the load exceeds the capacity, the queues become full and the routers have to discard some packets. Discarding packets does not reduce the number of packets in the network because the sources retransmit the packets, using time-out mechanisms, when the packets do not reach the destinations.

Congestion Control

As we discussed in Chapter 3, congestion control refers to techniques and mechanisms that can either prevent congestion before it happens or remove congestion after it has happened. In general, we can divide congestion control mechanisms into two broad categories: **open-loop congestion control** (prevention) and **closed-loop congestion control** (removal).

Open-Loop Congestion Control

In open-loop congestion control, policies are applied to prevent congestion before it happens. In these mechanisms, congestion control is handled by either the source or the destination. We give a brief list of policies that can prevent congestion.

Retransmission Policy Retransmission is sometimes unavoidable. If the sender feels that a sent packet is lost or corrupted, the packet needs to be retransmitted. Retransmission in general may increase congestion in the network. However, a good retransmission policy can prevent congestion. The retransmission policy and the retransmission timers must be designed to optimize efficiency and at the same time prevent congestion.

Window Policy The type of window at the sender may also affect congestion. The Selective Repeat window is better than the Go-Back- N window for congestion control. In the Go-Back- N window, when the timer for a packet times out, several packets may be resent, although some may have arrived safe and sound at the receiver. This duplication may make the congestion worse. The Selective Repeat window, on the other hand, tries to send the specific packets that have been lost or corrupted.

Acknowledgment Policy The acknowledgment policy imposed by the receiver may also affect congestion. If the receiver does not acknowledge every packet it receives, it may slow down the sender and help prevent congestion. Several approaches are used in this case. A receiver may send an acknowledgment only if it has a packet to be sent or a special timer expires. A receiver may decide to acknowledge only N packets at a time. We need to know that the acknowledgments are also part of the load in a network. Sending fewer acknowledgments means imposing less load on the network.

Discarding Policy A good discarding policy by the routers may prevent congestion and at the same time may not harm the integrity of the transmission. For example, in audio transmission, if the policy is to discard less sensitive packets when congestion is likely to happen, the quality of sound is still preserved and congestion is prevented or alleviated.

Admission Policy An admission policy, which is a quality-of-service mechanism (discussed in Chapter 8), can also prevent congestion in virtual-circuit networks. Switches in a flow first check the resource requirement of a flow before admitting it to the network. A router can deny establishing a virtual-circuit connection if there is congestion in the network or if there is a possibility of future congestion.

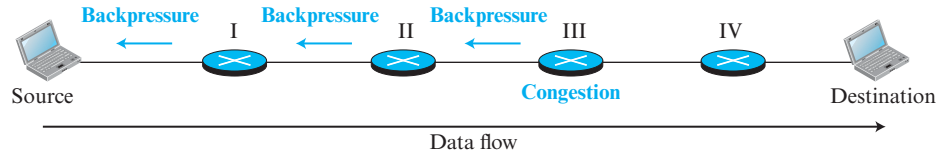
Closed-Loop Congestion Control

Closed-loop congestion control mechanisms try to alleviate congestion after it happens. Several mechanisms have been used by different protocols. We describe a few of them here.

Backpressure The technique of *backpressure* refers to a congestion control mechanism in which a congested node stops receiving data from the immediate upstream node or nodes. This may cause the upstream node or nodes to become congested, and they, in turn, reject data from their upstream node or nodes, and so on. Backpressure is a node-to-node congestion control that starts with a node and propagates, in the opposite direction of data flow, to the source. The backpressure technique can be applied only to virtual

circuit networks, in which each node knows the upstream node from which a flow of data is coming. Figure 4.14 shows the idea of backpressure.

Figure 4.14 Backpressure method for alleviating congestion

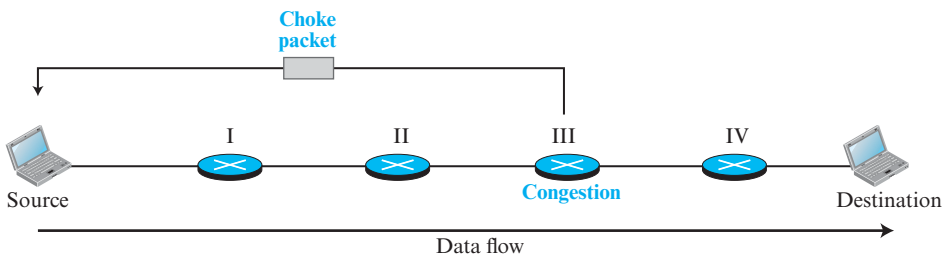


Node III in the figure has more input data than it can handle. It drops some packets in its input buffer and informs node II to slow down. Node II, in turn, may be congested because it is slowing down the output flow of data. If node II is congested, it informs node I to slow down, which in turn may create congestion. If so, node I informs the source of data to slow down. This, in time, alleviates the congestion. Note that the *pressure* on node III is moved backward to the source to remove the congestion.

It is important to mention that this type of congestion control can only be implemented in virtual-circuit. The technique cannot be implemented in a datagram network, in which a node (router) does not have the slightest knowledge of the upstream router.

Choke Packet A **choke packet** is a packet sent by a node to the source to inform it of congestion. Note the difference between the backpressure and choke-packet methods. In backpressure, the warning is from one node to its upstream node, although the warning may eventually reach the source station. In the choke-packet method, the warning is from the router, which has encountered congestion, directly to the source station. The intermediate nodes through which the packet has traveled are not warned. We will see an example of this type of control in ICMP (discussed later in the chapter). When a router in the Internet is overwhelmed with IP datagrams, it may discard some of them, but it informs the source host, using a source quench ICMP message. The warning message goes directly to the source station; the intermediate routers do not take any action. Figure 4.15 shows the idea of a choke packet.

Figure 4.15 Choke packet



Implicit Signaling In implicit signaling, there is no communication between the congested node or nodes and the source. The source guesses that there is congestion

somewhere in the network from other symptoms. For example, when a source sends several packets and there is no acknowledgment for a while, one assumption is that the network is congested. The delay in receiving an acknowledgment is interpreted as congestion in the network; the source should slow down. We saw this type of signaling when we discussed TCP congestion control in Chapter 3.

Explicit Signaling The node that experiences congestion can explicitly send a signal to the source or destination. The explicit-signaling method, however, is different from the choke-packet method. In the choke-packet method, a separate packet is used for this purpose; in the explicit-signaling method, the signal is included in the packets that carry data. Explicit signaling can occur in either the forward or the backward direction. This type of congestion control can be seen in an ATM network, discussed in Chapter 5.

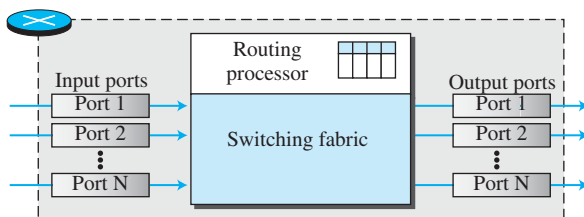
4.1.5 Structure of A Router

In our discussion of forwarding and routing, we represented a router as a black box that accepts incoming packets from one of the input ports (interfaces), uses a forwarding table to find the output port from which the packet departs, and sends the packet from this output port. In this section we open the black box and look inside. However, our discussion won't be very detailed; entire books have been written about routers. We just give an overview to the reader.

Components

We can say that a router has four components: *input ports*, *output ports*, the *routing processor*, and the *switching fabric*, as shown in Figure 4.16.

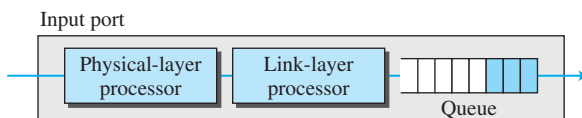
Figure 4.16 Router components



Input Ports

Figure 4.17 shows a schematic diagram of an input port.

Figure 4.17 Input port



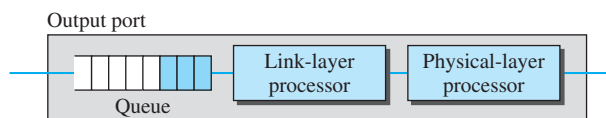
An input port performs the physical- and link-layer functions of the router. The bits are constructed from the received signal. The packet is decapsulated from the frame,

checked for errors, and discarded if corrupted. The packet is then ready to be processed by the network layer. In addition to a physical-layer processor and a link-layer processor, the input port has buffers (queues) to hold the packets before they are directed to the switching fabric.

Output Ports

An output port performs the same functions as an input port, but in the reverse order. First the outgoing packets are queued, then each packet is encapsulated in a frame, and finally the physical-layer functions are applied to the frame to create the signal to be sent on the line. Figure 4.18 shows a schematic diagram of an output port.

Figure 4.18 Output port



Routing Processor

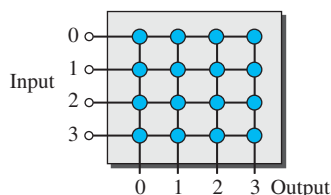
The routing processor performs the functions of the network layer. The destination address is used to find the address of the next hop and, at the same time, the output port number from which the packet is sent out. This activity is sometimes referred to as *table lookup* because the routing processor searches the forwarding table. In the newer routers, this function of the routing processor is being moved to the input ports to facilitate and expedite the process.

Switching Fabrics

The most difficult task in a router is to move the packet from the input queue to the output queue. The speed with which this is done affects the size of the input/output queue and the overall delay in packet delivery. In the past, when a router was actually a dedicated computer, the memory of the computer or a bus was used as the switching fabric. The input port stored the packet in memory; the output port got the packet from the memory. Today, routers use a variety of switching fabrics. We briefly discuss some of these fabrics here.

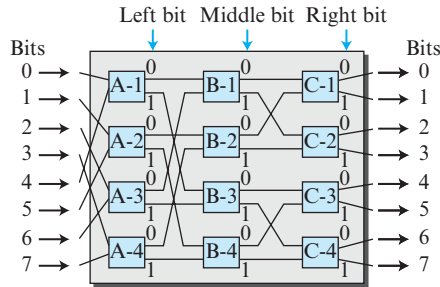
Crossbar Switch The simplest type of switching fabric is the crossbar switch, shown in Figure 4.19. A **crossbar switch** connects n inputs to n outputs in a grid, using electronic microswitches at each **crosspoint**.

Figure 4.19 Crossbar switch



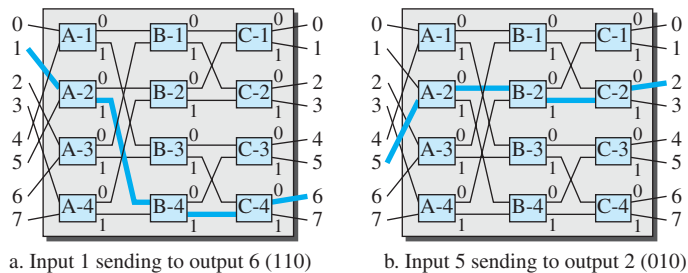
Banyan Switch More realistic than the crossbar switch is the **banyan switch** (named after the banyan tree) as shown in Figure 4.20.

Figure 4.20 Banyan switch



A banyan switch is a multistage switch with microswitches at each stage that route the packets based on the output port represented as a binary string. For n inputs and n outputs, we have $\log_2(n)$ stages with $n/2$ microswitches at each stage. The first stage routes the packet based on the highest order bit of the binary string. The second stage routes the packets based on the second-highest-order bit, and so on. Figure 4.21 shows a banyan switch with eight inputs and eight outputs. The number of stages is $\log_2(8) = 3$. Suppose a packet has arrived at input port 1 and must go to output port 6 (110 in binary). The first microswitch (A-2) routes the packet based on the first bit (1); the second microswitch (B-4) routes the packet based on the second bit (1); the third microswitch (C-4) routes the packet based on the third bit (0). In part b, a packet has arrived at input port 5 and must go to output port 2 (010 in binary). The first microswitch (A-2) routes the packet based on the first bit (0); the second microswitch (B-2) routes the packet based on the second bit (1); the third microswitch (C-2) routes the packet based on the third bit (0).

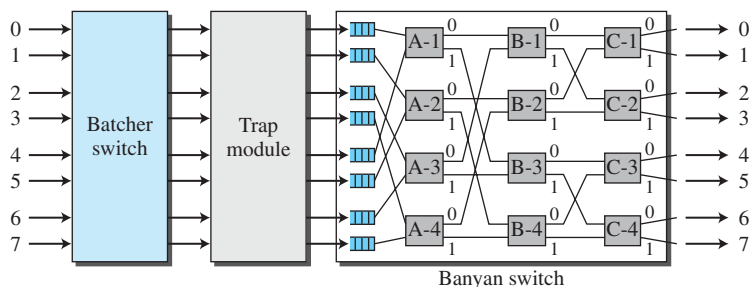
Figure 4.21 Examples of routing in a banyan switch



Batcher-Banyan Switch The problem with the banyan switch is the possibility of internal collision even when two packets are not heading for the same output port. We can solve this problem by sorting the arriving packets based on their destination port.

K. E. Batcher designed a switch that comes before the banyan switch and sorts the incoming packets according to their final destination. The combination is called the **Batcher-banyan switch** (see Figure 4.22).

Figure 4.22 Batcher-banyan switch



The sorting switch uses hardware merging techniques, but we will not discuss the details here. Normally, another hardware module, called a *trap*, is added between the Batcher switch and the banyan switch. The trap module prevents duplicate packets (the ones with the same output destination) from passing to the banyan switch simultaneously. Only one packet for each destination is allowed at each tick; if there is more than one, they wait for the next tick.

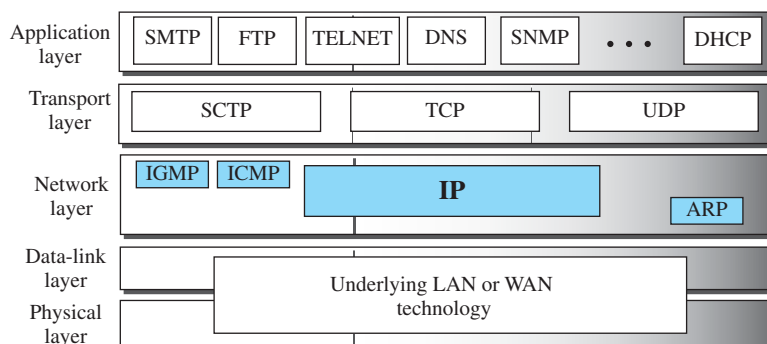
4.2 NETWORK-LAYER PROTOCOLS

In the first section of this chapter, we discuss general issues and services related to the network layer. In this section, we show how the network layer is implemented in the TCP/IP protocol suite. The protocols in the network layer have gone through several versions; in this section, we concentrate on the current version (4), in the last section of this chapter, we briefly discuss version 6, which is on the horizon, but not completely implemented yet.

The network layer in version 4 can be thought of as one main protocol and three auxiliary protocols. The main protocol, Internet Protocol version 4 (IPv4), is responsible for packetizing, forwarding, and delivery of a packet at the network layer. The Internet Control Message Protocol version 4 (ICMPv4) helps IPv4 to handle some errors that may occur in the network-layer delivery. The Internet Group Management Protocol (IGMP) is used to help IPv4 in multicasting. The Address Resolution Protocol (ARP) is used to glue the network and data-link layers in mapping network-layer addresses to link-layer addresses. Figure 4.23 shows the positions of these four protocols in the TCP/IP protocol suite.

We discuss IPv4 and ICMPv4 in this section. IGMP will be discussed when we talk about routing protocols. We postpone the discussion of ARP to Chapter 5 when we talk about link-layer addresses.

IPv4 is an unreliable and connectionless datagram protocol—a best-effort delivery service. The term *best-effort* means that IPv4 packets can be corrupted, lost, arrive out

Figure 4.23 Position of IP and other network-layer protocols in TCP/IP protocol suite

of order, or delayed, and may create congestion for the network. If reliability is important, IPv4 must be paired with a reliable transport-layer protocol such as TCP. An example of a more commonly understood best-effort delivery service is the post office. The post office does its best to deliver the regular mail but does not always succeed. If an unregistered letter is lost, it is up to the sender or would-be recipient to discover the loss and rectify the problem. The post office itself does not keep track of every letter and cannot notify a sender of loss or damage.

IPv4 is also a connectionless protocol for a packet-switched network that uses the datagram approach. This means that each datagram is handled independently, and each datagram can follow a different route to the destination. This implies that datagrams sent by the same source to the same destination could arrive out of order. Also, some could be lost or corrupted during transmission. Again, IPv4 relies on a higher-level protocol to take care of all these problems.

In this section, we begin by discussing the first service provided by IPv4, packetizing. We show how IPv4 defines the format of a packet in which the data coming from the upper layer or other protocols are encapsulated. We then discuss the IPv4 addresses. Finally, we discuss the routing and forwarding of IPv4 packets and the ICMP.

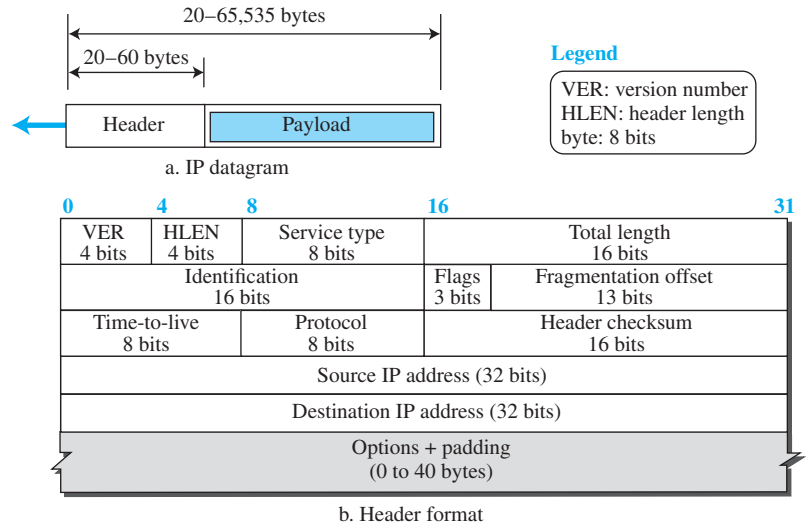
4.2.1 IPv4 Datagram Format

Packets used by the IP are called *datagrams*. Figure 4.24 shows the IPv4 datagram format. A datagram is a variable-length packet consisting of two parts: header and payload (data). The header is 20 to 60 bytes in length and contains information essential to routing and delivery. It is customary in TCP/IP to show the header in 4-byte sections.

Discussing the meaning and rationale for the existence of each field is essential to understanding the operation of IPv4; a brief description of each field is in order.

- ❑ **Version Number.** The 4-bit version number (VER) field defines the version of the IPv4 protocol, which, obviously, has the value of 4.
- ❑ **Header Length.** The 4-bit header length (HLEN) field defines the total length of the datagram header in 4-byte words. The IPv4 datagram has a variable-length header. When a device receives a datagram, it needs to know when the header stops and the data, which is encapsulated in the packet, starts. However, to make the

Figure 4.24 IP datagram

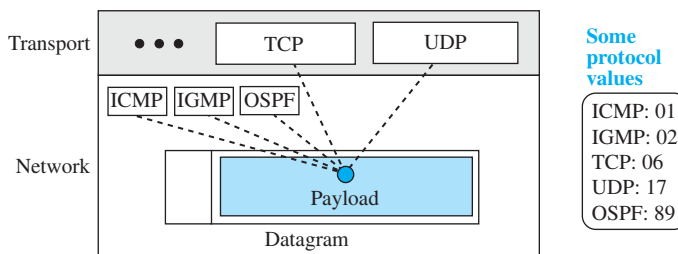


value of the header length (number of bytes) fit in a 4-bit header length, the total length of the header is calculated as 4-byte words. The total length is divided by 4 and the value is inserted in the field. The receiver needs to multiply the value of this field by 4 to find the total length.

- ❑ **Service Type.** In the original design of the IP header, this field was referred to as type of service (TOS), which defined how the datagram should be handled. In the late 1990s, IETF redefined the field to provide *differentiated services* (DiffServ). When we discuss differentiated services in Chapter 8, we will be in a better situation to define the bits in this field.
- ❑ **Total Length.** This 16-bit field defines the total length (header plus data) of the IP datagram in bytes. A 16-bit number can define a total length of up to 65,535 (when all bits are 1s). However, the size of the datagram is normally much less than this. This field helps the receiving device to know when the packet has completely arrived.
- ❑ **Identification, Flags, and Fragmentation Offset.** These three fields are related to the fragmentation of the IP datagram when the size of the datagram is larger than the underlying network can carry. We discuss the contents and importance of these fields when we talk about fragmentation in the next section.
- ❑ **Time-to-live.** Due to some malfunctioning of routing protocols (discussed later) a datagram may be circulating in the Internet, visiting some networks over and over without reaching the destination. This may create extra traffic in the Internet. The time-to-live (TTL) field is used to control the maximum number of hops (routers) visited by the datagram. When a source host sends the datagram, it stores a number in this field. This value is approximately two times the maximum number of routes between any two hosts. Each router that processes the datagram decrements this number by one. If this value, after being decremented, is zero, the router discards the datagram.

- **Protocol.** In TCP/IP, the data section of a packet, called the *payload*, carries the whole packet from another protocol. A datagram, for example, can carry a packet belonging to any transport-layer protocol such as UDP or TCP. A datagram can also carry a packet from other protocols that directly use the service of the IP, such as some routing protocols or some auxiliary protocols. The Internet authority has given any protocol that uses the service of IP a unique 8-bit number which is inserted in the protocol field. When the payload is encapsulated in a datagram at the source IP, the corresponding protocol number is inserted in this field; when the datagram arrives at the destination, the value of this field helps to define to which protocol the payload should be delivered. In other words, this field provides multiplexing at the source and demultiplexing at the destination, as shown in Figure 4.25. Note that the protocol fields at the network layer play the same role as the port numbers at the transport layer. However, we need two port numbers in a transport-layer packet because the port numbers at the source and destination are different, but we need only one protocol field because this value is the same for each protocol no matter whether it is located at the source or the destination.

Figure 4.25 Multiplexing and demultiplexing using the value of the protocol field



- **Header checksum.** IP is not a reliable protocol; it does not check whether the payload carried by a datagram is corrupted during the transmission. IP puts the burden of error checking of the payload on the protocol that owns the payload, such as UDP or TCP. The datagram header, however, is added by IP, and its error-checking is the responsibility of IP. Errors in the IP header can be a disaster. For example, if the destination IP address is corrupted, the packet can be delivered to the wrong host. If the protocol field is corrupted, the payload may be delivered to the wrong protocol. If the fields related to the fragmentation are corrupted, the datagram cannot be reassembled correctly at the destination, and so on. For these reasons, IP adds a header checksum field to check the header, but not the payload. We need to remember that, since the value of some fields, such as TTL, which are related to fragmentation and options, may change from router to router, the checksum needs to be recalculated at each router. As we discussed in Chapter 3, checksum in the Internet normally uses a 16-bit field, which is the complement of the sum of other fields calculated using the 1s complement arithmetic. We discuss checksum calculation in Chapter 5 when we talk about error detection.
- **Source and Destination Addresses.** These 32-bit source and destination address fields define the IP address of the source and destination respectively. The source

host should know its IP address. The destination IP address is either known by the protocol that uses the service of IP or is provided by the DNS as described in Chapter 2. Note that the value of these fields must remain unchanged during the time the IP datagram travels from the source host to the destination host. We discuss more about IP addresses and their structure later in the chapter.

- ❑ **Options.** A datagram header can have up to 40 bytes of options. Options can be used for network testing and debugging. Although options are not a required part of the IP header, option processing is required of the IP software. This means that all implementations must be able to handle options if they are present in the header. The existence of options in a header creates some burden on the datagram handling; some options can be changed by routers, which forces each router to recalculate the header checksum. There are one-byte and multi-byte options that we discuss in more detail on the book website in extra materials for Chapter 4.
- ❑ **Payload.** Payload, or data, is the main reason for creating a datagram. Payload is the packet coming from other protocols that use the service of IP. Comparing a datagram to a postal package, payload is the content of the package; the header is only the information written on the package.

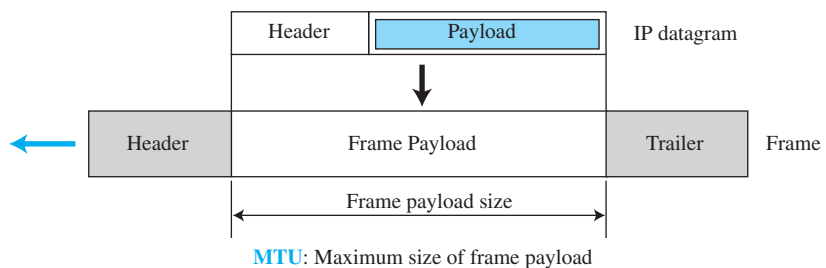
Fragmentation

A datagram can travel through different networks. Each router decapsulates the IP datagram from the frame it receives, processes it, and then encapsulates it in another frame. The format and size of the received frame depend on the protocol used by the physical network through which the frame has just traveled. The format and size of the sent frame depend on the protocol used by the physical network through which the frame is going to travel. For example, if a router connects a LAN to a WAN, it receives a frame in the LAN format and sends a frame in the WAN format.

Maximum Transfer Unit (MTU)

Each link-layer protocol (see Chapter 5) has its own frame format. One of the features of each format is the maximum size of the payload that can be encapsulated. In other words, when a datagram is encapsulated in a frame, the total size of the datagram must be less than this maximum size, which is defined by the restrictions imposed by the hardware and software used in the network (see Figure 4.26).

Figure 4.26 Maximum transfer unit (MTU)



The value of the MTU differs from one physical network protocol to another. For example, the value for a LAN is normally 1500 bytes, but for a WAN it can be larger or smaller.

In order to make the IP protocol independent of the physical network, the designers decided to make the maximum length of the IP datagram equal to 65,535 bytes. This makes transmission more efficient if one day we use a link-layer protocol with an MTU of this size. However, for other physical networks, we must divide the datagram to make it possible for it to pass through these networks. This is called **fragmentation**.

When a datagram is fragmented, each fragment has its own header with most of the fields repeated, but some have been changed. A fragmented datagram may itself be fragmented if it encounters a network with an even smaller MTU. In other words, a datagram may be fragmented several times before it reaches the final destination.

A datagram can be fragmented by the source host or any router in the path. The *reassembly* of the datagram, however, is done only by the destination host, because each fragment becomes an independent datagram. Whereas the fragmented datagram can travel through different routes, and we can never control or guarantee which route a fragmented datagram may take, all of the fragments belonging to the same datagram should finally arrive at the destination host. So it is logical to do the reassembly at the final destination. An even stronger objection for reassembling packets during the transmission is the loss of efficiency it incurs.

When we talk about fragmentation, we mean that the payload of the IP datagram is fragmented. However, most parts of the header, with the exception of some options, must be copied by all fragments. The host or router that fragments a datagram must change the values of three fields: flags, fragmentation offset, and total length. The rest of the fields must be copied. Of course, the value of the checksum must be recalculated regardless of fragmentation.

Fields Related to Fragmentation

We mentioned before that three fields in an IP datagram are related to fragmentation: *identification*, *flags*, and *fragmentation offset*. Let us explain these fields now.

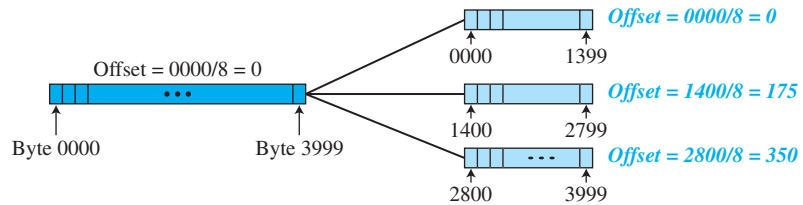
The 16-bit *identification field* identifies a datagram originating from the source host. The combination of the identification and source IP address must uniquely define a datagram as it leaves the source host. To guarantee uniqueness, the IP protocol uses a counter to label the datagrams. The counter is initialized to a positive number. When the IP protocol sends a datagram, it copies the current value of the counter to the identification field and increments the counter by one. As long as the counter is kept in the main memory, uniqueness is guaranteed. When a datagram is fragmented, the value in the identification field is copied into all fragments. In other words, all fragments have the same identification number, which is also the same as the original datagram. The identification number helps the destination in reassembling the datagram. It knows that all fragments having the same identification value should be assembled into one datagram.

The 3-bit *flags field* defines three flags. The leftmost bit is reserved (not used). The second bit (D bit) is called the *do not fragment* bit. If its value is 1, the machine must not fragment the datagram. If it cannot pass the datagram through any available physical network, it discards the datagram and sends an ICMP error message to the source host (discussed later). If its value is 0, the datagram can be fragmented if necessary. The

third bit (M bit) is called the *more fragment bit*. If its value is 1, it means the datagram is not the last fragment; there are more fragments after this one. If its value is 0, it means this is the last or only fragment.

The 13-bit *fragmentation offset field* shows the relative position of this fragment with respect to the whole datagram. It is the offset of the data in the original datagram measured in units of 8 bytes. Figure 4.27 shows a datagram with a data size of 4000 bytes fragmented into three fragments. The bytes in the original datagram are numbered 0 to 3999. The first fragment carries bytes 0 to 1399. The offset for this datagram is $0/8 = 0$. The second fragment carries bytes 1400 to 2799; the offset value for this fragment is $1400/8 = 175$. Finally, the third fragment carries bytes 2800 to 3999. The offset value for this fragment is $2800/8 = 350$.

Figure 4.27 Fragmentation example



Remember that the value of the offset is measured in units of 8 bytes. This is done because the length of the offset field is only 13 bits long and cannot represent a sequence of bytes greater than 8191. This forces hosts or routers that fragment datagrams to choose the size of each fragment so that the first byte number is divisible by 8.

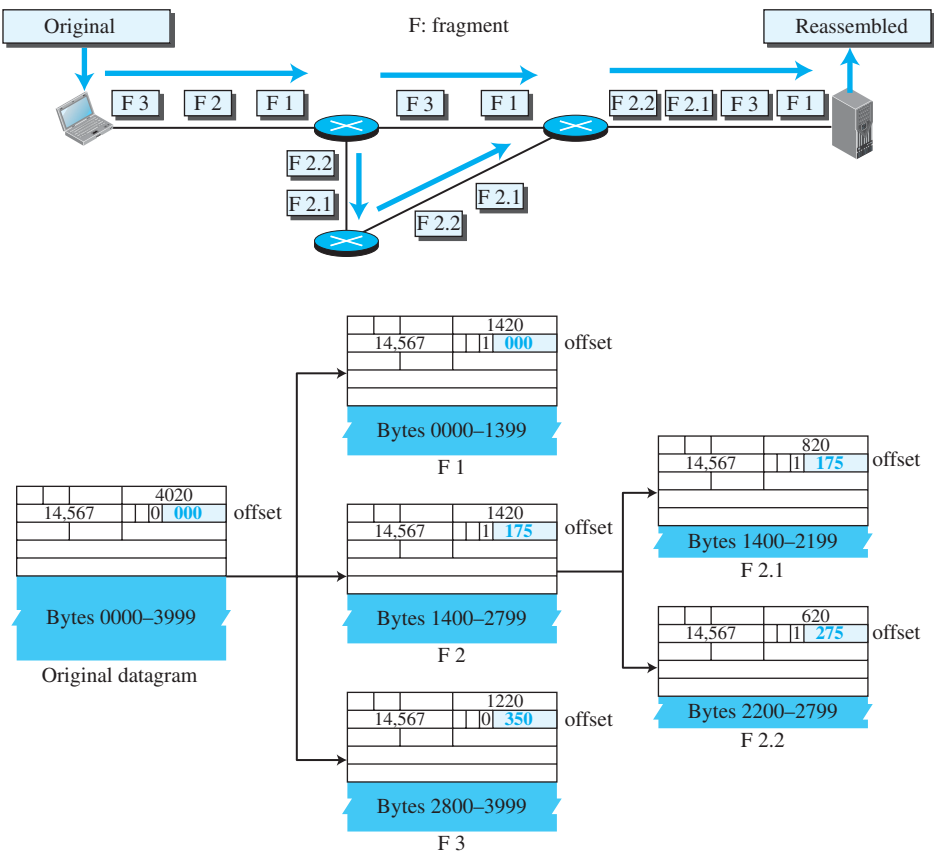
Figure 4.28 shows an expanded view of the fragments in the previous figure. The original packet starts at the client; the fragments are reassembled at the server. The value of the identification field is the same in all fragments, as is the value of the flags field with the more bit set for all fragments except the last. Also, the value of the offset field for each fragment is shown. Note that although the fragments arrived out of order at the destination, they can be correctly reassembled.

The figure also shows what happens if a fragment itself is fragmented. In this case the value of the offset field is always relative to the original datagram. For example, in the figure, the second fragment is itself fragmented later into two fragments of 800 bytes and 600 bytes, but the offset shows the relative position of the fragments to the original data.

It is obvious that even if each fragment follows a different path and arrives out of order, the final destination host can reassemble the original datagram from the fragments received (if none of them is lost) using the following strategy:

- a. The first fragment has an offset field value of zero.
- b. Divide the length of the first fragment by 8. The second fragment has an offset value equal to that result.
- c. Divide the total length of the first and second fragment by 8. The third fragment has an offset value equal to that result.
- d. Continue the process. The last fragment has its M bit set to 0.

Figure 4.28 Detailed fragmentation example



Security of IPv4 Datagrams

The IPv4 protocol, as well as the whole Internet, was started when the Internet users trusted each other. No security was provided for the IPv4 protocol. Today, however, the situation is different; the Internet is not secure anymore. Although we discuss network security in general and IP security in particular in Chapter 10, here we give a brief idea about the security issues in IP protocol and the solution. There are three security issues that are particularly applicable to the IP protocol: packet sniffing, packet modification, and IP spoofing.

Packet Sniffing

An intruder may intercept an IP packet and make a copy of it. Packet sniffing is a passive attack, in which the attacker does not change the contents of the packet. This type of attack is very difficult to detect because the sender and the receiver may never know that the packet has been copied. Although packet sniffing cannot be stopped, encryption of the packet can make the attacker's effort useless. The attacker may still sniff the packet, but the content is not detectable.

Packet Modification

The second type of attack is to modify the packet. The attacker intercepts the packet, changes its contents, and sends the new packet to the receiver. The receiver believes that the packet is coming from the original sender. This type of attack can be detected using a data integrity mechanism. The receiver, before opening and using the contents of the message, can use this mechanism to make sure that the packet has not been changed during the transmission. We discuss packet integrity in Chapter 10.

IP Spoofing

An attacker can masquerade as somebody else and create an IP packet that carries the source address of another computer. An attacker can send an IP packet to a bank pretending that it is coming from one of the customers. This type of attack can be prevented using an origin authentication mechanism (see Chapter 10).

IPSec

The IP packets today can be protected from the previously mentioned attacks using a protocol called IPSec (IP Security). This protocol, which is used in conjunction with the IP protocol, creates a connection-oriented service between two entities in which they can exchange IP packets without worrying about the three attacks discussed above. We will discuss IPSec in detail in Chapter 10; here it is enough to mention that IPSec provides the following four services:

- ❑ ***Defining Algorithms and Keys.*** The two entities that want to create a secure channel between themselves can agree on some available algorithms and keys to be used for security purposes.
- ❑ ***Packet Encryption.*** The packets exchanged between two parties can be encrypted for privacy using one of the encryption algorithms and a shared key agreed upon in the first step. This makes the packet sniffing attack useless.
- ❑ ***Data Integrity.*** Data integrity guarantees that the packet is not modified during the transmission. If the received packet does not pass the data integrity test, it is discarded. This prevents the second attack, packet modification, described above.
- ❑ ***Origin Authentication.*** IPSec can authenticate the origin of the packet to be sure that the packet is not created by an imposter. This can prevent IP spoofing attacks as described above.

4.2.2 IPv4 Addresses

The identifier used in the IP layer of the TCP/IP protocol suite to identify the connection of each device to the Internet is called the Internet address or IP address. An IPv4 address is a 32-bit address that uniquely and universally defines the connection of a host or a router to the Internet. The IP address is the address of the connection, not the host or the router, because if the device is moved to another network, the IP address may be changed.

IPv4 addresses are unique in the sense that each address defines one, and only one, connection to the Internet. If a device has two connections to the Internet, via two networks, it has two IPv4 addresses. IPv4 addresses are universal in the sense that the addressing system must be accepted by any host that wants to be connected to the Internet.

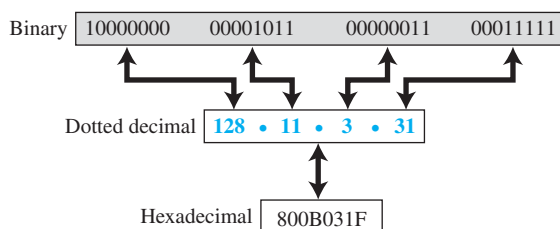
Address Space

A protocol like IPv4 that defines addresses has an address space. An address space is the total number of addresses used by the protocol. If a protocol uses b bits to define an address, the address space is 2^b because each bit can have two different values (0 or 1). IPv4 uses 32-bit addresses, which means that the address space is 2^{32} or 4,294,967,296 (more than four billion). If there were no restrictions, more than 4 billion devices could be connected to the Internet.

Notation

There are three common notations to show an IPv4 address: binary notation (base 2), dotted-decimal notation (base 256), and hexadecimal notation (base 16). In *binary notation*, an IPv4 address is displayed as 32 bits. To make the address more readable, one or more spaces are usually inserted between each octet (8 bits). Each octet is often referred to as a byte. To make the IPv4 address more compact and easier to read, it is usually written in decimal form with a decimal point (dot) separating the bytes. This format is referred to as *dotted-decimal notation*. Note that because each byte (octet) is only 8 bits, each number in the dotted-decimal notation is between 0 and 255. We sometimes see an IPv4 address in hexadecimal notation. Each hexadecimal digit is equivalent to four bits. This means that a 32-bit address has 8 hexadecimal digits. This notation is often used in network programming. Figure 4.29 shows an IP address in the three discussed notations.

Figure 4.29 Three different notations in IPv4 addressing



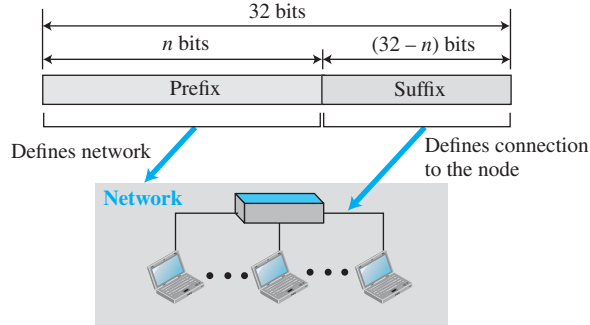
Hierarchy in Addressing

In any communication network that involves delivery, such as a telephone network or a postal network, the addressing system is hierarchical. In a postal network, the postal address (mailing address) includes the country, state, city, street, house number, and the name of the mail recipient. Similarly, a telephone number is divided into the country code, area code, local exchange, and the connection.

A 32-bit IPv4 address is also hierarchical, but divided only into two parts. The first part of the address, called the *prefix*, defines the network; the second part of the address, called the *suffix*, defines the node (connection of a device to the Internet). Figure 4.30 shows the prefix and suffix of a 32-bit IPv4 address. The prefix length is n bits and the suffix length is $(32 - n)$ bits.

A prefix can be fixed length or variable length. The network identifier in the IPv4 was first designed as a fixed-length prefix. This scheme, which is now obsolete, is referred to as classful addressing. The new scheme, which is referred to as classless

Figure 4.30 Hierarchy in addressing

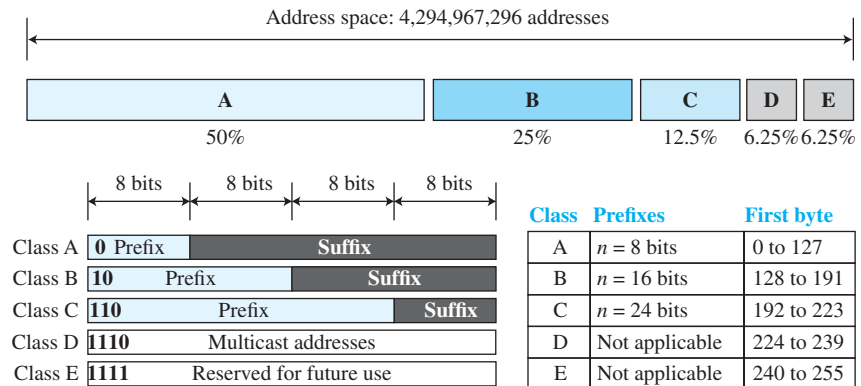


addressing, uses a variable-length network prefix. First, we briefly discuss classful addressing; then we concentrate on classless addressing.

Classful Addressing

When the Internet started, an IPv4 address was designed with a fixed-length prefix, but to accommodate both small and large networks, three fixed-length prefixes were designed instead of one ($n = 8$, $n = 16$, and $n = 24$). The whole address space was divided into five classes (class A, B, C, D, and E), as shown in Figure 4.31. This scheme is referred to as **classful addressing**.

Figure 4.31 Occupation of the address space in classful addressing



In class A, the network length is 8 bits, but since the first bit, which is 0, defines the class, we can have only seven bits as the network identifier. This means there are only $2^7 = 128$ networks in the world that can have a class A address.

In class B, the network length is 16 bits, but since the first two bits, which are (10)₂, define the class, we can have only 14 bits as the network identifier. This means there are only $2^{14} = 16,384$ networks in the world that can have a class B address.

All addresses that start with $(110)_2$ belong to class C. In class C, the network length is 24 bits, but since three bits define the class, we can have only 21 bits as the network identifier. This means there are $2^{21} = 2,097,152$ networks in the world that can have a class C address.

Class D is not divided into prefix and suffix. It is used for multicast addresses. All addresses that start with 1111 in binary belong to class E. As in Class D, Class E is not divided into prefix and suffix and is used as reserve.

Address Depletion

The reason that classful addressing has become obsolete is address depletion. Since the addresses were not distributed properly, the Internet was faced with the problem of the addresses being rapidly used up, resulting in no more addresses available for organizations and individuals that needed to be connected to the Internet. To understand the problem, let us think about class A. This class can be assigned to only 128 organizations in the world, but each organization needs to have one single network (seen by the rest of the world) with 16,777,216 nodes (computers in this single network). Since there may be only a few organizations that are this large, most of the addresses in this class were wasted (unused). Class B addresses were designed for midsize organization, but many of the addresses in this class also remained unused. Class C addresses have a completely different flaw in design. The number of addresses that can be used in each network (256) was so small that most companies were not comfortable using a block in this address. Class E addresses were almost never used, wasting the whole class.

Subnetting and Supernetting

To alleviate address depletion, two strategies were proposed and, to some extent, implemented: subnetting and supernetting. In subnetting, a class A or class B block is divided into several subnets. Each subnet has a larger prefix length than the original network. For example, if a network in class A is divided into four subnets, each subnet has a prefix of $n_{\text{sub}} = 10$. At the same time, if all of the addresses in a network are not used, subnetting allows the address to be divided among several organizations. This idea did not work because most large organizations were not happy about dividing the block and giving some of the unused addresses to smaller organizations.

While subnetting was devised to divide a large block into smaller ones, supernetting was devised to combine several class C blocks into a larger block to be attractive to organizations that need more than the 256 addresses available in a class C block. This idea did not work either because it makes the routing of packets more difficult.

Advantage of Classful Addressing

Although classful addressing had several problems and became obsolete, it had one advantage: Given an address, we can easily find the class of the address and, since the prefix length for each class is fixed, we can find the prefix length immediately. In other words, the prefix length in classful addressing is inherent in the address; no extra information is needed to extract the prefix and the suffix.

Classless Addressing

Subnetting and supernetting in classful addressing did not really solve the address depletion problem. With the growth of the Internet, it was clear that a larger address space was needed as a long-term solution. The larger address space, however, requires

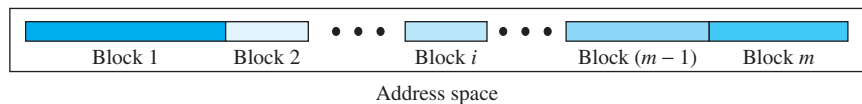
that the length of IP addresses also be increased, which means the format of the IP packets needs to be changed. Although the long-range solution has already been devised and is called IPv6 (discussed later), a short-term solution was also devised to use the same address space but to change the distribution of addresses to provide a fair share to each organization. The short-term solution still uses IPv4 addresses, but it is called *classless addressing*. In other words, the class privilege was removed from the distribution to compensate for the address depletion.

There was another motivation for classless addressing. During the 1990s, Internet Service Providers (ISPs) came into prominence. An ISP is an organization that provides Internet access for individuals, small businesses, and midsize organizations that do not want to create an Internet site and become involved in providing Internet services (such as electronic mail) for their employees. An ISP can provide these services. An ISP is granted a large range of addresses and then subdivides the addresses (in groups of 1, 2, 4, 8, 16, and so on), giving a range of addresses to a household or a small business. The customers are connected via a dial-up modem, DSL, or cable modem to the ISP. However, each customer needs some IPv4 addresses.

In 1996, the Internet authorities announced a new architecture called **classless addressing**. In classless addressing, variable-length blocks are used that belong to no classes. We can have a block of 1 address, 2 addresses, 4 addresses, 128 addresses, and so on.

In classless addressing, the whole address space is divided into variable length blocks. The prefix in an address defines the block (network); the suffix defines the node (device). Theoretically, we can have a block of $2^0, 2^1, 2^2, \dots, 2^{32}$ addresses. One of the restrictions, as we discuss later, is that the number of addresses in a block needs to be a power of 2. An organization can be granted one block of addresses. Figure 4.32 shows the division of the whole address space into nonoverlapping blocks.

Figure 4.32 Variable-length blocks in classless addressing



Unlike classful addressing, the prefix length in classless addressing is variable. We can have a prefix length that ranges from 0 to 32. The size of the network is inversely proportional to the length of the prefix. A small prefix means a larger network; a large prefix means a smaller network.

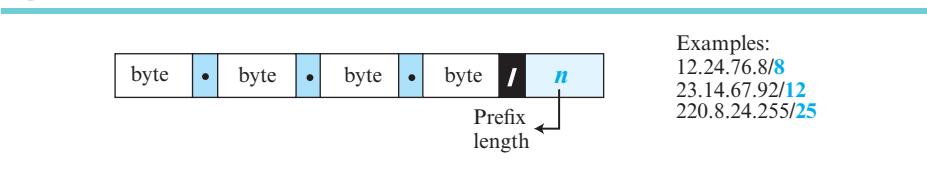
We need to emphasize that the idea of classless addressing can be easily applied to classful addressing. An address in class A can be thought of as a classless address in which the prefix length is 8. An address in class B can be thought of as a classless address in which the prefix is 16, and so on. In other words, classful addressing is a special case of classless addressing.

Prefix Length: Slash Notation

The first question that we need to answer in classless addressing is how to find the prefix length if an address is given. Since the prefix length is not inherent in the address,

we need to separately give the length of the prefix. In this case, the prefix length, n , is added to the address, separated by a slash. The notation is informally referred to as *slash notation* and formally as **classless interdomain routing** or **CIDR** (pronounced *cider*) strategy. An address in classless addressing can then be represented as shown in Figure 4.33.

Figure 4.33 Slash notation (CIDR)

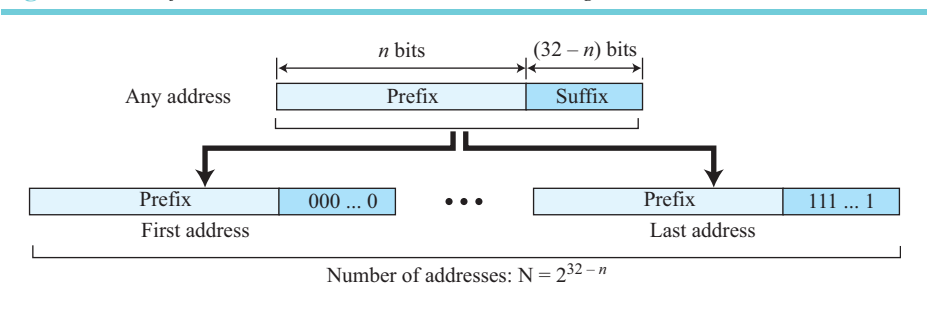


In other words, an address in classless addressing does not, per se, define the block or network to which the address belongs; we need to give the prefix length also.

Extracting Information from an Address

Given any address in the block, we normally like to know three pieces of information about the block to which the address belongs: the number of addresses, the first address in the block, and the last address. Since the value of prefix length, n , is given, we can easily find these three pieces of information, as shown in Figure 4.34.

Figure 4.34 Information extraction in classless addressing



1. The number of addresses in the block is found as $N = 2^{32 - n}$.
2. To find the first address, we keep the n leftmost bits and set the $(32 - n)$ rightmost bits all to 0s.
3. To find the last address, we keep the n leftmost bits and set the $(32 - n)$ rightmost bits all to 1s.

Example 4.1

A classless address is given as 167.199.170.82/27. We can find the above three pieces of information as follows. The number of addresses in the network is $2^{32 - n} = 2^5 = 32$ addresses.

The first address can be found by keeping the first 27 bits and changing the rest of the bits to 0s.

Address: 167.199.170.82/27 10100111 11000111 10101010 01010010
First address: 167.199.170.64/27 10100111 11000111 10101010 01000000

The last address can be found by keeping the first 27 bits and changing the rest of the bits to 1s.

Address: 167.199.170.82/27	10100111	11000111	10101010	01011111
Last address: 167.199.170.95/27	10100111	11000111	10101010	01011111

Address Mask

Another way to find the first and last addresses in the block is to use the address mask. The address mask is a 32-bit number in which the n leftmost bits are set to 1s and the rest of the bits ($32 - n$) are set to 0s. A computer can easily find the address mask because it is the complement of $(2^{32-n} - 1)$. The reason for defining a mask in this way is that it can be used by a computer program to extract the information in a block, using the three bit-wise operations NOT, AND, and OR.

1. The number of addresses in the block $N = \text{NOT}(\text{Mask}) + 1$.
2. The first address in the block = (Any address in the block) **AND** (Mask).
3. The last address in the block = (Any address in the block) **OR** [(**NOT** (Mask))].

Example 4.2

We repeat Example 4.1 using the mask. The mask in dotted-decimal notation is 256.256.256.224. The AND, OR, and NOT operations can be applied to individual bytes using calculators and applets at the book website.

Number of addresses in the block:	$N = \text{NOT}(\text{mask}) + 1 = 0.0.0.31 + 1 = 32$ addresses
First address:	First = (address) AND (mask) = 167.199.170. 82
Last address:	Last = (address) OR (NOT mask) = 167.199.170. 255

Example 4.3

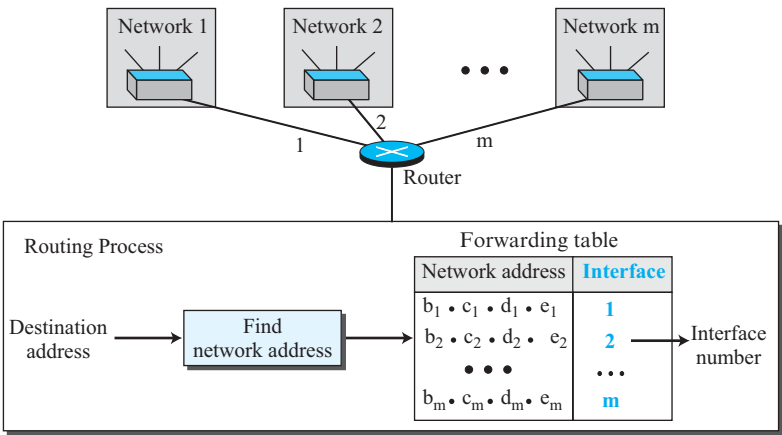
In classless addressing, an address cannot per se define the block the address belongs to. For example, the address 230.8.24.56 can belong to many blocks. Some of them are shown below with the value of the prefix associated with that block.

Prefix length:16	→	Block:	230.8.0.0	to	230.8.255.255
Prefix length:20	→	Block:	230.8.16.0	to	230.8.31.255
Prefix length:26	→	Block:	230.8.24.0	to	230.8.24.63
Prefix length:27	→	Block:	230.8.24.32	to	230.8.24.63
Prefix length:29	→	Block:	230.8.24.56	to	230.8.24.63
Prefix length:31	→	Block:	230.8.24.56	to	230.8.24.57

Network Address

The above examples show that, given any address, we can find all information about the block. The first address, **network address**, is particularly important because it is used in routing a packet to its destination network. For the moment, let us assume that an internet is made of m networks and a router with m interfaces. When a packet arrives at the router from any source host, the router needs to know to which network the packet should be sent: from which interface the packet should be sent out. When the packet arrives at the network, it reaches its destination host using another strategy that we discuss later. Figure 4.35 shows the idea. After the network address has been found,

Figure 4.35 Network address



the router consults its forwarding table to find the corresponding interface from which the packet should be sent out. The network address is actually the identifier of the network; each network is identified by its network address.

Block Allocation

The next issue in classless addressing is block allocation. How are the blocks allocated? The ultimate responsibility of block allocation is given to a global authority called the Internet Corporation for Assigned Names and Numbers (ICANN). However, ICANN does not normally allocate addresses to individual Internet users. It assigns a large block of addresses to an ISP (or a larger organization that is considered an ISP in this case). For the proper operation of the CIDR, two restrictions need to be applied to the allocated block.

1. The number of requested addresses, N , needs to be a power of 2. The reason is that $N = 2^{32-n}$ or $n = 32 - \log_2 N$. If N is not a power of 2, we cannot have an integer value for n .
2. The requested block needs to be allocated where there are a contiguous number of available addresses in the address space. However, there is a restriction on choosing the first address in the block. The first address needs to be divisible by the number of addresses in the block. The reason is that the first address needs to be the prefix followed by $(32 - n)$ number of 0s. The decimal value of the first address is then

$$\text{first address} = (\text{prefix in decimal}) \times 2^{32-n} = (\text{prefix in decimal}) \times N.$$

Example 4.4

An ISP has requested a block of 1000 addresses. Since 1000 is not a power of 2, 1024 addresses are granted. The prefix length is calculated as $n = 32 - \log_2 1024 = 22$. An available block, 18.14.12.0/22, is granted to the ISP. It can be seen that the first address in decimal is 302,910,464, which is divisible by 1024.

Subnetting

More levels of hierarchy can be created using subnetting. An organization (or an ISP) that is granted a range of addresses may divide the range into several subranges and assign each subrange to a subnetwork (or subnet). Note that nothing stops the organization from creating more levels. A subnetwork can be divided into several sub-subnetworks. A sub-subnetwork can be divided into several sub-sub-subnetworks, and so on.

Designing Subnets The subnetworks in a network should be carefully designed to enable the routing of packets. We assume the total number of addresses granted to the organization is N , the prefix length is n , the assigned number of addresses to each subnetwork is N_{sub} , and the prefix length for each subnetwork is n_{sub} . Then the following steps need to be carefully followed to guarantee the proper operation of the subnetworks.

- The number of addresses in each subnetwork should be a power of 2.
- The prefix length for each subnetwork should be found using the following formula:

$$n_{\text{sub}} = 32 - \log_2 N_{\text{sub}}$$

- The starting address in each subnetwork should be divisible by the number of addresses in that subnetwork. This can be achieved if we first assign addresses to larger subnetworks.

Finding Information about Each Subnetwork After designing the subnetworks, the information about each subnetwork, such as first and last address, can be found using the process we described to find the information about each network in the Internet.

Example 4.5

An organization is granted a block of addresses with the beginning address 14.24.74.0/24. The organization needs to have 3 subblocks of addresses to use in its three subnets: one subblock of 10 addresses, one subblock of 60 addresses, and one subblock of 120 addresses. Design the subblocks.

Solution

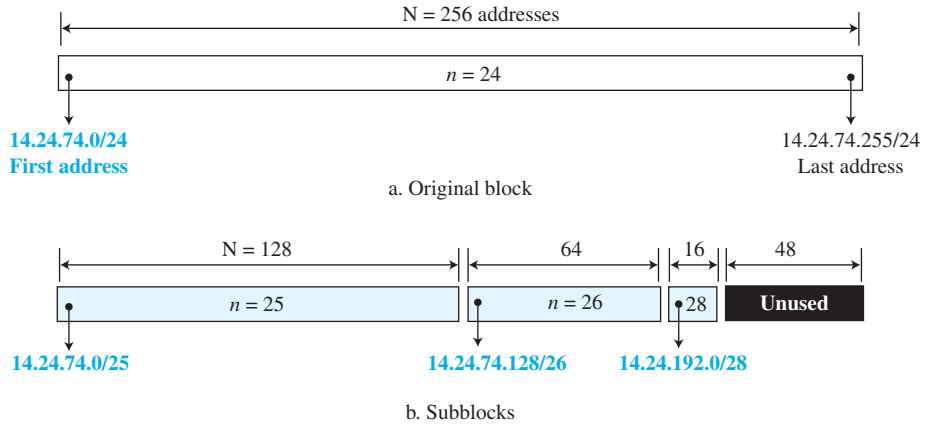
There are $2^{32-24} = 256$ addresses in this block. The first address is 14.24.74.0/24; the last address is 14.24.74.255/24. To satisfy the third requirement, we assign addresses to subblocks, starting with the largest and ending with the smallest one.

- a. The number of addresses in the largest subblock, which requires 120 addresses, is not a power of 2. We allocate 128 addresses. The subnet mask for this subnet can be found as $n_1 = 32 - \log_2 128 = 25$. The first address in this block is 14.24.74.0/25; the last address is 14.24.74.127/25.
- b. The number of addresses in the second largest subblock, which requires 60 addresses, is not a power of 2 either. We allocate 64 addresses. The subnet mask for this subnet can be found as $n_2 = 32 - \log_2 64 = 26$. The first address in this block is 14.24.74.128/26; the last address is 14.24.74.191/26.
- c. The number of addresses in the smallest subblock, which requires 10 addresses, is not a power of 2 either. We allocate 16 addresses. The subnet mask for this subnet can be found as $n_3 = 32 - \log_2 16 = 28$. The first address in this block is 14.24.74.192/28; the last address is 14.24.74.207/28.

If we add all addresses in the previous subblocks, the result is 208 addresses, which means 48 addresses are left in reserve. The first address in this range is 14.24.74.208. The

last address is 14.24.74.255. We don't know about the prefix length yet. Figure 4.36 shows the configuration of blocks. We have shown the first address in each block.

Figure 4.36 Solution to Example 4.5



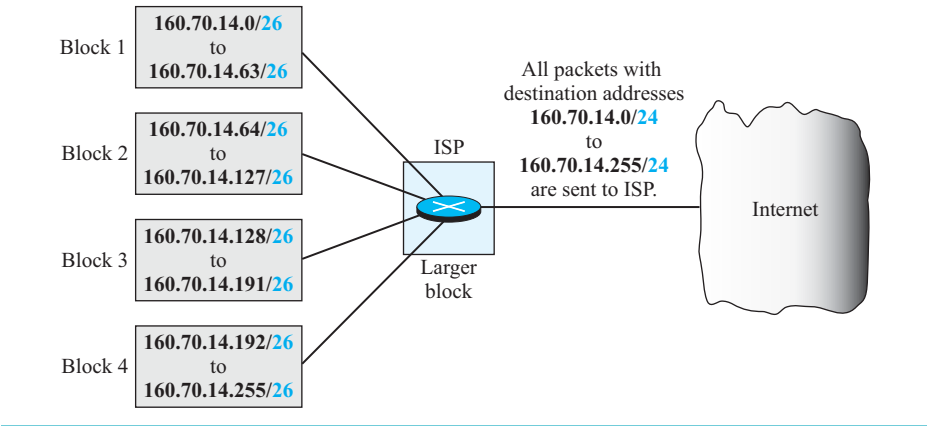
Address Aggregation

One of the advantages of the CIDR strategy is **address aggregation** (sometimes called *address summarization* or *route summarization*). When blocks of addresses are combined to create a larger block, routing can be done based on the prefix of the larger block. ICANN assigns a large block of addresses to an ISP. Each ISP in turn divides its assigned block into smaller subblocks and grants the subblocks to its customers.

Example 4.6

Figure 4.37 shows how four small blocks of addresses are assigned to four organizations by an ISP. The ISP combines these four blocks into one single block and advertises the larger block to the rest of the world. Any packet destined for this larger block should be sent to this ISP. It is the responsibility of the ISP to forward the packet to the appropriate organization. This is similar to

Figure 4.37 Example of address aggregation



routing we can find in a postal network. All packages coming from outside a country are sent first to the capital and then distributed to the corresponding destination.

Special Addresses

Before finishing the topic of addresses in IPv4, we need to mention five special addresses that are used for special purposes: *this-host* address, *limited-broadcast* address, *loopback* address, *private* addresses, and *multicast* addresses.

This-host Address The only address in the block **0.0.0.0/32** is called the *this-host* address. It is used whenever a host needs to send an IP datagram but it does not know its own address to use as the source address. We will see an example of this case in the next section.

Limited-broadcast Address. The only address in the block **255.255.255.255/32** is called the *limited-broadcast* address. It is used whenever a router or a host needs to send a datagram to all devices in a network. The routers in the network, however, block the packet having this address as the destination; the packet cannot travel outside the network.

Loopback Address. The block **127.0.0.0/8** is called the *loopback* address. A packet with one of the addresses in this block as the destination address never leaves the host; it will remain in the host. Any address in the block is used to test a piece of software in the machine. For example, we can write a client and a server program in which one of the addresses in the block is used as the server address. We can test the programs using the same host to see if they work before running them on different computers.

Private addresses Four blocks are assigned as private addresses: **10.0.0.0/8**, **172.16.0.0/12**, **192.168.0.0/16**, and **169.254.0.0/16**. We will see the applications of these address when we discuss NAT later in the chapter.

Multicast addresses The block **224.0.0.0/4** is reserved for multicast addresses. We discuss these addresses later in the chapter.

Dynamic Host Configuration Protocol (DHCP)

We have seen that a large organization or an ISP can receive a block of addresses directly from ICANN and a small organization can receive a block of addresses from an ISP. After a block of addresses are assigned to an organization, the network administration can manually assign addresses to the individual hosts or routers. However, address assignment in an organization can be done automatically using the **Dynamic Host Configuration Protocol (DHCP)**. DHCP is an application-layer program, using the client-server paradigm, that actually helps TCP/IP at the network layer.

DHCP has found such widespread use in the Internet that it is often called a *plug-and-play protocol*. It can be used in many situations. A network manager can configure DHCP to assign permanent IP addresses to the host and routers. DHCP can also be configured to provide temporary, on demand, IP addresses to hosts. The second capability can provide a temporary IP address to a traveller to connect her laptop to the Internet while she is staying in the hotel. It also allows an ISP with 1000 granted addresses to provide services to 4000 households, assuming not more than one-fourth of customers use the Internet at the same time.

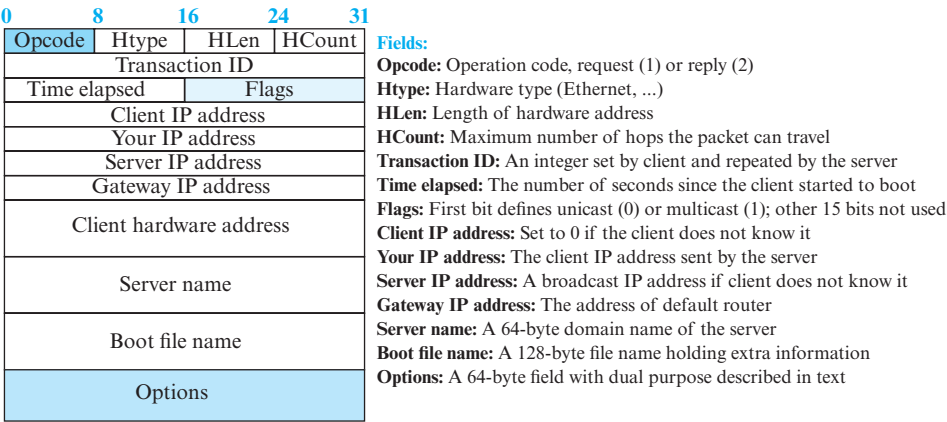
In addition to its IP address, a computer also needs to know the network prefix (or address mask). Most computers also need two other pieces of information, such as the

address of a default router to be able to communicate with other networks and the address of a name server to be able to use names instead of addresses, as we saw in Chapter 2. In other words, four pieces of information are normally needed: the computer address, the prefix, the address of a router, and the IP address of a name server. DHCP can be used to provide these pieces of information to the host.

DHCP Message Format

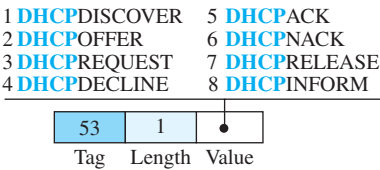
DHCP is a client-server protocol in which the client sends a request message and the server returns a response message. Before we discuss the operation of DHCP, let us show the general format of the DHCP message in Figure 4.38. Most of the fields are explained in the figure, but we need to discuss the option field, which plays a very important role in DHCP.

Figure 4.38 *DHCP message format*



The 64-byte option field has a dual purpose. It can carry either additional information or some specific vendor information. The server uses a number, called a **magic cookie**, in the format of an IP address with the value of 99.130.83.99. When the client finishes reading the message, it looks for this magic cookie. If present, the next 60 bytes are options. An option is composed of three fields: a 1-byte tag field, a 1-byte length field, and a variable-length value field. There are several tag fields that are mostly used by vendors. If the tag field is 53, the value field defines one of the 8 message types shown in Figure 4.39. We show how these message types are used by DHCP.

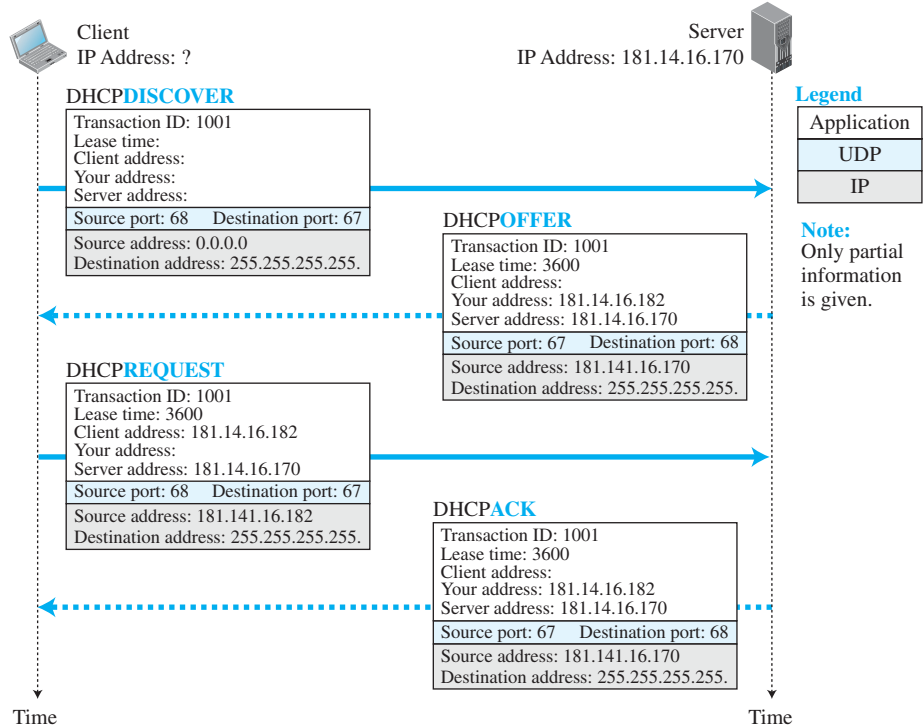
Figure 4.39 *Option format*



DHCP Operation

Figure 4.40 shows a simple scenario.

Figure 4.40 Operation of DHCP



1. The joining host creates a **DHCPDISCOVER** message in which only the transaction-ID field is set to a random number. No other field can be set because the host has no knowledge to do so. This message is encapsulated in a UDP user datagram with the source port set to 68 and the destination port set to 67. We will discuss the reason for using two well-known port numbers later. The user datagram is encapsulated in an IP datagram with the source address set to **0.0.0.0** (“this host”) and the destination address set to **255.255.255.255** (broadcast address). The reason is that the joining host knows neither its own address nor the server address.
2. The DHCP server or servers (if more than one) responds with a **DHCPOFFER** message in which the your-IP-address field defines the offered IP address for the joining host and the server-IP-address includes the IP address of the server. The message also includes the lease time for which the host can keep the IP address. This message is encapsulated in a user datagram with the same port numbers, but in the reverse order. The user datagram in turn is encapsulated in a datagram with the server address as the source IP address, but the destination address is a broadcast address, in which the server allows other DHCP servers to receive the offer and give a better offer if they can.

3. The joining host receives one or more offers and selects the best of them. The joining host then sends a **DHCPREQUEST** message to the server that has given the best offer. The fields with known value are set. The message is encapsulated in a user datagram with port numbers as the first message. The user datagram is encapsulated in an IP datagram with the source address set to the new client address, but the destination address still is set to the broadcast address to let the other servers know that their offer was not accepted.
4. Finally, the selected server responds with an a **DHCPACK** message to the client if the offered IP address is valid. If the server cannot keep its offer (for example, if the address is offered to another host in between), the server sends a **DHCPNACK** message and the client needs to repeat the process. This message is also broadcast to let other servers know that the request is accepted or rejected.

Two Well-Known Ports

We said that the DHCP uses two well-known ports (68 and 67) instead of one well-known and one ephemeral. The reason for choosing the well-known port 68 instead of an ephemeral port for the client is that the response from the server to the client is broadcast. Remember that an IP datagram with the limited broadcast message is delivered to every host on the network. Now assume that a DHCP client and a DAYTIME client, for example, are both waiting to receive a response from their corresponding server and both have accidentally used the same temporary port number (56017, for example). Both hosts receive the response message from the DHCP server and deliver the message to their clients. The DHCP client processes the message; the DAYTIME client is totally confused with a strange message received. Using a well-known port number prevents this problem from happening. The response message from the DHCP server is not delivered to the DAYTIME client, which is running on the port number 56017, not 68. The temporary port numbers are selected from a different range than the well-known port numbers.

The curious reader may ask what happens if two DHCP clients are running at the same time. This can happen after a power failure and power restoration. In this case the messages can be distinguished by the value of the transaction ID, which separates each response from the other.

Using FTP

The server does not send all of the information that a client may need for joining the network. In the **DHCPACK** message, the server defines the pathname of a file in which the client can find complete information such as the address of the DNS server. The client can then use a file transfer protocol to obtain the rest of the needed information.

Error Control

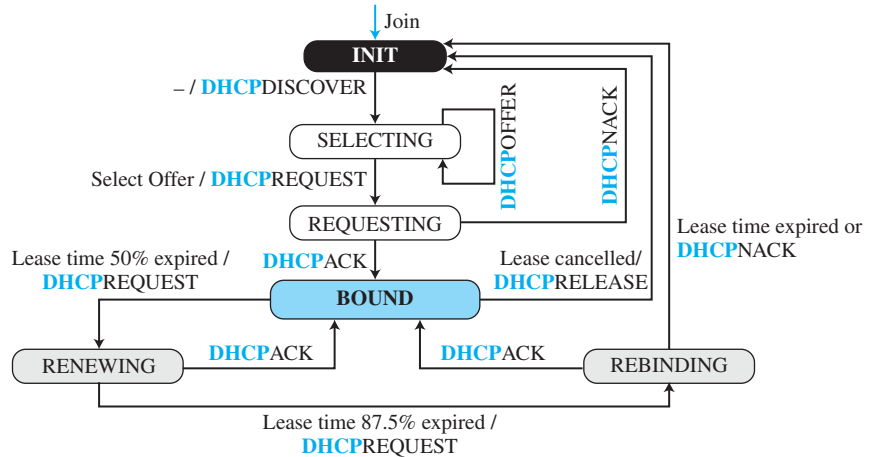
DHCP uses the service of UDP, which is not reliable. To provide error control, DHCP uses two strategies. First, DHCP requires that UDP uses the checksum. As we saw in Chapter 3, the use of the checksum in UDP is optional. Second, the DHCP client uses timers and a retransmission policy if it does not receive the DHCP reply to a request. However, to prevent a traffic jam when several hosts need to retransmit a request (for example, after a power failure), DHCP forces the client to use a random number to set its timers.

Transition States

The previous scenarios we discussed for the operation of the DHCP were very simple. To provide dynamic address allocation, the DHCP client acts as a state machine that

performs transitions from one state to another depending on the messages it receives or sends. Figure 4.41 shows the transition diagram with the main states.

Figure 4.41 *FSM for the DHCP client*



When the DHCP client first starts, it is in the **INIT** state (initializing state). The client broadcasts a discover message. When it receives an offer, the client goes to the **SELECTING** state. While it is there, it may receive more offers. After it selects an offer, it sends a request message and goes to the **REQUESTING** state. If an ACK arrives while the client is in this state, it goes to the **BOUND** state and uses the IP address. When the lease is 50 percent expired, the client tries to renew it by moving to the **RENEWING** state. If the server renews the lease, the client moves to the **BOUND** state again. If the lease is not renewed and the lease time is 75 percent expired, the client moves to the **REBINDING** state. If the server agrees with the lease (ACK message arrives), the client moves to the **BOUND** state and continues using the IP address; otherwise, the client moves to the **INIT** state and requests another IP address. Note that the client can use the IP address only when it is in the **BOUND**, **RENEWING**, or **REBINDING** state. The above procedure requires that the client uses three timers: *renewal timer* (set to 50 percent of the lease time), *rebinding timer* (set to 75 percent of the lease time), and *expiration timer* (set to the lease time).

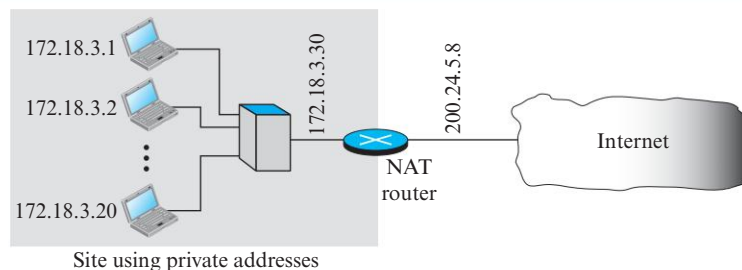
NAT

The distribution of addresses through ISPs has created a new problem. Assume that an ISP has granted a small range of addresses to a small business or a household. If the business grows or the household needs a larger range, the ISP may not be able to grant the demand because the addresses before and after the range may have already been allocated to other networks. In most situations, however, only a portion of computers in a small network need access to the Internet simultaneously. This means that the number of allocated addresses does not have to match the number of computers in the network. For example, assume that in a small business with 20 computers the maximum number of computers that access the Internet simultaneously is only 4. Most of the computers are either doing some task that does not need Internet access or communicating with

each other. This small business can use the TCP/IP protocol for both internal and universal communication. The business can use 20 (or 25) addresses from the private block addresses (discussed before) for internal communication; five addresses for universal communication can be assigned by the ISP.

A technology that can provide the mapping between the private and universal addresses, and at the same time support virtual private networks, which we discuss in Chapter 10, is network address translation (NAT). The technology allows a site to use a set of private addresses for internal communication and a set of global Internet addresses (at least one) for communication with the rest of the world. The site must have only one single connection to the global Internet through a NAT-capable router that runs NAT software. Figure 4.42 shows a simple implementation of NAT.

Figure 4.42 NAT

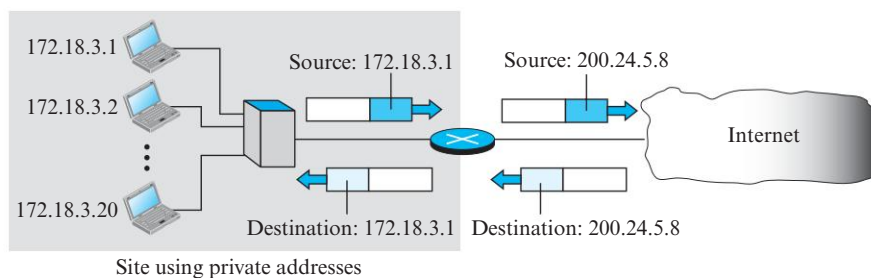


As the figure shows, the private network uses private addresses. The router that connects the network to the global address uses one private address and one global address. The private network is invisible to the rest of the Internet; the rest of the Internet sees only the NAT router with the address 200.24.5.8.

Address Translation

All of the outgoing packets go through the NAT router, which replaces the source address in the packet with the global NAT address. All incoming packets also pass through the NAT router, which replaces the destination address in the packet (the NAT router global address) with the appropriate private address. Figure 4.43 shows an example of address translation.

Figure 4.43 Address translation

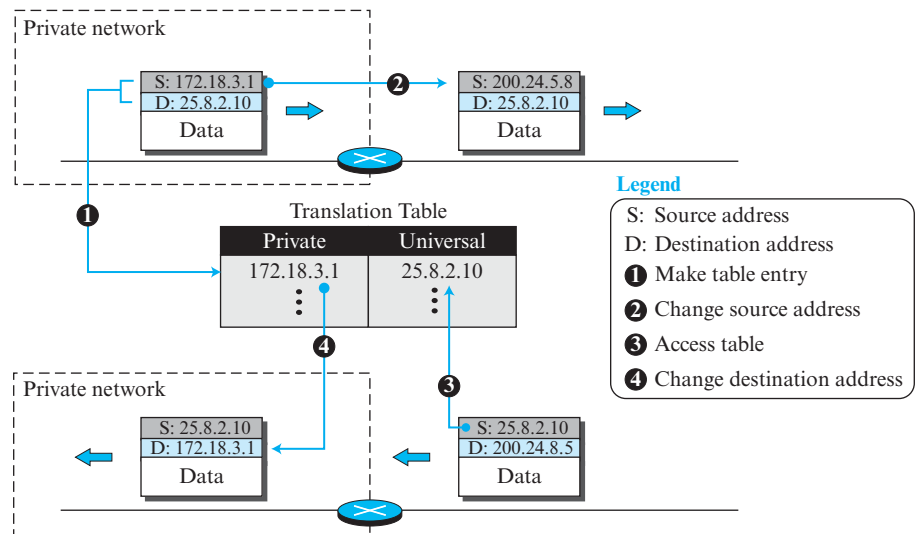


Translation Table

The reader may have noticed that translating the source addresses for an outgoing packet is straightforward. But how does the NAT router know the destination address for a packet coming from the Internet? There may be tens or hundreds of private IP addresses, each belonging to one specific host. The problem is solved if the NAT router has a translation table.

Using One IP Address In its simplest form, a translation table has only two columns: the private address and the external address (destination address of the packet). When the router translates the source address of the outgoing packet, it also makes note of the destination address—where the packet is going. When the response comes back from the destination, the router uses the source address of the packet (as the external address) to find the private address of the packet. Figure 4.44 shows the idea.

Figure 4.44 Translation



In this strategy, communication must always be initiated by the private network. The NAT mechanism described requires that the private network start the communication. As we will see, NAT is used mostly by ISPs that assign one single address to a customer. The customer, however, may be a member of a private network that has many private addresses. In this case, communication with the Internet is always initiated from the customer site, using a client program such as HTTP, TELNET, or FTP to access the corresponding server program. For example, when e-mail that originates from outside the network site is received by the ISP e-mail server, it is stored in the mailbox of the customer until retrieved with a protocol such as POP.

Using a Pool of IP Addresses The use of only one global address by the NAT router allows only one private-network host to access a given external host. To remove this restriction, the NAT router can use a pool of global addresses. For example, instead of

using only one global address (200.24.5.8), the NAT router can use four addresses (200.24.5.8, 200.24.5.9, 200.24.5.10, and 200.24.5.11). In this case, four private-network hosts can communicate with the same external host at the same time because each pair of addresses defines a separate connection. However, there are still some drawbacks. No more than four connections can be made to the same destination. No private-network host can access two external server programs (e.g., HTTP and TELNET) at the same time. And, likewise, two private-network hosts cannot access the same external server program (e.g., HTTP or TELNET) at the same time.

Using Both IP Addresses and Port Addresses To allow a many-to-many relationship between private-network hosts and external server programs, we need more information in the translation table. For example, suppose two hosts inside a private network with addresses 172.18.3.1 and 172.18.3.2 need to access the HTTP server on external host 25.8.3.2. If the translation table has five columns, instead of two, that include the source and destination port addresses and the transport-layer protocol, the ambiguity is eliminated. Table 4.1 shows an example of such a table.

Table 4.1 Five-column translation table

Private address	Private port	External address	External port	Transport protocol
172.18.3.1	1400	25.8.3.2	80	TCP
172.18.3.2	1401	25.8.3.2	80	TCP
⋮	⋮	⋮	⋮	⋮

Note that when the response from HTTP comes back, the combination of source address (25.8.3.2) and destination port address (1401) defines the private network host to which the response should be directed. Note also that for this translation to work, the ephemeral port addresses (1400 and 1401) must be unique.

4.2.3 Forwarding of IP Packets

We discussed the concept of forwarding at the network layer earlier in this chapter. In this section, we extend the concept to include the role of IP addresses in forwarding. As we discussed before, forwarding means to place the packet in its route to its destination. Since the Internet today is made of a combination of links (networks), forwarding means to deliver the packet to the next hop (which can be the final destination or the intermediate connecting device). Although the IP protocol was originally designed as a connectionless protocol, today the tendency is to change it to connection-oriented protocol. We discuss both cases.

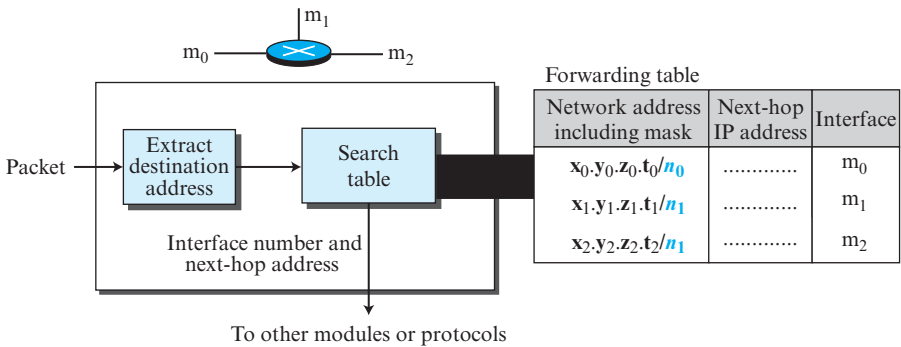
When IP is used as a connectionless protocol, forwarding is based on the destination address of the IP datagram; when the IP is used as a connection-oriented protocol, forwarding is based on the label attached to an IP datagram.

Forwarding Based On Destination Address

We first discuss forwarding based on the destination address. This is a traditional approach, which is prevalent today. In this case, forwarding requires a host or a router to have a forwarding table. When a host has a packet to send or when a router has received a packet to be forwarded, it looks at this table to find the next hop to deliver the packet to.

In classless addressing, the whole address space is one entity; there are no classes. This means that forwarding requires one row of information for each block involved. The table needs to be searched based on the network address (first address in the block). Unfortunately, the destination address in the packet gives no clue about the network address. To solve the problem, we need to include the mask ($/n$) in the table. In other words, a classless forwarding table needs to include four pieces of information: the mask, the network address, the interface number, and the IP address of the next router (needed to find the link-layer address of the next hop, as we discuss in Chapter 5). However, we often see in the literature that the first two pieces are combined. For example, if n is 26 and the network address is 180.70.65.192, then one can combine the two as one piece of information: 180.70.65.192/26. Figure 4.45 shows a simple forwarding module and forwarding table for a router with only three interfaces.

Figure 4.45 Simplified forwarding module in classless address



The job of the forwarding module is to search the table, row by row. In each row, the n leftmost bits of the destination address (prefix) are kept and the rest of the bits (suffix) are set to 0s. If the resulting address (which we call the *network address*), matches with the address in the first column, the information in the next two columns is extracted; otherwise the search continues. Normally, the last row has a default value in the first column (not shown in the figure), which indicates all destination addresses that did not match the previous rows.

Sometimes, the literature explicitly shows the value of the n leftmost bits that should be matched with the n leftmost bits of the destination address. The concept is the same, but the presentation is different. For example, instead of giving the address-mask combination of 180.70.65.192/26, we can give the value of the 26 leftmost bits as shown below.

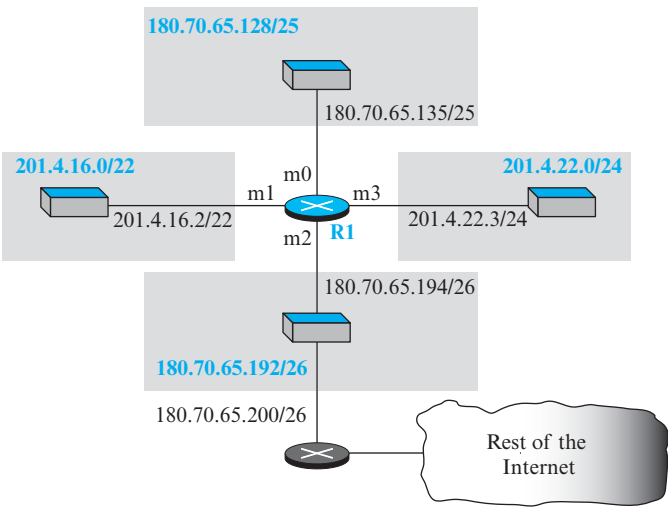
10110100 01000110 01000001 11

Note that we still need to use an algorithm to find the prefix and compare it with the bit pattern. In other words, the algorithm is still needed, but the presentation is different. We use this format in our forwarding tables in the exercises when we use smaller address spaces just for practice.

Example 4.7

Make a forwarding table for router R1 using the configuration in Figure 4.46.

Figure 4.46 Configuration for Example 4.7



Solution

Table 4.2 shows the corresponding table.

Table 4.2 Forwarding table for router R1 in Figure 4.46

Network address/mask	Next hop	Interface
180.70.65.192/26	—	m2
180.70.65.128/25	—	m0
201.4.22.0/24	—	m3
201.4.16.0/22	—	m1
Default	180.70.65.200	m2

Example 4.8

Instead of Table 4.2, we can use Table 4.3, in which the network address/mask is given in bits.

Table 4.3 Forwarding table for router R1 in Figure 4.46 using prefix bits

Leftmost bits in the destination address	Next hop	Interface
10110100 01000110 01000001 11	—	m2
10110100 01000110 01000001 1	—	m0
11001001 00000100 00011100	—	m3
11001001 00000100 000100	—	m1
Default	180.70.65.200	m2

When a packet arrives whose leftmost 26 bits in the destination address match the bits in the first row, the packet is sent out from interface m2. When a packet arrives whose leftmost 25 bits in the address match the bits in the second row, the packet is sent out from interface m0. And so on. The table clearly shows that the first row has the

longest prefix and the fourth row has the shortest prefix. The longer prefix means a smaller range of addresses; the shorter prefix means a larger range of addresses.

Example 4.9

Show the forwarding process if a packet arrives at R1 in Figure 4.46 with the destination address 180.70.65.140.

Solution

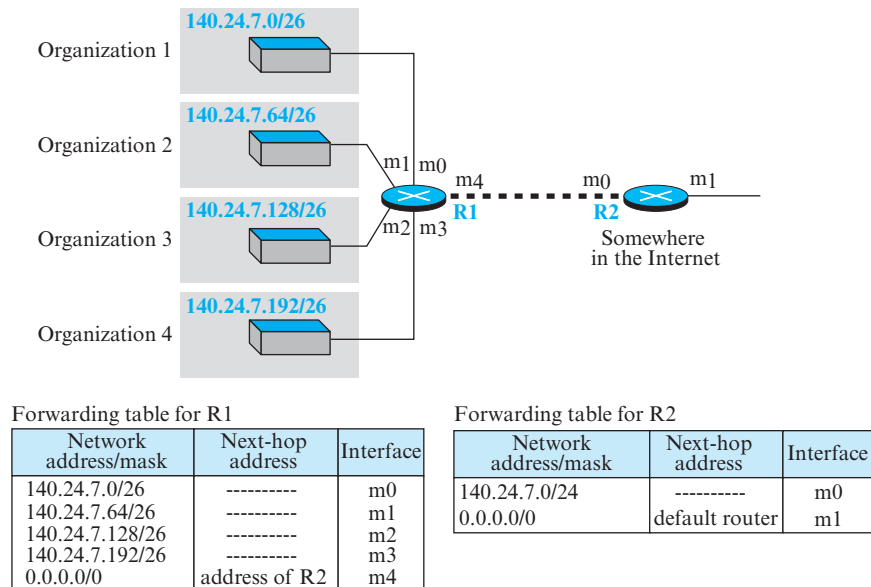
The router performs the following steps:

1. The first mask (/26) is applied to the destination address. The result is 180.70.65.128, which does not match the corresponding network address.
2. The second mask (/25) is applied to the destination address. The result is 180.70.65.128, which matches the corresponding network address. The next-hop address and the interface number m0 are extracted for forwarding the packet (see Chapter 5).

Address Aggregation

When we use classful addressing, there is only one entry in the forwarding table for each site outside the organization. The entry defines the site even if that site is subnetted. When a packet arrives at the router, the router checks the corresponding entry and forwards the packet accordingly. When we use classless addressing, it is likely that the number of forwarding table entries will increase. This is because the intent of classless addressing is to divide up the whole address space into manageable blocks. The increased size of the table results in an increase in the amount of time needed to search the table. To alleviate the problem, the idea of address aggregation was designed. In Figure 4.47 we have two routers.

Figure 4.47 Address aggregation

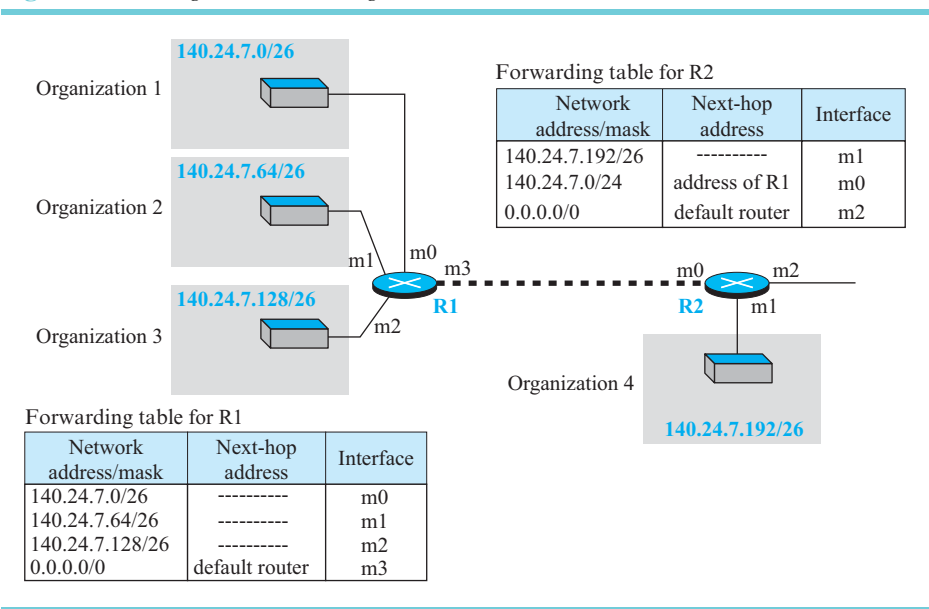


R1 is connected to networks of four organizations that each use 64 addresses. R2 is somewhere far from R1. R1 has a longer forwarding table because each packet must be correctly routed to the appropriate organization. R2, on the other hand, can have a very small forwarding table. For R2, any packet with destination 140.24.7.0 to 140.24.7.255 is sent out from interface m0 regardless of the organization number. This is called address aggregation because the blocks of addresses for four organizations are aggregated into one larger block. R2 would have a longer forwarding table if each organization had addresses that could not be aggregated into one block.

Longest Mask Matching

What happens if one of the organizations in the previous figure is not geographically close to the other three? For example, if organization 4 cannot be connected to router R1 for some reason, can we still use the idea of address aggregation and still assign block 140.24.7.192/26 to organization 4? The answer is yes, because routing in classless addressing uses another principle, *longest mask matching*. This principle states that the forwarding table is sorted from the longest mask to the shortest mask. In other words, if there are three masks, /27, /26, and /24, the mask /27 must be the first entry and /24 must be the last. Let us see if this principle solves the situation in which organization 4 is separated from the other three organizations. Figure 4.48 shows the situation.

Figure 4.48 Longest mask matching



Suppose a packet arrives at router R2 for organization 4 with destination address 140.24.7.200. The first mask at router R2 is applied, which gives the network address 140.24.7.192. The packet is routed correctly from interface m1 and reaches organization 4. If, however, the forwarding table was not stored with the longest prefix first, applying the /24 mask would result in the incorrect routing of the packet to router R1.

Hierarchical Routing

To solve the problem of gigantic forwarding tables, we can create a sense of hierarchy in the forwarding tables. In Chapter 1, we mentioned that the Internet today has a sense of hierarchy. We said that the Internet is divided into backbone and national ISPs. National ISPs are divided into regional ISPs, and regional ISPs are divided into local ISPs. If the forwarding table has a sense of hierarchy like the Internet architecture, the forwarding table can decrease in size.

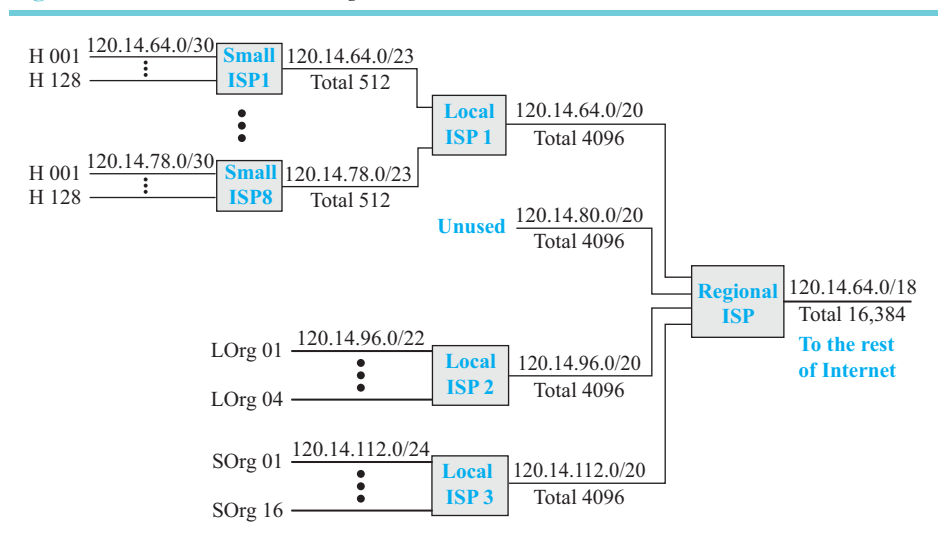
Let us take the case of a local ISP. A local ISP can be assigned a single, but large, block of addresses with a certain prefix length. The local ISP can divide this block into smaller blocks of different sizes, and assign these to individual users and organizations, both large and small. If the block assigned to the local ISP starts with a.b.c.d/n, the ISP can create blocks starting with e.f.g.h/m, where m may vary for each customer and is greater than n.

How does this reduce the size of the forwarding table? The rest of the Internet does not have to be aware of this division. All customers of the local ISP are defined as a.b.c.d/n to the rest of the Internet. Every packet destined for one of the addresses in this large block is routed to the local ISP. There is only one entry in every router in the world for all of these customers. They all belong to the same group. Of course, inside the local ISP, the router must recognize the subblocks and route the packet to the destined customer. If one of the customers is a large organization, it also can create another level of hierarchy by subnetting and dividing its subblock into smaller subblocks (or sub-subblocks). In classless routing, the levels of hierarchy are unlimited as long as we follow the rules of classless addressing.

Example 4.10

As an example of hierarchical routing, let us consider Figure 4.49. A regional ISP is granted 16,384 addresses starting from 120.14.64.0. The regional ISP has decided to divide this block into 4 subblocks, each with 4096 addresses. Three of these subblocks are assigned to three local ISPs, the second subblock is reserved for future use. Note that the mask for each block is /20 because the original block with mask /18 is divided into 4 blocks.

Figure 4.49 Hierarchical routing with ISPs



The first local ISP has divided its assigned subblock into 8 smaller blocks and assigned each to a small ISP. Each small ISP provides services to 128 households (H001 to H128), each using four addresses. Note that the mask for each small ISP is now /23 because the block is further divided into 8 blocks. Each household has a mask of /30, because a household has only 4 addresses ($2^{32-30} = 4$). The second local ISP has divided its block into 4 blocks and has assigned the addresses to 4 large organizations (LOrg01 to LOrg04). Note that each large organization has 1024 addresses and the mask is /22.

The third local ISP has divided its block into 16 blocks and assigned each block to a small organization (SOrg01 to SOrg16). Each small organization has 256 addresses and the mask is /24. There is a sense of hierarchy in this configuration. All routers in the Internet send a packet with destination address 120.14.64.0 to 120.14.127.255 to the regional ISP. The regional ISP sends every packet with destination address 120.14.64.0 to 120.14.79.255 to Local ISP1. Local ISP1 sends every packet with destination address 120.14.64.0 to 120.14.64.3 to H001.

Geographical Routing

To decrease the size of the forwarding table even further, we need to extend hierarchical routing to include geographical routing. We must divide the entire address space into a few large blocks. We assign a block to America, a block to Europe, a block to Asia, a block to Africa, and so on. The routers of ISPs outside of Europe will have only one entry for packets to Europe in their forwarding tables. The routers of ISPs outside of America will have only one entry for packets to America in their forwarding tables, and so on.

Forwarding Table Search Algorithms

In classless addressing, there is no network information in the destination address. The simplest, but not the most efficient, search method is called the longest prefix match (as we discussed before). The forwarding table can be divided into buckets, one for each prefix. The router first tries the longest prefix. If the destination address is found in this bucket, the search is complete. If the address is not found, the next prefix is searched, and so on. It is obvious that this type of search takes a long time.

One solution is to change the data structure used for searching. Instead of a list, other data structures (such as a tree or a binary tree) can be used. One candidate is a trie (a special kind of tree). However, this discussion is beyond the scope of this book.

Forwarding Based on Label

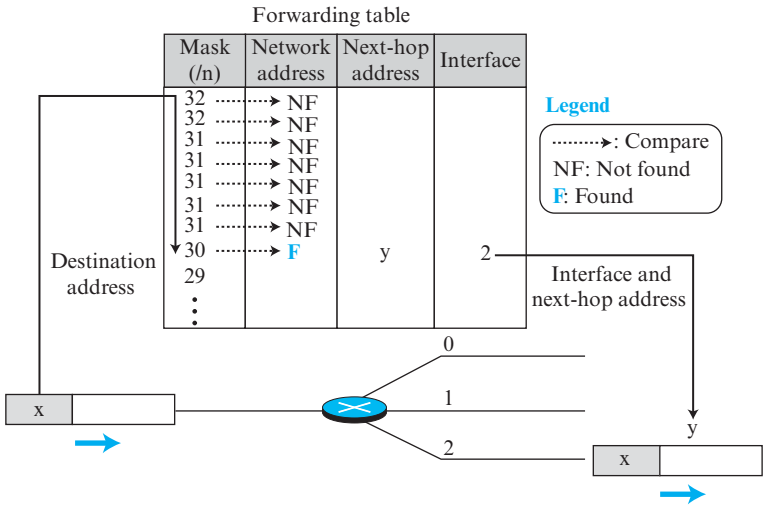
In the 1980s, an effort started to somehow change IP to behave like a connection-oriented protocol in which the routing is replaced by switching. As we discussed earlier in the chapter, in a connectionless network (datagram approach), a router forwards a packet based on the destination address in the header of the packet. On the other hand, in a connection-oriented network (virtual-circuit approach), a switch forwards a packet based on the label attached to the packet. Routing is normally based on searching the contents of a table; switching can be done by accessing a table using an index. In other words, routing involves searching; switching involves accessing.

Example 4.11

Figure 4.50 shows a simple example of searching in a forwarding table using the longest mask algorithm. Although there are some more efficient algorithms today, the principle is the same.

When the forwarding algorithm gets the destination address of the packet, it needs to delve into the mask column. For each entry, it needs to apply the mask to find the destination network address. It then needs to check the network addresses in the table until it finds the

Figure 4.50 Example 4.11: Forwarding based on destination address

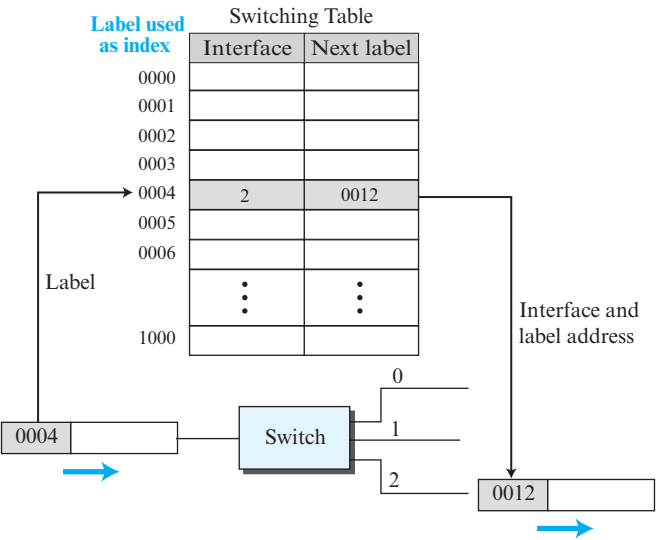


match. The router then extracts the next-hop address and the interface number to be delivered to the data-link layer.

Example 4.12

Figure 4.51 shows a simple example of using a label to access a switching table. Since the labels are used as the index to the table, finding the information in the table is immediate.

Figure 4.51 Example 4.12: Forwarding based on label



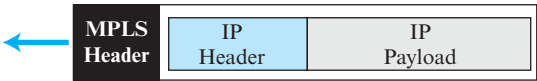
Multi-Protocol Label Switching (MPLS)

During the 1980s, several vendors created routers that implement switching technology. Later IETF approved a standard that is called Multi-Protocol Label Switching. In this standard, some conventional routers in the Internet can be replaced by MPLS routers, which can behave like a router and a switch. When behaving like a router, MPLS can forward the packet based on the destination address; when behaving like a switch, it can forward a packet based on the label.

A New Header

To simulate connection-oriented switching using a protocol like IP, the first thing that is needed is to add a field to the packet that carries the label discussed later. The IPv4 packet format does not allow this extension (although this field is provided in the IPv6 packet format, as we will see later). The solution is to encapsulate the IPv4 packet in an MPLS packet (as though MPLS is a layer between the data-link layer and the network layer). The whole IP packet is encapsulated as the payload in an MPLS packet and an MPLS header is added. Figure 4.52 shows the encapsulation.

Figure 4.52 MPLS header added to an IP packet



The MPLS header is actually a stack of subheaders that is used for multilevel hierarchical switching as we will discuss shortly. Figure 4.53 shows the format of an MPLS header in which each subheader is 32 bits (4 bytes) long.

Figure 4.53 MPLS header made of a stack of labels

0	20	24	31
Label		Exp S	TTL
Label		Exp S	TTL
...			
Label		Exp S	TTL

The following is a brief description of each field:

- ❑ **Label.** This 20-bit field defines the label that is used to index the forwarding table in the router.
- ❑ **Exp.** This 3-bit field is reserved for experimental purposes.
- ❑ **S.** The one-bit stack field defines the situation of the subheader in the stack. When the bit is 1, it means that the header is the last one in the stack.
- ❑ **TTL.** This 8-bit field is similar to the TTL field in the IP datagram. Each visited router decrements the value of this field. When it reaches zero, the packet is discarded to prevent looping.

Hierarchical Switching

A stack of labels in MPLS allows hierarchical switching. This is similar to conventional hierarchical routing. For example, a packet with two labels can use the top label to forward the packet through switches outside an organization; the bottom label can be used to route the packet inside the organization to reach the destination subnet.

4.2.4 ICMPv4

The IPv4 has no error-reporting or error-correcting mechanism. What happens if something goes wrong? What happens if a router must discard a datagram because it cannot find a route to the final destination, or because the time-to-live field has a zero value? What happens if the final destination host must discard the received fragments of a datagram because it has not received all fragments within a predetermined time limit? These are examples of situations where an error has occurred and the IP protocol has no built-in mechanism to notify the original host.

The IP protocol also lacks a mechanism for host and management queries. A host sometimes needs to determine if a router or another host is alive. And sometimes a network manager needs information from another host or router.

The **Internet Control Message Protocol version 4 (ICMPv4)** has been designed to compensate for the above two deficiencies. It is a companion to the IP protocol. ICMP itself is a network-layer protocol. However, its messages are not passed directly to the data-link layer as would be expected. Instead, the messages are first encapsulated inside IP datagrams before going to the lower layer. When an IP datagram encapsulates an ICMP message, the value of the protocol field in the IP datagram is set to 1 to indicate that the IP payload is an ICMP message.

Messages

ICMPv4 messages are divided into two broad categories: *error-reporting messages* and *query messages*. The error-reporting messages report problems that a router or a host (destination) may encounter when it processes an IP packet. The query messages, which occur in pairs, help a host or a network manager get specific information from a router or another host. For example, hosts can discover their neighbor hosts or routers on their network and routers can help a node redirect its messages.

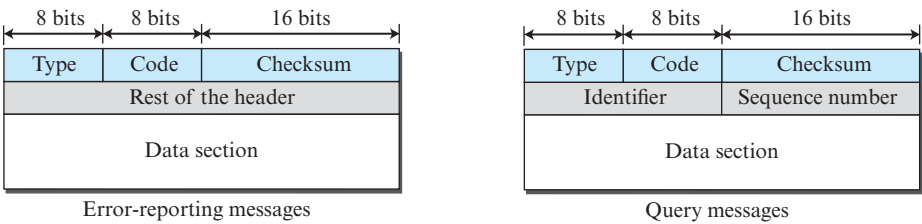
Message Format

An ICMPv4 message has an 8-byte header and a variable-size data section. Although the general format of the header is different for each message type, the first 4 bytes are common to all. As Figure 4.54 shows, the first field, ICMP type, defines the type of the message. The code field specifies the reason for the particular message type. The last common field is the checksum field. The rest of the header is specific to each message type. The data section in error messages carries information for finding the original packet that had the error. In query messages, the data section carries extra information based on the type of query.

Error Reporting Messages

Since IP is an unreliable protocol, one of the main responsibilities of ICMP is to report some errors that may occur during the processing of the IP datagram. ICMP does not correct errors, it simply reports them. Error correction is left to the higher-level protocols. Error messages are always sent to the original source because the only information

Figure 4.54 General format of ICMP messages



Type and code values

Error-reporting messages	Query messages
03: Destination unreachable (codes 0 to 15)	08 and 00: Echo request and reply (only code 0)
04: Source quench (only code 0)	13 and 14: Timestamp request and reply (only code 0)
05: Redirection (codes 0 to 3)	
11: Time exceeded (codes 0 and 1)	
12: Parameter problem (codes 0 and 1)	

Note: See the book website for more explanation about the code values.

available in the datagram about the route is the source and destination IP addresses. ICMP uses the source IP address to send the error message to the source (originator) of the datagram. To make the error-reporting process simple, ICMP follows some rules in reporting messages. First, no error message will be generated for a datagram having a multicast address or special address (such as *this host* or *loopback*). Second, no ICMP error message will be generated in response to a datagram carrying an ICMP error message. Third, no ICMP error message will be generated for a fragmented datagram that is not the first fragment.

Note that all error messages contain a data section that includes the IP header of the original datagram plus the first 8 bytes of data in that datagram. The original datagram header is added to give the original source, which receives the error message, information about the datagram itself. The 8 bytes of data are included because the first 8 bytes provide information about the port numbers (UDP and TCP) and sequence number (TCP). This information is needed so the source can inform the protocols (TCP or UDP) about the error.

The most widely used error message is the *destination unreachable* (type 3). This message uses different codes (0 to 15) to define the type of error message and the reason why a datagram has not reached its final destination. For example, code 0 tells the source that a host is unreachable. This may happen, for example, when we use the HTTP protocol to access a web page, but the server is down. The message “destination host is not reachable” is created and sent back to the source.

Another error message is called the *source quench* (type 4) message, which informs the sender that the network has encountered congestion and the datagram has been dropped; the source needs to slow down sending more datagrams. In other words, ICMP adds a kind of congestion control mechanism to the IP protocol by using this type of message.

The *redirection message* (type 5) is used when the source uses a wrong router to send out its message. The router redirects the message to the appropriate router, but

informs the source that it needs to change its default router in the future. The IP address of the default router is sent in the message.

We discussed the purpose of the *time-to-live* (TTL) field in the IP datagram and explained that it prevents a datagram from being aimlessly circulated in the Internet. When the TTL value becomes 0, the datagram is dropped by the visiting router and a *time exceeded* message (type 11) with code 0 is sent to the source to inform it about the situation. The time-exceeded message (with code 1) can also be sent when not all fragments of a datagram arrive within a predefined period of time.

Finally, a *parameter problem* message (type 12) can be sent when either there is a problem in the header of a datagram (code 0) or some options are missing or cannot be interpreted (code 1).

Query Messages

Interestingly, query messages in ICMP can be used independently without relation to an IP datagram. Of course, a query message needs to be encapsulated in a datagram, as a carrier. Query messages are used to probe or test the liveliness of hosts or routers in the internet, find the one-way or the round-trip time for an IP datagram between two devices, or even find out whether the clocks in two devices are synchronized. Naturally, query messages come in pairs: request and reply.

The *echo request* (type 8) and the *echo reply* (type 0) pair of messages are used by a host or a router to test the liveliness of another host or router. A host or router sends an echo request message to another host or router; if the latter is alive, it responds with an echo reply message. We shortly see the applications of this pair in two debugging tools: *ping* and *traceroute*.

The *timestamp request* (type 13) and the *timestamp reply* (type 14) pair of messages are used to find the round-trip time between two devices or to check whether the clocks in two devices are synchronized. The timestamp request message sends a 32-bit number, which defines the time the message is sent. The timestamp reply resends that number, but also includes two new 32-bit numbers representing the time the request was received and the time the response was sent. If all timestamps represent Universal time, the sender can calculate the one-way and round-trip time.

Example 4.13

One of the tools that a host can use to test the liveliness of another host is the *ping* program. The *ping* program takes advantage of the ICMP echo request and echo reply messages. A host can send an echo request (type 8, code 0) message to another host, which, if alive, can send back an echo reply (type 0, code 0) message. To some extent, the *ping* program can also measure the reliability and congestion of the router between the two hosts by sending a set of request-reply messages.

The following shows how we send a *ping* message to the auniversity.edu site. We set the identifier field in the echo request and reply message and start the sequence number from 0; this number is incremented by one each time a new message is sent. Note that *ping* can calculate the round-trip time. It inserts the sending time in the data section of the message. When the packet arrives, it subtracts the arrival time from the departure time to get the *round-trip time* (rtt).

```
$ ping auniversity.edu
```

```
PING auniversity.edu (152.181.8.3) 56 (84) bytes of data.
```

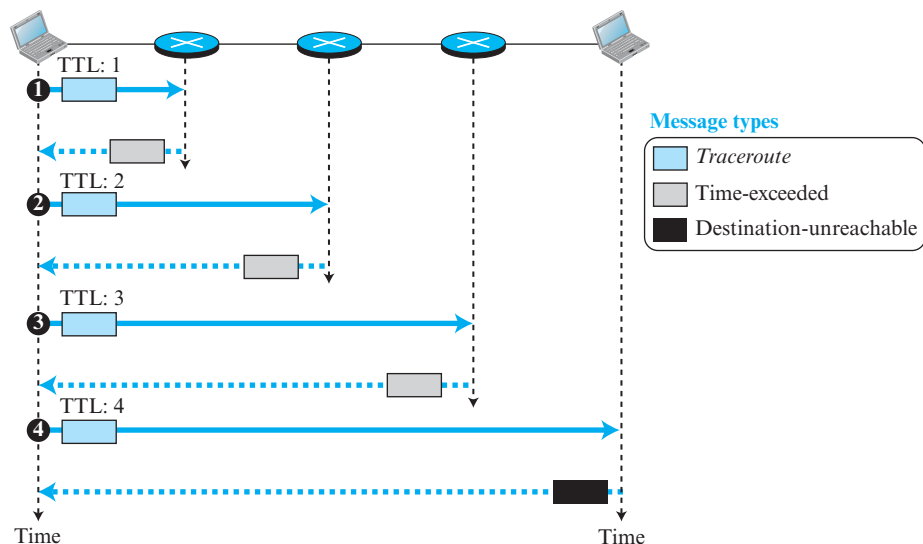
```
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=0    ttl=62    time=1.91 ms
```

```
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=1    ttl=62    time=2.04 ms
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=2    ttl=62    time=1.90 ms
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=3    ttl=62    time=1.97 ms
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=4    ttl=62    time=1.93 ms
64 bytes from auniversity.edu (152.181.8.3): icmp_seq=5    ttl=62    time=2.00 ms
--- auniversity.edu statistics ---
6 packets transmitted, 6 received, 0% packet loss
rtt min/avg/max = 1.90/1.95/2.04 ms
```

Example 4.14

The *traceroute* program in UNIX or *tracert* in Windows can be used to trace the path of a packet from a source to the destination. It can find the IP addresses of all the routers that are visited along the path. The program is usually set to check for the maximum of 30 hops (routers) to be visited. The number of hops in the Internet is normally less than this. The *traceroute* program is different from the *ping* program. The *ping* program gets help from two query messages; the *traceroute* program gets help from two *error-reporting* messages: time-exceeded and destination-unreachable. The *traceroute* is an application-layer program, but only the client program is needed, because, as we can see, the client program never reaches the application layer in the destination host. In other words, there is no *traceroute* server program. The *traceroute* application program is encapsulated in a UDP user datagram, but *traceroute* intentionally uses a port number that is not available at the destination. If there are n routers in the path, the *traceroute* program sends $(n + 1)$ messages. The first n messages are discarded by the n routers, one by each router; the last message is discarded by the destination host. The *traceroute* client program uses the $(n + 1)$ ICMP error-reporting messages received to find the path between the routers. We will show shortly that the *traceroute* program does not need to know the value of n ; it is found automatically. Figure 4.55 shows an example in which $n = 3$.

Figure 4.55 Example of traceroute program



The first *traceroute* message is sent with time-to-live (TTL) value set to 1; the message is discarded at the first router and a time-exceeded ICMP error message is sent, from which the *traceroute* program can find the IP address of the first router (the source IP address of the error message) and the router name (in the data section of the message). The second *traceroute* message is sent with TTL set to 2, which can find the IP address and the name of the second router. Similarly, the third message can find the information about router 3. The fourth message, however, reaches the destination host. This host is also dropped, but for another reason. The destination host cannot find the port number specified in the UDP user datagram. This time ICMP sends a different message, the destination-unreachable message with code 3 to show the port number is not found. After receiving this different ICMP message, the *traceroute* program knows that the final destination is reached. It uses the information in the received message to find the IP address and the name of the final destination.

The *traceroute* program also sets a timer to find the round-trip time for each router and the destination. Most *traceroute* programs send three messages to each device, with the same TTL value, to be able to find a better estimate for the round-trip time. The following shows an example of a *traceroute* program, which uses three probes for each device and gets three RTTs.

\$ traceroute printers.com

traceroute to printers.com (13.1.69.93), 30 hops max, 38 byte packets

1	route.front.edu	(153.18.31.254)	0.622 ms	0.891 ms	0.875 ms
2	ceneric.net	(137.164.32.140)	3.069 ms	2.875 ms	2.930 ms
3	satire.net	(132.16.132.20)	3.071 ms	2.876 ms	2.929 ms
4	alpha.printers.com	(13.1.69.93)	5.922 ms	5.048 ms	4.922 ms

Here there are three hops between source and destination. Note that some round-trip times look unusual. It could be that a router is too busy to process the packet immediately.

4.3 UNICAST ROUTING

In an internet, the goal of the network layer is to deliver a datagram from its source to its destination or destinations. If a datagram is destined for only one destination (one-to-one delivery), we have *unicast routing*. If the datagram is destined for several destinations (one-to-many delivery), we have *multicast routing*. Finally, if the datagram is supposed to be delivered to all hosts in the internet (one-to-all), we have *broadcast routing*. In this section and the next, we discuss only unicast routing; multicast and broadcast routing will be discussed later in the chapter.

Unicast routing in the Internet, with a large number of routers and a huge number of hosts, can be done only by using hierarchical routing: routing in several steps using different routing algorithms. In this section, we first discuss the general concept of unicast routing in an *internet*: an internetwork made of networks connected by routers. After the routing concepts and algorithms are understood, we show how we can apply them to the Internet using hierarchical routing.

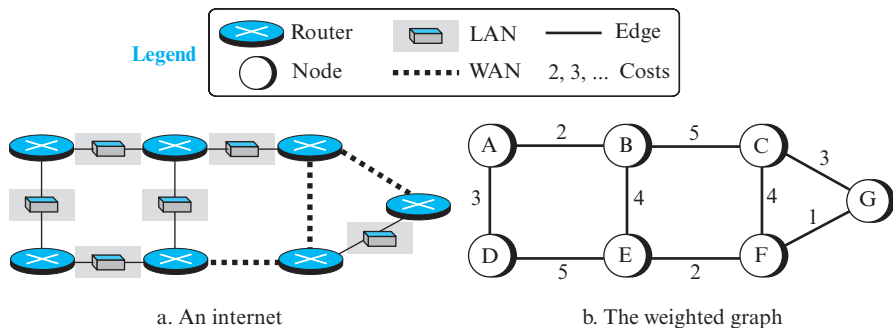
4.3.1 General Idea

In unicast routing, a packet is routed, hop by hop, from its source to its destination by the help of forwarding tables. The source host needs no forwarding table because it delivers its packet to the default router in its local network. The destination host needs no forwarding table either because it receives the packet from its default router in its local network. This means that only the routers that glue together the networks in the internet need forwarding tables. With the above explanation, routing a packet from its source to its destination means routing the packet from a *source router* (the default router of the source host) to a *destination router* (the router connected to the destination network). Although a packet needs to visit the source and the destination routers, the question is what other routers the packet should visit. In other words, there are several routes that a packet can travel from the source to the destination; what must be determined is which route the packet should take.

An Internet as a Graph

To find the best route, an internet can be modeled as a *graph*. A graph in computer science is a set of *nodes* and *edges* (lines) that connect the nodes. To model an internet as a graph, we can think of each router as a node and each network between a pair of routers as an edge. An internet is, in fact, modeled as a *weighted graph*, in which each edge is associated with a cost. If a weighted graph is used to represent a geographical area, the nodes can be cities and the edges can be roads connecting the cities; the weights, in this case, are distances between cities. In routing, however, the cost of an edge has a different interpretation in different routing protocols, which we discuss in a later section. For the moment, we assume that there is a cost associated with each edge. If there is no edge between the nodes, the cost is infinity. Figure 4.56 shows how an internet can be modeled as a graph.

Figure 4.56 An internet and its graphical representation



Least-Cost Routing

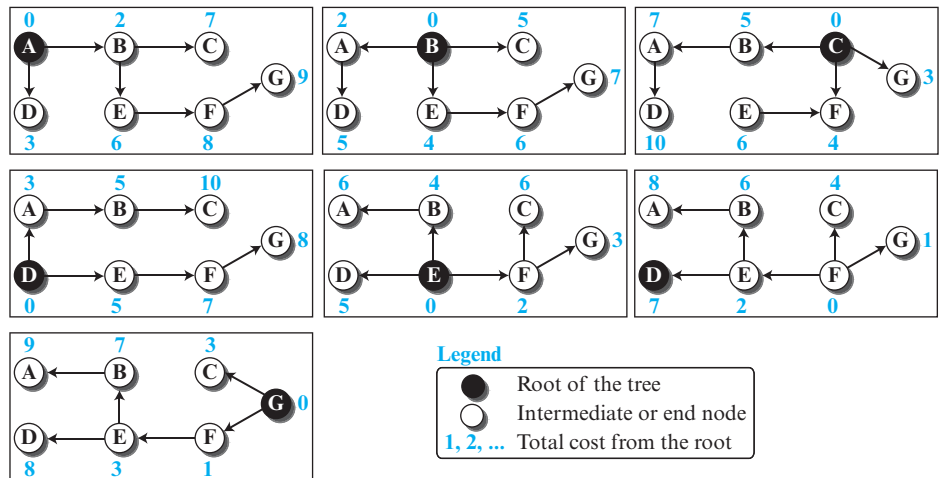
When an internet is modeled as a weighted graph, one of the ways to interpret the *best* route from the source router to the destination router is to find the *least cost* between the two. In other words, the source router chooses a route to the destination router in such a way that the total cost for the route is the least cost among all possible routes. In

Figure 4.56, the best route between A and E is A-B-E, with the cost of 6. This means that each router needs to find the least-cost route between itself and the all other routers to be able to route a packet using this criteria.

Least-Cost Trees

If there are N routers in an internet, there are $(N - 1)$ least-cost paths from each router to any other router. This means we need $N \times (N - 1)$ least-cost paths for the whole internet. If we have only 10 routers in an internet, we need 90 least-cost paths. A better way to see all of these paths is to combine them in a **least-cost tree**. A least-cost tree is a tree with the source router as the root that spans the whole graph (visits all other nodes) and in which the path between the root and any other node is the shortest. In this way, we can have only one shortest-path tree for each node; we have N least-cost path trees for the whole internet. We show how to create a least-cost tree for each node later in this section; for the moment, Figure 4.57 shows the seven least-cost trees for the internet in Figure 4.56.

Figure 4.57 Least-cost trees for nodes in the internet of Figure 4.56



The least-cost trees for a weighted graph can have several properties if they are created using consistent criteria.

1. The least-cost route from X to Y in X's tree is the inverse of the least cost route from Y to X in Y's tree; the cost in both directions is the same. For example, in Figure 4.57, the route from A to F in A's tree is (A → B → E → F), but the route from F to A in F's tree is (F → E → B → A), which is the inverse of the first route. The cost is 8 in each case.
2. Instead of travelling from X to Z using X's tree, we can travel from X to Y using X's tree and continue from Y to Z using Y's tree. For example, in Figure 4.57, we can go from A to G in A's tree using the route (A → B → E → F → G). We can also go from A to E in A's tree (A → B → E) and then continue in E's tree using the route (E → F → G). The combination of the two routes in the second case is the same route as in the first case. The cost in the first case is 9; the cost in the second case is also 9 (6 + 3).

4.3.2 Routing Algorithms

After discussing the general idea behind least-cost trees and the forwarding tables that can be made from them, now we concentrate on the routing algorithms. Several routing algorithms have been designed in the past. The differences between these methods are in the way they interpret the least cost and the way they create the least-cost tree for each node. In this section, we discuss the common algorithm; later we show how a routing protocol in the Internet implements one of these algorithms.

Distance-Vector Routing

The **distance-vector (DV) routing** uses the goal we discussed in the introduction to find the best route. In distance-vector routing, the first thing each node creates is its own least-cost tree with the rudimentary information it has about its immediate neighbors. The incomplete trees are exchanged between immediate neighbors to make the trees more and more complete and to represent the whole internet. We can say that in distance-vector routing, a router continuously tells all of its neighbors about what it knows about the whole internet (although the knowledge can be incomplete).

Before we show how incomplete least-cost trees can be combined to make complete ones, we need to discuss two important topics: the Bellman-Ford equation and the concept of distance vectors, which we cover next.

Bellman-Ford Equation

The heart of distance-vector routing is the famous **Bellman-Ford** equation. This equation is used to find the least cost (shortest distance) between a source node, x , and a destination node, y , through some intermediary nodes ($a, b, c, \dots$) when the costs between the source and the intermediary nodes and the least costs between the intermediary nodes and the destination are given. The following shows the general case in which D_{ij} is the shortest distance and c_{ij} is the cost between node i and j .

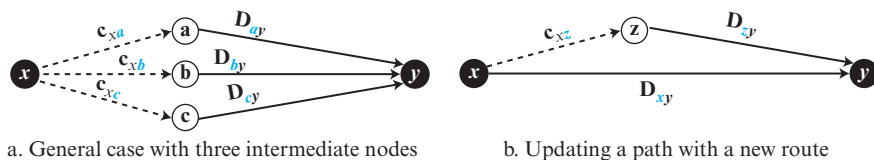
$$D_{xy} = \min \{ (c_{xa} + D_{ay}), (c_{xb} + D_{by}), (c_{xc} + D_{cy}), \dots \}$$

In distance-vector routing, normally we want to update an existing least cost with a least cost through an intermediary node, such as z , if the latter is shorter. In this case, the equation becomes simpler, as shown below:

$$D_{xy} = \min \{ D_{xy}, (c_{xz} + D_{zy}) \}$$

Figure 4.58 shows the idea graphically for both cases.

Figure 4.58 Graphical idea behind Bellman-Ford equation



We can say that the Bellman-Ford equation enables us to build a new least-cost path from previously-established least-cost paths. In Figure 4.58, we can think of $(a \rightarrow y)$,

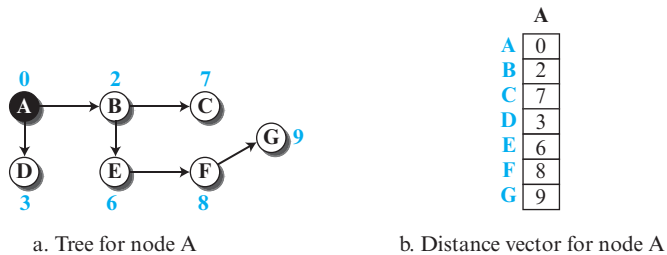
($b \rightarrow y$), and ($c \rightarrow y$) as previously-established least-cost paths and ($x \rightarrow y$) as the new least-cost path. We can even think of this equation as the builder of a new least-cost tree from previously established least-cost trees if we use the equation repeatedly. In other words, the use of this equation in distance-vector routing is a witness that this method also uses least-cost trees, but this use may be in the background.

We will shortly show how we use the Bellman-Ford equation and the concept of distance vectors to build least-cost paths for each node in distance-vector routing, but first we need to discuss the concept of a distance vector.

Distance Vectors

The concept of a **distance vector** is the rationale for the name *distance-vector routing*. A least-cost tree is a combination of least-cost paths from the root of the tree to all destinations. These paths are graphically glued together to form the tree. Distance-vector routing unglues these paths and creates a *distance vector*, a one-dimensional array to represent the tree. Figure 4.59 shows the tree for node A in the internet in Figure 4.56 and the corresponding distance vector.

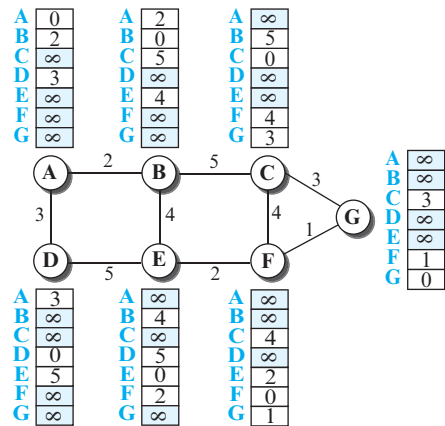
Figure 4.59 The distance vector corresponding to a tree



Note that the *name* of the distance vector defines the root, the *indexes* define the destinations, and the *value* of each cell defines the least cost from the root to the destination. A distance vector does not give the path to the destinations as the least-cost tree does; it gives only the least costs to the destinations. Later we show how we can change a distance vector to a forwarding table, but we first need to find all distance vectors for an internet.

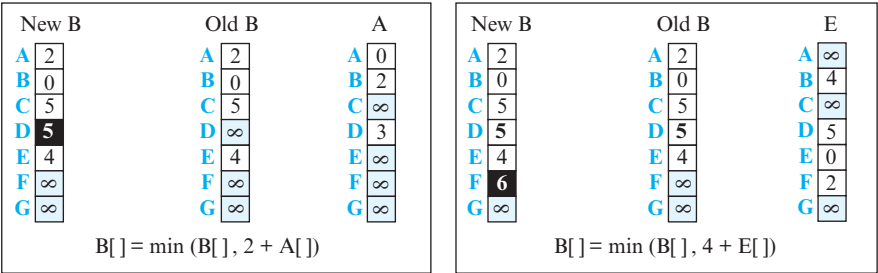
We know that a distance vector can represent least-cost paths in a least-cost tree, but the question is how each node in an internet originally creates the corresponding vector. Each node in an internet, when it is booted, creates a very rudimentary distance vector with the minimum information the node can obtain from its neighborhood. The node sends some greeting messages out of its interfaces and discovers the identity of the immediate neighbors and the distance between itself and each neighbor. It then makes a simple distance vector by inserting the discovered distances in the corresponding cells and leaves the value of other cells as infinity. Do these distance vectors represent least-cost paths? They do, considering the limited information a node has. When we know only one distance between two nodes, it is the least cost. Figure 4.60 shows all distance vectors for our internet. However, we need to mention that these vectors are made asynchronously, when the corresponding node has been booted; the existence of all of them in a figure does not mean synchronous creation of them.

Figure 4.60 The first distance vector for an internet



These rudimentary vectors cannot help the internet to effectively forward a packet. For example, node A thinks that it is not connected to node G because the corresponding cell shows the least cost of infinity. To improve these vectors, the nodes in the internet need to help each other by exchanging information. After each node has created its vector, it sends a copy of the vector to all its immediate neighbors. After a node receives a distance vector from a neighbor, it updates its distance vector using the Bellman-Ford equation (second case). However, we need to understand that we need to update, not only one least cost, but N of them in which N is the number of the nodes in the internet. If we are using a program, we can do this using a loop; if we are showing the concept on paper, we can show the whole vector instead of the N separate equations. We show the whole vector instead of seven equations for each update in Figure 4.61. The figure shows two asynchronous events, happening one after another with some time in

Figure 4.61 Updating distance vectors



a. First event: B receives a copy of A's vector.

b. Second event: B receives a copy of E's vector.

Note:
X[]: the whole vector

between. In the first event, node A has sent its vector to node B. Node B updates its vector using the cost $c_{BA} = 2$. In the second event, node E has sent its vector to node B. Node B updates its vector using the cost $c_{EA} = 4$.

After the first event, node B has one improvement in its vector: its least cost to node D has changed from infinity to 5 (via node A). After the second event, node B has one more improvement in its vector; its least cost to node F has changed from infinity to 6 (via node E). We hope that we have convinced the reader that exchanging vectors eventually stabilizes the system and allows all nodes to find the ultimate least cost between themselves and any other node. We need to remember that after updating a node, it immediately sends its updated vector to all neighbors. Even if its neighbors have received the previous vector, the updated one may help more.

Distance-Vector Routing Algorithm

Now we can give a simplified pseudocode for the distance-vector routing algorithm, as shown in Table 4.4. The algorithm is run by its node independently and asynchronously.

Table 4.4 *Distance-Vector Routing Algorithm for A Node*

1	Distance_Vector_Routing ()
2	{
3	<i>// Initialize (create initial vectors for the node)</i>
4	$D[\text{myself}] = 0$
5	for ($y = 1$ to N)
6	{
7	if (y is a neighbor)
8	$D[y] = c[\text{myself}][y]$
9	else
10	$D[y] = \infty$
11	}
12	send vector $\{D[1], D[2], \dots, D[N]\}$ to all neighbors
13	<i>// Update (improve the vector with the vector received from a neighbor)</i>
14	repeat (forever)
15	{
16	wait (for a vector D_w from a neighbor w or any change in the link)
17	for ($y = 1$ to N)
18	{
19	$D[y] = \min [D[y], (c[\text{myself}][w] + D_w[y])]$ <i>// Bellman-Ford equation</i>
20	}
21	if (any change in the vector)
22	send vector $\{D[1], D[2], \dots, D[N]\}$ to all neighbors
23	}
24	} <i>// End of Distance Vector</i>

Lines 4 to 11 initialize the vector for the node. Lines 14 to 23 show how the vector can be updated after receiving a vector from the immediate neighbor. The *for* loop in

lines 17 to 20 allows all entries (cells) in the vector to be updated after receiving a new vector. Note that the node sends its vector in line 12, after being initialized, and in line 22, after it is updated.

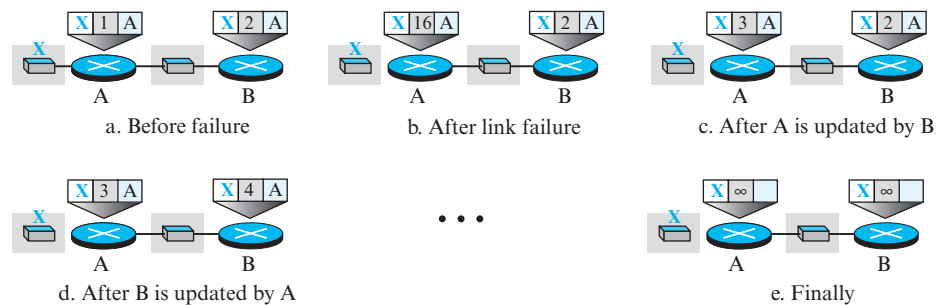
Count to Infinity

A problem with distance-vector routing is that any decrease in cost (good news) propagates quickly, but any increase in cost (bad news) will propagate slowly. For a routing protocol to work properly, if a link is broken (cost becomes infinity), every other router should be aware of it immediately, but in distance-vector routing, this takes some time. The problem is referred to as *count to infinity*. It sometimes takes several updates before the cost for a broken link is recorded as infinity by all routers.

Two-Node Loop

One example of count to infinity is the two-node loop problem. To understand the problem, let us look at the scenario depicted in Figure 4.62.

Figure 4.62 Two-node instability



The figure shows a system with three nodes. We have shown only the portions of the forwarding table needed for our discussion. At the beginning, both nodes A and B know how to reach node X. But suddenly, the link between A and X fails. Node A changes its table. If A can send its table to B immediately, everything is fine. However, the system becomes unstable if B sends its forwarding table to A before receiving A's forwarding table. Node A receives the update and, assuming that B has found a way to reach X, immediately updates its forwarding table. Now A sends its new update to B. Now B thinks that something has been changed around A and updates its forwarding table. The cost of reaching X increases gradually until it reaches infinity. At this moment, both A and B know that X cannot be reached. However, during this time the system is not stable. Node A thinks that the route to X is via B; node B thinks that the route to X is via A. If A receives a packet destined for X, the packet goes to B and then comes back to A. Similarly, if B receives a packet destined for X, it goes to A and comes back to B. Packets bounce between A and B, creating a two-node loop problem. A few solutions have been proposed for instability of this kind.

Split Horizon One solution to instability is called *split horizon*. In this strategy, instead of flooding the table through each interface, each node sends only part of its table through each interface. If, according to its table, node B thinks that the optimum

route to reach X is via A, it does not need to advertise this piece of information to A; the information has come from A (A already knows). Taking information from node A, modifying it, and sending it back to node A is what creates the confusion. In our scenario, node B eliminates the last line of its forwarding table before it sends it to A. In this case, node A keeps the value of infinity as the distance to X. Later, when node A sends its forwarding table to B, node B also corrects its forwarding table. The system becomes stable after the first update: both node A and B know that X is not reachable.

Poisoned Reverse Using the split-horizon strategy has one drawback. Normally, the corresponding protocol uses a timer, and if there is no news about a route, the node deletes the route from its table. When node B in the previous scenario eliminates the route to X from its advertisement to A, node A cannot guess whether this is due to the split-horizon strategy (the source of information was A) or because B has not received any news about X recently. In the poisoned reverse strategy B can still advertise the value for X, but if the source of information is A, it can replace the distance with infinity as a warning: “Do not use this value; what I know about this route comes from you.”

Three-Node Instability

The two-node instability can be avoided using split horizon combined with poisoned reverse. However, if the instability is between three nodes, stability cannot be guaranteed.

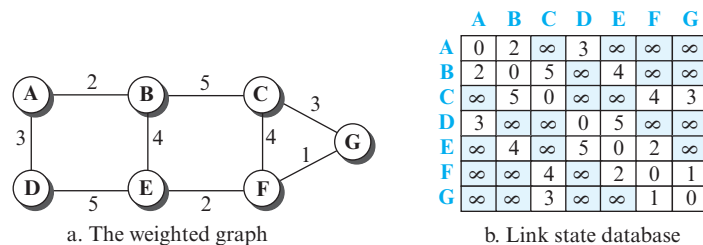
Link-State Routing

A routing algorithm that directly follows our discussion for creating least-cost trees and forwarding tables is **link-state (LS) routing**. This method uses the term *link-state* to define the characteristic of a link (an edge) that represents a network in the internet. In this algorithm the cost associated with an edge defines the state of the link. Links with lower costs are preferred to links with higher costs; if the cost of a link is infinity, it means that the link does not exist or has been broken.

Link-State Database (LSDB)

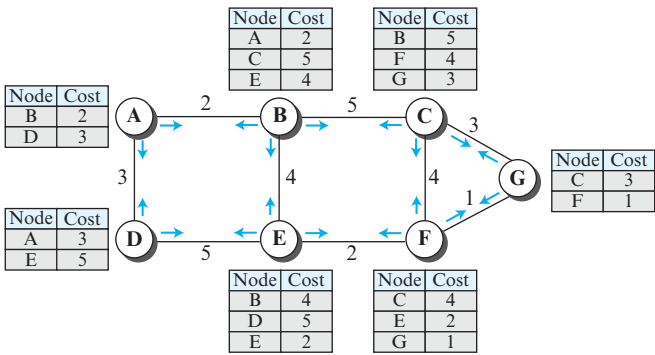
To create a least-cost tree with this method, each node needs to have a complete *map* of the network, which means it needs to know the state of each link. The collection of states for all links is called the **link-state database (LSDB)**. There is only one LSDB for the whole internet; each node needs to have a duplicate of it to be able to create the least-cost tree. Figure 4.63 shows an example of an LSDB for the graph in Figure 4.56. The LSDB can be represented as a two-dimensional array (matrix) in which the value of each cell defines the cost of the corresponding link.

Figure 4.63 Example of a link-state database



Now the question is how each node can create this LSDB that contains information about the whole internet. This can be done by a process called **flooding**. Each node can send some greeting messages to all its immediate neighbors (those nodes to which it is connected directly) to collect two pieces of information for each neighboring node: the identity of the node and the cost of the link. The combination of these two pieces of information is called the *LS packet* (LSP); the LSP is sent out of each interface, as shown in Figure 4.64 for our internet in Figure 4.56. When a node receives an LSP from one of its interfaces, it compares the LSP with the copy it may already have. If the newly arrived LSP is older than the one it has (found by checking the sequence number), it discards the LSP. If it is newer or the first one received, the node discards the old LSP (if there is one) and keeps the received one. It then sends a copy of it out of each interface except the one from which the packet arrived. This guarantees that flooding stops somewhere in the network (where a node has only one interface). We need to convince ourselves that, after receiving all new LSPs, each node creates the comprehensive LSDB as shown in Figure 4.64. This LSDB is the same for each node and shows the whole map of the internet. In other words, a node can make the whole map if it needs to, using this LSDB.

Figure 4.64 LSPs created and sent out by each node to build LSDB



We can compare the link-state routing algorithm with the distance-vector routing algorithm. In the distance-vector routing algorithm, each router tells its neighbors what it knows about the whole internet; in the link-state routing algorithm, each router tells the whole internet what it knows about its neighbors.

Formation of Least-Cost Trees

To create a least-cost tree for itself, using the shared LSDB, each node needs to run the famous **Dijkstra Algorithm**. This iterative algorithm uses the following steps:

1. The node chooses itself as the root of the tree, creating a tree with a single node, and sets the total cost of each node based on the information in the LSDB.
2. The node selects one node, among all nodes not in the tree, which is closest to the root, and adds this to the tree. After this node is added to the tree, the cost of all other nodes not in the tree needs to be updated because the paths may have been changed.
3. The node repeats step 2 until all nodes are added to the tree.

We need to convince ourselves that the above three steps finally create the least-cost tree. Table 4.5 shows a simplified version of Dijkstra's algorithm.

Table 4.5 *Dijkstra's Algorithm*

```

1  Dijkstra's Algorithm ( )
2  {
3      // Initialization
4      Tree = {root}           // Tree is made only of the root
5      for (y = 1 to N)        // N is the number of nodes
6      {
7          if (y is the root)
8              D[y] = 0         // D[y] is shortest distance from root to node y
9          else if (y is a neighbor)
10             D[y] = c[root][y] // c[x][y] is cost between nodes x and y in LSDB
11          else
12             D[y] = ∞
13      }
14      // Calculation
15      repeat
16      {
17          find a node w, with D[w] minimum among all nodes not in the Tree
18          Tree = Tree ∪ {w}      // Add w to tree
19          // Update distances for all neighbor of w
20          for (every node x, which is neighbor of w and not in the Tree)
21          {
22              D[x] = min{D[x], (D[w] + c[w][x])}
23          }
24      } until (all nodes included in the Tree)
25 } // End of Dijkstra

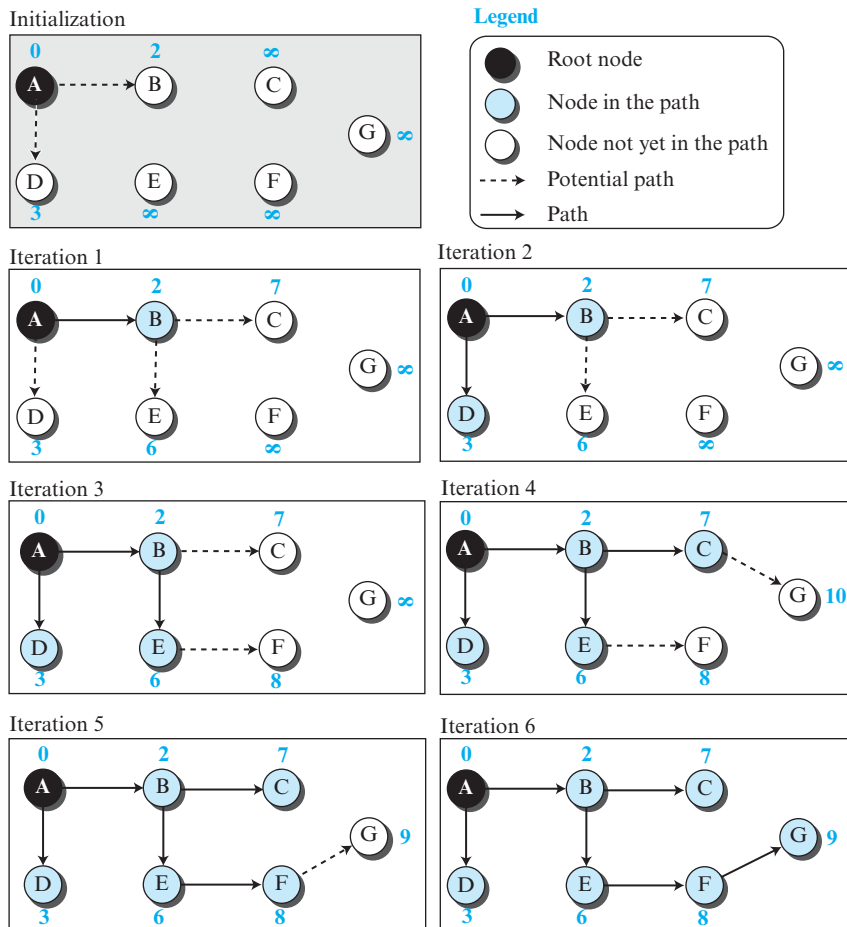
```

Lines 4 to 13 implement step 1 in the algorithm. Lines 16 to 23 implement step 2 in the algorithm. Step 2 is repeated until all nodes are added to the tree.

Figure 4.65 shows the formation of the least-cost tree for the graph in Figure 4.63 using the Dijkstra algorithm. We need to go through an initialization step and six iterations to find the least-cost tree.

Path-Vector Routing

Both link-state and distance-vector routing are based on the least-cost goal. However, there are instances where this goal is not the priority. For example, assume that there are some routers in the internet that a sender wants to prevent its packets from going through. For example, a router may belong to an organization that does not provide enough security or that belongs to a commercial rival of the sender which might inspect the packets

Figure 4.65 *Least-cost tree*

for obtaining information. Least-cost routing does not prevent a packet from passing through an area when that area is in the least-cost path. In other words, the least-cost goal, applied by LS or DV routing, does not allow a sender to apply specific policies to the route a packet may take. Aside from safety and security, there are occasions, as discussed in the next section, in which the mere goal of routing is reachability: to allow the packet to reach its destination more efficiently without assigning costs to the route.

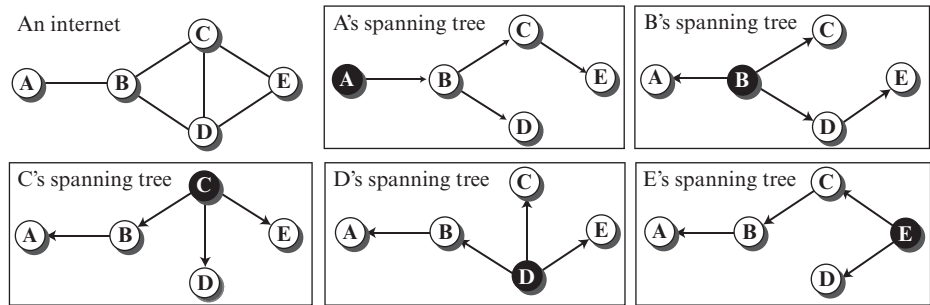
To respond to these demands, a third routing algorithm, called **path-vector (PV) routing** has been devised. Path-vector routing does not have the drawbacks of LS or DV routing as described above because it is not based on least-cost routing. The best route is determined by the source using the policy it imposes on the route. In other words, the source can control the path. Although path-vector routing is not actually used in an internet, and is mostly designed to route a packet between ISPs, we discuss the principle of this method in this section as though applied to an internet. In the next section, we show how it is used in the Internet.

Spanning Trees

In path-vector routing, the path from a source to all destinations is also determined by the *best* spanning tree. The best spanning tree, however, is not the least-cost tree; it is the tree determined by the source when it imposes its own policy. If there is more than one route to a destination, the source can choose the route that meets its policy best. A source may apply several policies at the same time. One of the common policies uses the minimum number of nodes to be visited (something similar to least-cost). Another common policy is to avoid some nodes as the middle node in a route.

Figure 4.66 shows a small internet with only five nodes. Each source has created its own spanning tree that meets its policy. The policy imposed by all sources is to use the minimum number of nodes to reach a destination. The spanning tree selected by A and E is such that the communication does not pass through D as a middle node. Similarly, the spanning tree selected by B is such that the communication does not pass through C as a middle node.

Figure 4.66 Spanning trees in path-vector routing



Creation of Spanning Trees

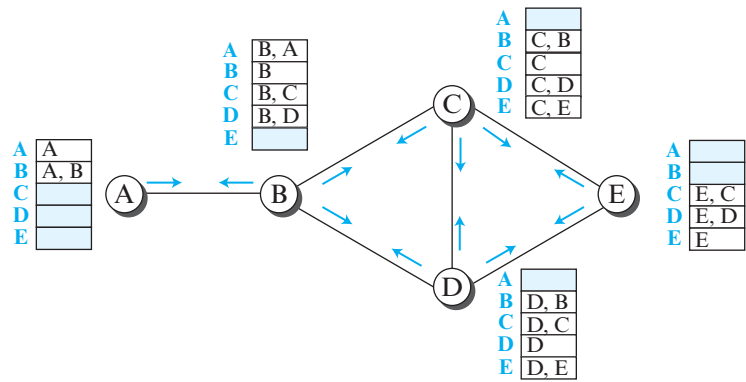
Path-vector routing, like distance-vector routing, is an asynchronous and distributed routing algorithm. The spanning trees are made, gradually and asynchronously, by each node. When a node is booted, it creates a *path vector* based on the information it can obtain about its immediate neighbor. A node sends greeting messages to its immediate neighbors to collect these pieces of information. Figure 4.67 shows all of these path vectors for our internet in Figure 4.66. Note, however, that we do not mean that all of these tables are created simultaneously; they are created when each node is booted. The figure also shows how these path-vectors are sent to immediate neighbors after they have been created (arrows).

Each node, after the creation of the initial path vector, sends it to all its immediate neighbors. Each node, when it receives a path vector from a neighbor, updates its path vector using an equation similar to the Bellman-Ford, but applying its own policy instead of looking for the least cost. We can define this equation as

$$\text{Path}(x, y) = \text{best} \{ \text{Path}(x, y), [(x + \text{Path}(v, y))] \} \quad \text{for all } v\text{'s in the internet.}$$

In this equation, the operator (+) means to add x to the beginning of the path. We also need to be cautious to avoid adding a node to an empty path because an empty path means one that does not exist.

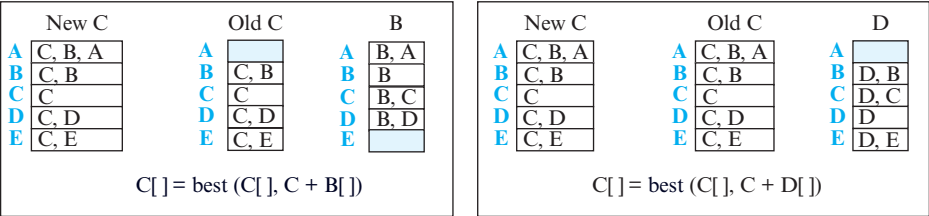
Figure 4.67 Path vectors made at booting time



The policy is defined by selecting the *best* of multiple paths. Path-vector routing also imposes one more condition on this equation: If Path (x, y) includes x, that path is discarded to avoid a loop in the path. In other words, x does not want to visit itself when it selects a path to y.

Figure 4.68 shows the path vector of node C after two events. In the first event, node C receives a copy of B's vector, which improves its vector: now it knows how to reach node A. In the second event, node C receives a copy of D's vector, which does not change its vector. As a matter of fact the vector for node C after the first event is stabilized and serves as its forwarding table.

Figure 4.68 Updating path vectors



Note:
X []: vector X
Y: node Y

Path-Vector Algorithm

Based on the initialization process and the equation used in updating each forwarding table after receiving path vectors from neighbors, we can write a simplified version of the path vector algorithm as shown in Table 4.6.

Table 4.6 Path-vector algorithm for a node

```

1  Path_Vector_Routing ( )
2  {
3      // Initialization
4      for (y = 1 to N)
5      {
6          if (y is myself)
7              Path[y] = myself
8          else if (y is a neighbor)
9              Path[y] = myself + neighbor node
10         else
11             Path[y] = empty
12     }
13     Send vector {Path[1], Path[2], ..., Path[y]} to all neighbors
14     // Update
15     repeat (forever)
16     {
17         wait (for a vector Pathw from a neighbor w)
18         for (y = 1 to N)
19         {
20             if (Pathw includes myself)
21                 discard the path // Avoid any loop
22             else
23                 Path[y] = best {Path[y], (myself + Pathw[y])}
24         }
25         If (there is a change in the vector)
26             Send vector {Path[1], Path[2], ..., Path[y]} to all neighbors
27     }
28 } // End of Path Vector

```

Lines 4 to 12 show the initialization for the node. Lines 17 to 24 show how the node updates its vector after receiving a vector from the neighbor. The update process is repeated forever. We can see the similarities between this algorithm and the DV algorithm.

4.3.3 Unicast Routing Protocols

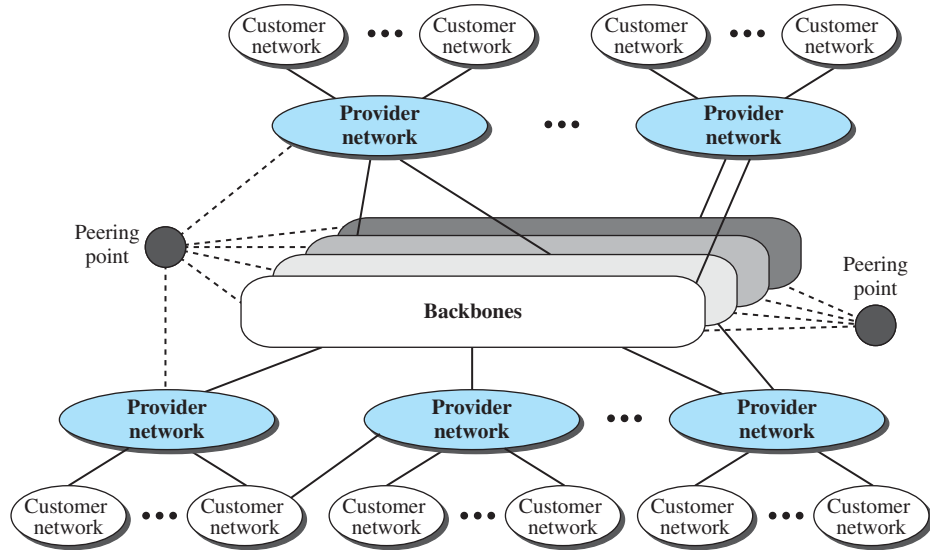
In the previous section, we discussed unicast routing algorithms; in this section, we discuss unicast routing protocols used in the Internet. Although three protocols we discuss here are based on the corresponding algorithms we discussed before, a protocol is more than an algorithm. A protocol needs to define its domain of operation, the messages exchanged, communication between routers, and interaction with protocols in other

domains. After an introduction, we discuss three common protocols used in the Internet: Routing Information Protocol (RIP), based on the distance-vector algorithm, Open Shortest Path First (OSPF), based on the link-state algorithm, and Border Gateway Protocol (BGP), based on the path-vector algorithm.

Internet Structure

Before discussing unicast routing protocols, we need to understand the structure of today's Internet. The Internet has changed from a tree-like structure, with a single backbone, to a multi-backbone structure run by different private corporations today. Although it is difficult to give a general view of the Internet today, we can say that the Internet has a structure similar to what is shown in Figure 4.69.

Figure 4.69 Internet structure



There are several *backbones* run by private communication companies that provide global connectivity. These backbones are connected by some *peering points* that allow connectivity between backbones. At a lower level, there are some *provider networks* that use the backbones for global connectivity but provide services to Internet customers. Finally, there are some *consumer networks* that use the services provided by the provider networks. Any of these three entities (backbone, provider network, or customer network) can be called an Internet Service Provider or ISP. They provide services, but at different levels.

Hierarchical Routing

The Internet today is made of a huge number of networks and routers that connect them. It is obvious that routing in the Internet cannot be done using one single protocol for two reasons: a scalability problem and an administrative issue. *Scalability problem* means that the size of the forwarding tables becomes huge, searching for a destination

in a forwarding table becomes time-consuming, and updating creates a huge amount of traffic. The *administrative issue* is related to the Internet structure described in Figure 4.69. As the figure shows, each ISP is run by an administrative authority. The administrator needs to have control in its system. The organization must be able to use as many subnets and routers as it needs, may desire that the routers be from a particular manufacturer, may wish to run a specific routing algorithm to meet the needs of the organization, and may want to impose some policy on the traffic passing through its ISP.

Hierarchical routing means considering each ISP as an **autonomous system (AS)**. Each AS can run a routing protocol that meets its needs, but the global Internet runs a global protocol to glue all ASs together. The routing protocol run in each AS is referred to as *intra-AS routing protocol*, *intradomain routing protocol*, or *interior gateway protocol (IGP)*; the global routing protocol is referred to as *inter-AS routing protocol*, *interdomain routing protocol*, or *exterior gateway protocol (EGP)*. We can have several intradomain routing protocols, and each AS is free to choose one, but it should be clear that we should have only one interdomain protocol that handles routing between these entities. Presently, the two common intradomain routing protocols are RIP and OSPF; the only interdomain routing protocol is BGP. The situation may change when we move to IPv6.

Autonomous Systems

As we said before, each ISP is an autonomous system when it comes to managing networks and routers under its control. Although we may have small, medium-size, and large ASs, each AS is given an autonomous number (ASN) by the ICANN. Each ASN is a 16-bit unsigned integer that uniquely defines an AS. The autonomous systems, however, are not categorized according to their size; they are categorized according to the way they are connected to other ASs. We have stub ASs, multihomed ASs, and transient ASs. The type, as we see will later, affects the operation of the interdomain routing protocol in relation to that AS.

- ❑ **Stub AS.** A stub AS has only one connection to another AS. The data traffic can be either initiated or terminated in a stub AS; the data cannot pass through it. A good example of a stub AS is the customer network which is either the source or the sink of data.
- ❑ **Multihomed AS.** A multihomed AS can have more than one connection to other ASs, but it does not allow data traffic to pass through it. A good example of such an AS is some of the customer ASs that may use the services of more than one provider network, but their policy does not allow data to be passed through them.
- ❑ **Transient AS.** A transient AS is connected to more than one other ASs and also allows the traffic to pass through. The provider networks and the backbone are good examples of transient ASs.

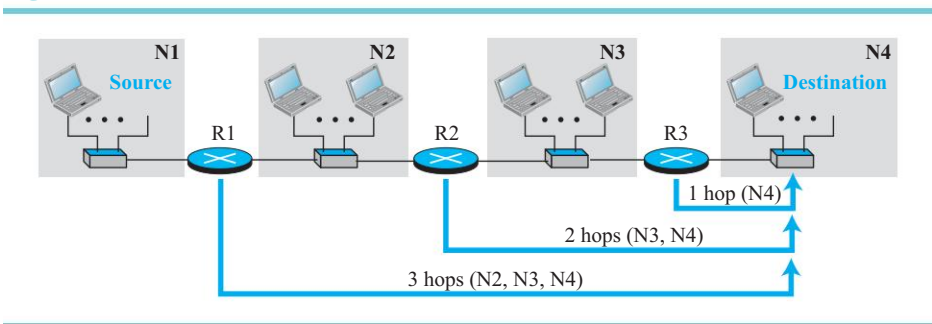
Routing Information Protocol (RIP)

The **Routing Information Protocol (RIP)** is one of the most widely used intradomain routing protocols based on the distance-vector routing algorithm we described earlier. RIP was started as part of the Xerox Network System (XNS), but it was the Berkeley Software Distribution (BSD) version of UNIX that helped make the use of RIP widespread.

Hop Count

A router in this protocol basically implements the distance-vector routing algorithm shown in Table 4.4. However, the algorithm has been modified as described below. First, since a router in an AS needs to know how to forward a packet to different networks (subnets) in an AS, RIP routers advertise the cost of reaching different networks instead of reaching other nodes in a theoretical graph. In other words, the cost is defined between a router and the network in which the destination host is located. Second, to make the implementation of the cost simpler (independent from performance factors of the routers and links, such as delay, bandwidth, and so on), the cost is defined as the number of hops, which means the number of networks (subnets) a packet needs to travel through from the source router to the final destination host. Note that the network in which the source host is connected is not counted in this calculation because the source host does not use a forwarding table; the packet is delivered to the default router. Figure 4.70 shows the concept of hop count advertised by three routers from a source host to a destination host. In RIP, the maximum cost of a path can be 15, which means 16 is considered as infinity (no connection). For this reason, RIP can be used only in autonomous systems in which the diameter of the AS is not more than 15 hops.

Figure 4.70 Hop counts in RIP



Forwarding Tables

Although the distance-vector algorithm we discussed in the previous section is concerned with exchanging distance vectors between neighboring nodes, the routers in an autonomous system need to keep forwarding tables to forward packets to their destination networks. A forwarding table in RIP is a three-column table in which the first column is the address of the destination network, the second column is the address of the next router to which the packet should be forwarded, and the third column is the cost (the number of hops) to reach the destination network. Figure 4.71 shows the three forwarding tables for the routers in Figure 4.70. Note that the first and the third columns together convey the same information as does a distance vector, but the cost shows the number of hops to the destination networks.

Although a forwarding table in RIP defines only the next router in the second column, it gives the information about the whole least-cost tree based on the second property of these trees, discussed in the previous section. For example, R1 defines that the next router for the path to N4 is R2; R2 defines that the next router to N4 is

Figure 4.71 Forwarding tables

Forwarding table for R1			Forwarding table for R2			Forwarding table for R3		
Destination network	Next router	Cost in hops	Destination network	Next router	Cost in hops	Destination network	Next router	Cost in hops
N1	—	1	N1	R1	2	N1	R2	3
N2	—	1	N2	—	1	N2	R2	2
N3	R2	2	N3	—	1	N3	—	1
N4	R2	3	N4	R3	2	N4	—	1

R3; R3 defines that there is no next router for this path. The tree is then $R1 \rightarrow R2 \rightarrow R3 \rightarrow N4$.

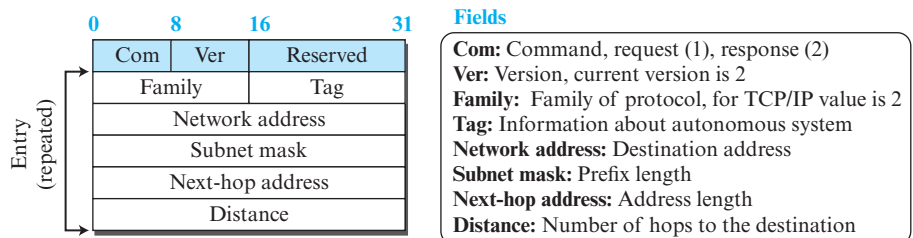
A question often asked about the forwarding table is what the use of the third column is. The third column is not needed for forwarding the packet, but it is needed for updating the forwarding table when there is a change in the route, as we will see shortly.

RIP Implementation

RIP is implemented as a process that uses the service of UDP on the well-known port number 520. In BSD, RIP is a daemon process (a process running at the background), named *routed* (abbreviation for *route daemon* and pronounced *route-dee*). This means that, although RIP is a routing protocol to help IP to route its datagrams through the AS, the RIP messages are encapsulated inside UDP user datagrams, which in turn are encapsulated inside IP datagrams. In other words, RIP runs at the application layer, but creates forwarding tables for IP at the network layer.

RIP has gone through two versions: RIP-1 and RIP-2. The second version is backward compatible with the first version; it allows the use of more information in the RIP messages that were set to 0 in the first version. We discuss only RIP-2 in this section.

RIP Messages Two RIP processes, a client and a server, like any other processes, need to exchange messages. RIP-2 defines the format of the message, as shown in Figure 4.72. Part of the message, which we call entry, can be repeated as needed in a message. Each entry carries the information related to one line in the forwarding table of the router that sends the message.

Figure 4.72 RIP message format

RIP has two types of messages: request and response. A request message is sent by a router that has just come up or by a router that has some time-out entries. A request message can ask about specific entries or all entries. A response (or update) message can be either solicited or unsolicited. A solicited response message is sent only in answer to a request message. It contains information about the destination specified in the corresponding request message. An unsolicited response message, on the other hand, is sent periodically, every 30 seconds or when there is a change in the forwarding table.

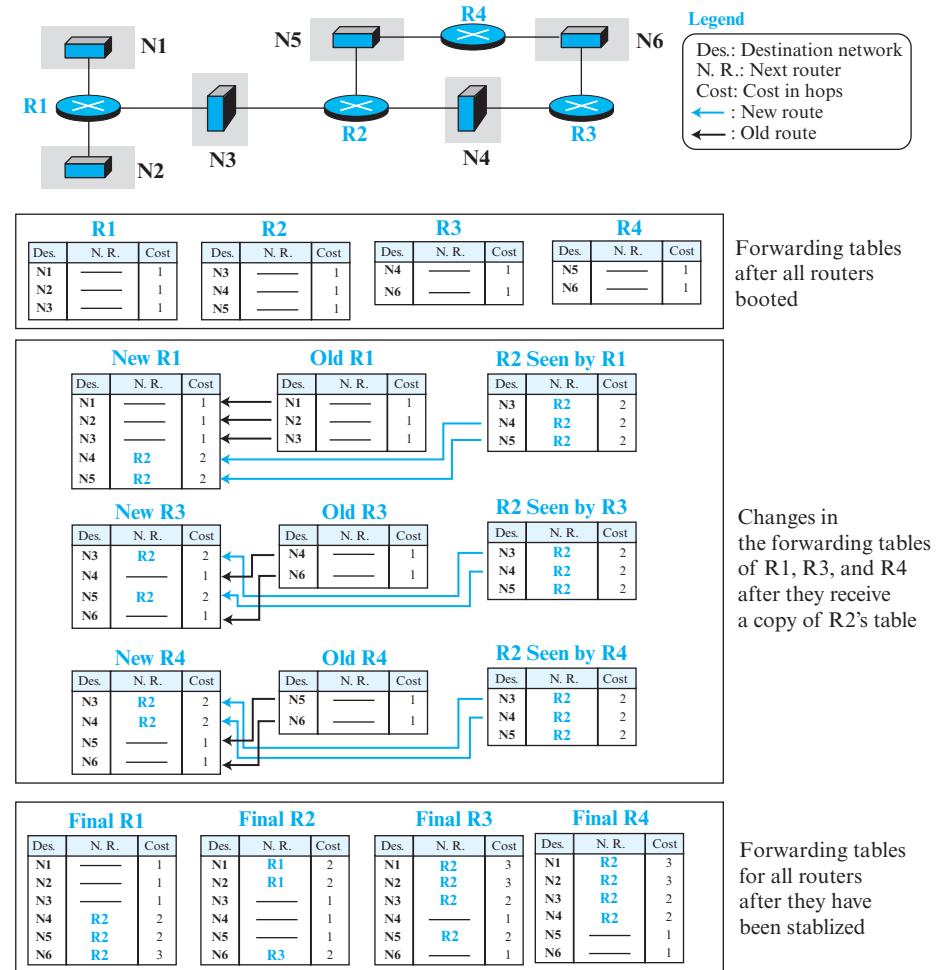
RIP Algorithm RIP implements the same algorithm as the distance-vector routing algorithm we discussed in the previous section. However, some changes need to be made to the algorithm to enable a router to update its forwarding table:

- ❑ Instead of sending only distance vectors, a router needs to send the whole contents of its forwarding table in a response message.
- ❑ The receiver adds one hop to each cost and changes the next router field to the address of the sending router. We call each route in the modified forwarding table the *received route* and each route in the old forwarding table the *old route*. The received router select the old routes as the new ones except in the following three cases:
 1. If the received route does not exist in the old forwarding table, it should be added to the route.
 2. If the cost of the received route is lower than the cost of the old one, the received route should be selected as the new one.
 3. If the cost of the received route is higher than the cost of the old one, but the value of the next router is the same in both routes, the receive route should be selected as the new one. This is the case where the route was actually advertised by the same router in the past, but now the situation has been changed. For example, suppose a neighbor has previously advertised a route to a destination with cost 3, but now there is no path between this neighbor and that destination. The neighbor advertises this destination with cost value infinity (16 in RIP). The receiving router must not ignore this value even though its old route has a lower cost to the same destination.
- ❑ The new forwarding table needs to be sorted according to the destination route (mostly using the longest prefix first).

Example 4.15

Figure 4.73 shows a more realistic example of the operation of RIP in an autonomous system. First, the figure shows all forwarding tables after all routers have been booted. Then we show changes in some tables when some update messages have been exchanged. Finally, we show the stabilized forwarding tables when there is no more change.

Timers in RIP RIP uses three timers to support its operation. The *periodic timer* controls the advertising of regular update messages. Each router has one periodic timer that is randomly set to a number between 25 and 35 seconds (to prevent all routers sending their messages at the same time and creating excess traffic). The timer counts down; when zero is reached, the update message is sent, and the timer is randomly set once

Figure 4.73 Example of an autonomous system using RIP

again. The *expiration timer* governs the validity of a route. When a router receives update information for a route, the expiration timer is set to 180 seconds for that particular route. Every time a new update for the route is received, the timer is reset. If there is a problem on an internet and no update is received within the allotted 180 seconds, the route is considered expired and the hop count of the route is set to 16, which means the destination is unreachable. Every route has its own expiration timer. The *garbage collection timer* is used to purge a route from the forwarding table. When the information about a route becomes invalid, the router does not immediately purge that route from its table. Instead, it continues to advertise the route with a metric value of 16. At the same time, a garbage collection timer is set to 120 seconds for that route. When the count reaches zero, the route is purged from the table. This timer allows neighbors to become aware of the invalidity of a route prior to purging.

Performance

Before ending this section, let us briefly discuss the performance of RIP:

- ❑ **Update Messages.** The update messages in RIP have a very simple format and are sent only to neighbors; they are local. They do not normally create traffic because the routers try to avoid sending them at the same time.
- ❑ **Convergence of Forwarding Tables.** RIP uses the distance-vector algorithm, which can converge slowly if the domain is large, but, since RIP allows only 15 hops in a domain (16 is considered as infinity), there is normally no problem in convergence. The only problems that may slow down convergence are count-to-infinity and loops created in the domain; use of poison-reverse and split-horizon strategies added to the RIP extension may alleviate the situation.
- ❑ **Robustness.** As we said before, distance-vector routing is based on the concept that each router sends what it knows about the whole domain to its neighbors. This means that the calculation of the forwarding table depends on information received from immediate neighbors, which in turn receive their information from their own neighbors. If there is a failure or corruption in one router, the problem will be propagated to all routers and the forwarding in each router will be affected.

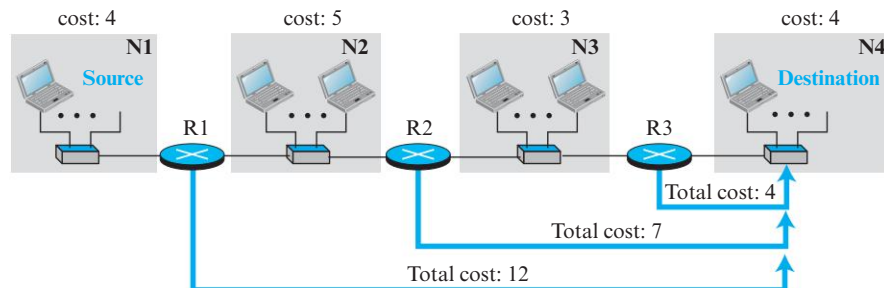
Open Shortest Path First (OSPF)

Open Shortest Path First (OSPF) is also an intradomain routing protocol like RIP, but it is based on the link-state routing protocol we described earlier in the chapter. OSPF is an *open* protocol, which means that the specification is a public document.

Metric

In OSPF, like RIP, the cost of reaching a destination from the host is calculated from the source router to the destination network. However, each link (network) can be assigned a weight based on the throughput, round-trip time, reliability, and so on. An administration can also decide to use the hop count as the cost. An interesting point about the cost in OSPF is that different service types (TOSs) can have different weights as the cost. Figure 4.74 shows the idea of the cost from a router to the destination host network. We can compare the figure with Figure 4.70 for the RIP.

Figure 4.74 Metric in OSPF



Forwarding Tables

Each OSPF router can create a forwarding table after finding the shortest-path tree between itself and the destination using Dijkstra's algorithm, described earlier in the chapter. Figure 4.75 shows the forwarding tables for the simple AS in Figure 4.74. Comparing the forwarding tables for the OSPF and RIP in the same AS, we find that the only difference is the cost values. In other words, if we use the hop count for OSPF, the tables will be exactly the same. The reason for this consistency is that both protocols use the shortest-path trees to define the best route from a source to a destination.

Figure 4.75 Forwarding tables in OSPF

Forwarding table for R1			Forwarding table for R2			Forwarding table for R3		
Destination network	Next router	Cost	Destination network	Next router	Cost	Destination network	Next router	Cost
N1	—	4	N1	R1	9	N1	R2	12
N2	—	5	N2	—	5	N2	R2	8
N3	R2	8	N3	—	3	N3	—	3
N4	R2	12	N4	R3	7	N4	—	4

Areas

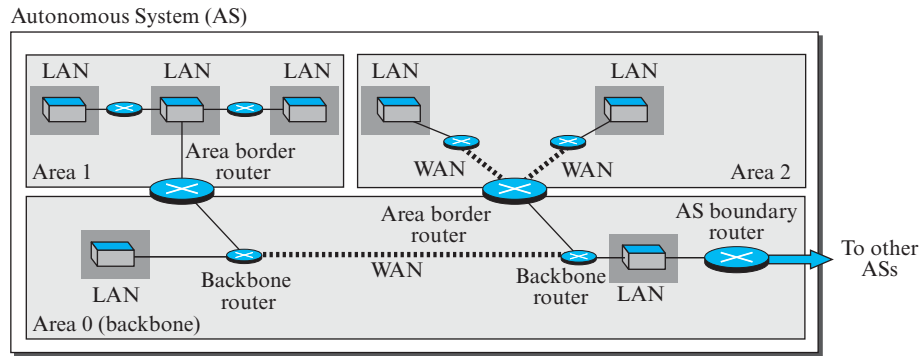
Compared with RIP, which is normally used in small ASs, OSPF was designed to be able to handle routing in a small or large autonomous system. However, the formation of shortest-path trees in OSPF requires that all routers flood the whole AS with their LSPs to create the global LSDB. Although this may not create a problem in a small AS, it may have created a huge volume of traffic in a large AS. To prevent this, the AS needs to be divided into small sections called *areas*. Each area acts as a small independent domain for flooding LSPs. In other words, OSPF uses another level of hierarchy in routing: the first level is the autonomous system, the second is the area.

However, each router in an area needs to know the information about the link states not only in its area but also in other areas. For this reason, one of the areas in the AS is designated as the *backbone area*, responsible for gluing the areas together. The routers in the backbone area are responsible for passing the information collected by each area to all other areas. In this way, a router in an area can receive all LSPs generated in other areas. For the purpose of communication, each area has an area identification. The area identification of the backbone is zero. Figure 4.76 shows an autonomous system and its areas.

Link-State Advertisement

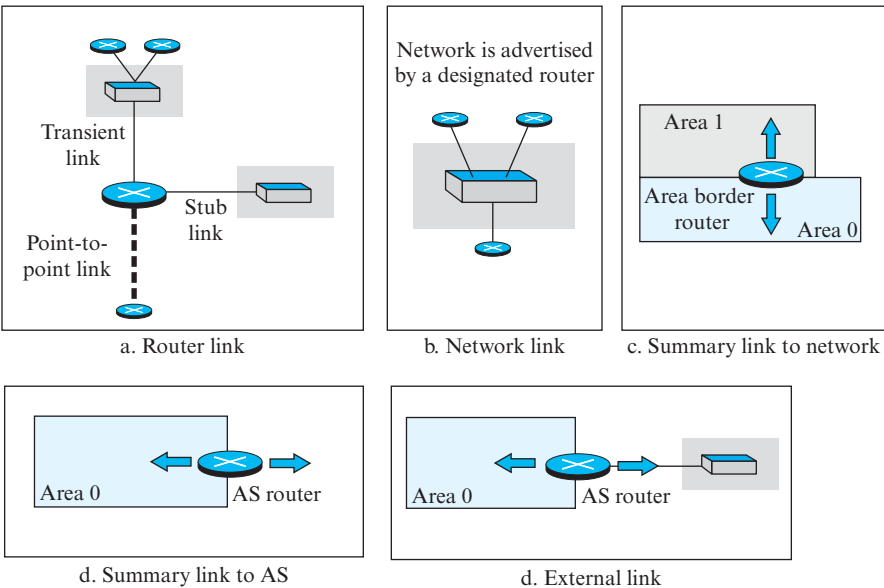
OSPF is based on the link-state routing algorithm, which requires that a router advertise the state of each link to all neighbors for the formation of the LSDB. When we discussed the link-state algorithm, we used the graph theory and assumed that each router is a node and each network between two routers is an edge. The situation is different in the real world, in which we need to advertise the existence of different entities as nodes, the different types of links that connect each node to its neighbors, and the different types of cost associated with each link. This means we need different types of advertisements, each capable of advertising different situations. We can have five types of

Figure 4.76 Areas in an autonomous system



link-state advertisements: *router link*, *network link*, *summary link to network*, *summary link to AS border router*, and *external link*. Figure 4.77 shows these five advertisements and their uses.

Figure 4.77 Five different LSPs



❑ **Router link.** A router link advertises the existence of a router as a node. In addition to giving the address of the announcing router, this type of advertisement can define one or more types of links that connect the advertising router to other entities. A *transient link* announces a link to a transient network, a network that is connected to the rest of the networks by one or more routers. This type of

advertisement should define the address of the transient network and the cost of the link. A *stub link* advertises a link to a stub network, a network that is not a through network. Again, the advertisement should define the address of the network and the cost. A *point-to-point link* should define the address of the router at the end of the point-to-point line and the cost to get there.

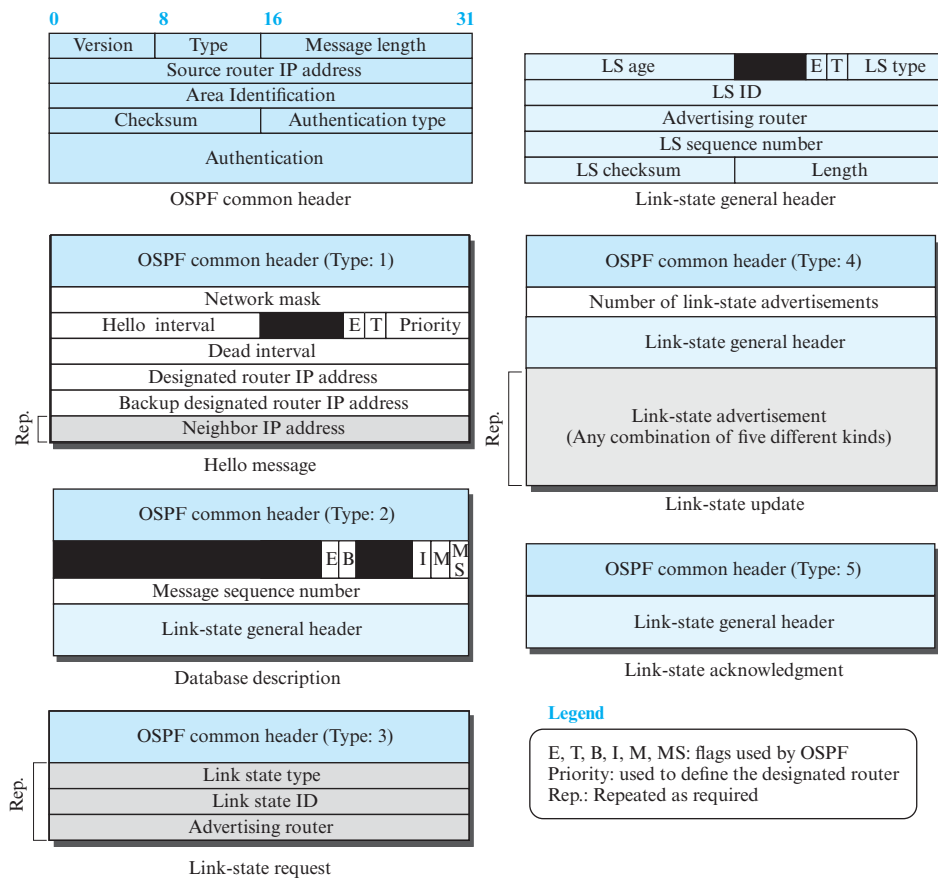
- ❑ **Network link.** A network link advertises the network as a node. However, since a network cannot do announcements itself (it is a passive entity), one of the routers is assigned as the designated router and does the advertising. In addition to the address of the designated router, this type of LSP announces the IP address of all routers (including the designated router as a router and not as speaker of the network), but no cost is advertised because each router announces the cost to the network when it sends a router link advertisement.
- ❑ **Summary link to network.** This is done by an area border router; it advertises the summary of links collected by the backbone to an area or the summary of links collected by the area to the backbone. As we discussed earlier, this type of information exchange is needed to glue the areas together.
- ❑ **Summary link to AS.** This is done by an AS router that advertises the summary links from other ASs to the backbone area of the current AS, information which later can be disseminated to the areas so that they will know about the networks in other ASs. The need for this type of information exchange is better understood when we discuss inter-AS routing (BGP).
- ❑ **External link.** This is also done by an AS router to announce the existence of a single network outside the AS to the backbone area to be disseminated into the areas.

OSPF Implementation

OSPF is implemented as a program in the network layer that uses the service of the IP for propagation. An IP datagram that carries a message from OSPF sets the value of the protocol field to 89. This means that, although OSPF is a routing protocol to help IP to route its datagrams inside an AS, the OSPF messages are encapsulated inside datagrams. OSPF has gone through two versions: version 1 and version 2. Most implementations use version 2.

OSPF Messages OSPF is a very complex protocol; it uses five different types of messages. In Figure 4.78, we first show the format of the OSPF common header (which is used in all messages) and the link-state general header (which is used in some messages). We then give the outlines of five message types used in OSPF. The *hello* message (type 1) is used by a router to introduce itself to the neighbors and announces all neighbors that it already knows. The *database description* message (type 2) is normally sent in response to the hello message to allow a newly joined router to acquire the full LSDB. The *link-state request* message (type 3) is sent by a router that needs information about a specific LS. The *link-state update* message (type 4) is the main OSPF message used for building the LSDB. This message, in fact, has five different versions (router link, network link, summary link to network, summary link to AS border router, and external link), as we discussed before. The *link-state acknowledgment* message (type 5) is used to create reliability in OSPF; each router that receives a link-state update message needs to acknowledge it.

Figure 4.78 OSPF message formats



Authentication As Figure 4.78 shows, the OSPF common header has the provision for authentication of the message sender. As we will discuss in Chapter 10, this prevents a malicious entity from sending OSPF messages to a router and causing the router to become part of the routing system to which it actually does not belong.

OSPF Algorithm OSPF implements the link-state routing algorithm we discussed in the previous section. However, some changes and augmentations need to be added to the algorithm:

- ❑ After each router has created the shortest-path tree, the algorithm needs to use it to create the corresponding routing algorithm.
- ❑ The algorithm needs to be augmented to handle sending and receiving all five types of messages.

Performance

Before ending this section, let us briefly discuss the performance of OSPF:

- ❑ **Update Messages.** The link-state messages in OSPF have a somewhat complex format. They also are flooded to the whole area. If the area is large, these messages may create heavy traffic and use a lot of bandwidth.
- ❑ **Convergence of Forwarding Tables.** When the flooding of LSPs is completed, each router can create its own shortest-path tree and forwarding table; convergence is fairly quick. However, each router needs to run the Dijkstra's algorithm, which may take some time.
- ❑ **Robustness.** The OSPF protocol is more robust than RIP because, after receiving the completed LSDB, each router is independent and does not depend on other routers in the area. Corruption or failure in one router does not affect other routers as seriously as in RIP.

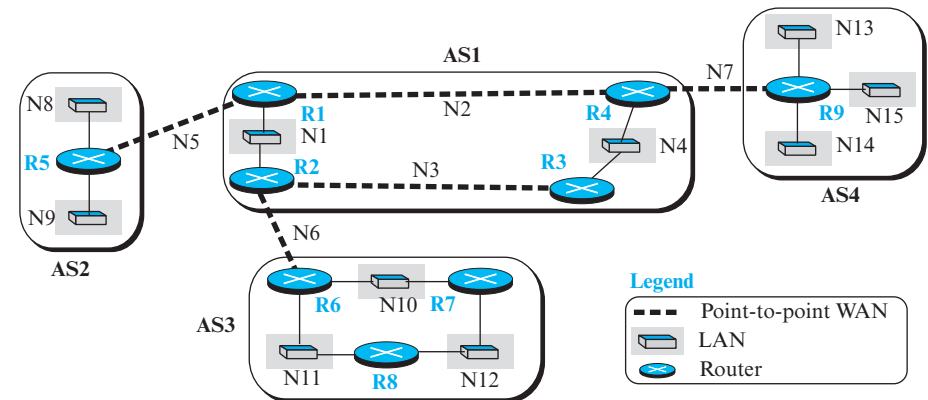
Border Gateway Protocol Version 4 (BGP4)

The Border Gateway Protocol version 4 (BGP4) is the only interdomain routing protocol used in the Internet today. BGP4 is based on the path-vector algorithm we described before, but it is tailored to provide information about the reachability of networks in the Internet.

Introduction

BGP, and in particular BGP4, is a complex protocol. In this section, we introduce the basics of BGP and its relationship with intradomain routing protocols (RIP or OSPF). Figure 4.79 shows an example of an internet with four autonomous systems. AS2, AS3, and AS4 are *stub* autonomous systems; AS1 is a *transient* one. In our example, data exchange between AS2, AS3, and AS4 should pass through AS1.

Figure 4.79 A sample internet with four ASs



Each autonomous system in this figure uses one of the two common intradomain routing protocols, RIP or OSPF. Each router in each AS knows how to reach a network that is in its own AS, but it does not know how to reach a network in another AS.

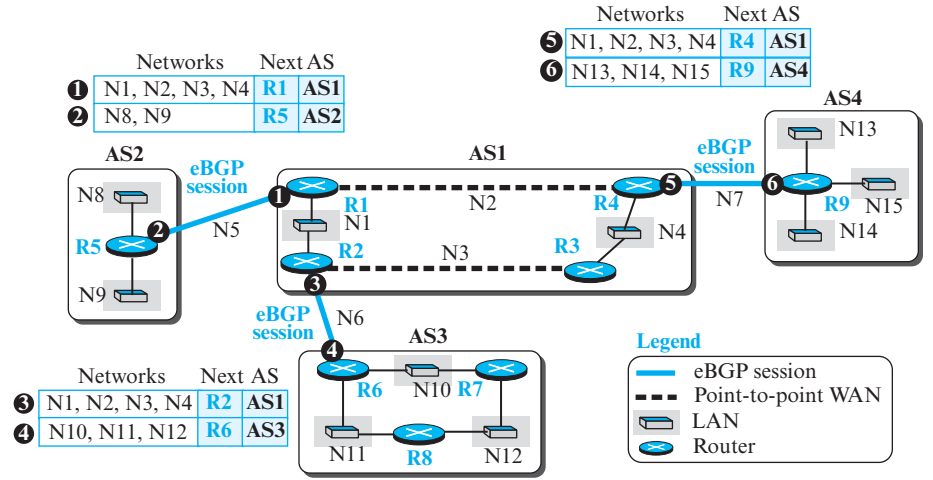
To enable each router to route a packet to any network in the internet, we first install a variation of BGP4, called *external BGP (eBGP)*, on each *border router* (the

one at the edge of each AS which is connected to a router at another AS). We then install the second variation of BGP, called *internal BGP (iBGP)*, on all routers. This means that the border routers will be running three routing protocols (intradomain, eBGP, and iBGP), but other routers are running two protocols (intradomain and iBGP). We discuss the effect of each BGP variation separately.

Operation of External BGP (eBGP) We can say that BGP protocol is a kind of point-to-point protocol. When the software is installed on two routers, they try to create a TCP connection using the well-known port 179. In other words, a pair of client and server processes continuously communicate with each other to exchange messages. The two routers that run the BGP processes are called *BGP peers* or *BGP speakers*. We discuss different types of messages exchanged between two peers, but for the moment we are interested in only the update messages (discussed later) that announce reachability of networks in each AS.

The eBGP variation of BGP allows two physically connected border routers in two different ASs to form pairs of eBGP speakers and exchange messages. The routers that are eligible in our example in Figure 4.79 form three pairs: R1-R5, R2-R6, and R4-R9. The connection between these pairs is established over three physical WANs (N5, N6, and N7). However, there is a need for a logical TCP connection to be created over the physical connection to make the exchange of information possible. Each logical connection in BGP parlance is referred to as a *session*. This means that we need three sessions in our example, as shown in Figure 4.80.

Figure 4.80 eBGP operation



The figure also shows the simplified update messages sent by routers involved in the eBGP sessions. The circled number defines the sending router in each case. For example, message number 1 is sent by router R1 and tells router R5 that N1, N2, N3, and N4 can be reached through router R1 (R1 gets this information from the corresponding intradomain forwarding table). Router R5 can now add these pieces of

information at the end of its forwarding table. When R5 receives any packet destined for these four networks, it can use its forwarding table and find that the next router is R1.

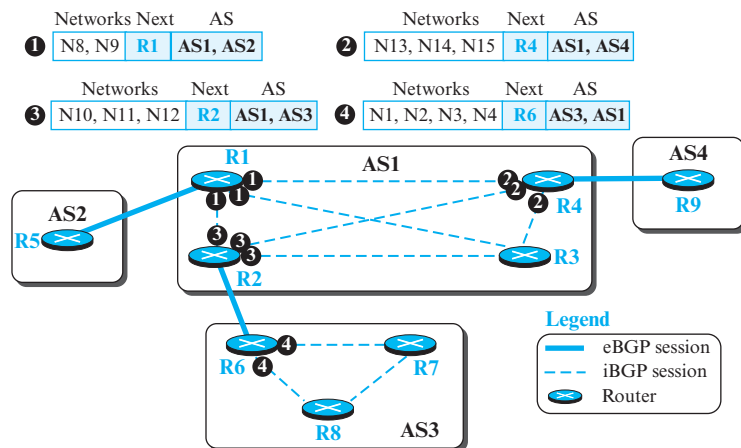
The reader may have noticed that the messages exchanged during three eBGP sessions help some routers know how to route packets to some networks in the internet, but the reachability information is not complete. There are two problems that need to be addressed:

1. Some border routers do not know how to route a packet destined for non-neighbor ASs. For example, R5 does not know how to route packets destined for networks in AS3 and AS4. Routers R6 and R9 are in the same situation as R5: R6 does not know about networks in AS4; R9 does not know about networks in AS3.
2. None of non-border routers know how to route a packet destined for any networks in other ASs.

To address the above two problems, we need to allow all pairs of routers (border or non-border) to run the second variation of the BGP protocol, iBGP.

Operation of Internal BGP (iBGP) The iBGP protocol is similar to the eBGP protocol in that it uses the service of TCP on the well-known port 179, but it creates a session between any possible pair of routers inside an autonomous system. However, some points should be made clear. First, if an AS has only one router, there cannot be an iBGP session. For example, we cannot create an iBGP session inside AS2 or AS4 in our internet. Second, if there are n routers in an autonomous system, there should be $[n \times (n - 1) / 2]$ iBGP sessions in that autonomous system (a fully connected mesh) to prevent loops in the system. In other words, each router needs to advertise its own reachability to the peer in the session instead of flooding what it receives from another peer in another session. Figure 4.81 shows the combination of eBGP and iBGP sessions in our internet.

Figure 4.81 Combination of eBGP and iBGP sessions in our internet



Note that we have not shown the physical networks inside ASs because a session is made on an overlay network (TCP connection), possibly spanning more than one

physical network as determined by the route dictated by intradomain routing protocol. Also note that in this stage only four messages are exchanged. The first message (numbered 1) is sent by R1 announcing that networks N8 and N9 are reachable through the path AS1-AS2, but the next router is R1. This message is sent, through separate sessions, to R2, R3, and R4. Routers R2, R4, and R6 do the same thing but send different messages to different destinations. The interesting point is that, at this stage, R3, R7, and R8 create sessions with their peers, but they actually have no message to send.

The updating process does not stop here. For example, after R1 receives the update message from R2, it combines the reachability information about AS3 with the reachability information it already knows about AS1 and sends a new update message to R5. Now R5 knows how to reach networks in AS1 and AS3. The process continues when R1 receives the update message from R4. The point is that we need to make certain that at a point in time there are no changes in the previous updates and that all information is propagated through all ASs. At this time, each router combines the information received from eBGP and iBGP and creates what we may call a path table after applying the criteria for finding the best path, including routing policies that we discuss later. To demonstrate, we show the path tables in Figure 4.82 for the routers in Figure 4.79. For example, router R1 now knows that any packet destined for networks N8 or N9 should go through AS1 and AS2 and the next router to deliver the packet to is router R5. Similarly, router R4 knows that any packet destined for networks N10, N11, or N12 should go through AS1 and AS3 and the next router to deliver this packet to is router R1, and so on.

Figure 4.82 Finalized BGP path tables

Networks	Next	Path	Networks	Next	Path	Networks	Next	Path
N8, N9	R5	AS1, AS2	N8, N9	R1	AS1, AS2	N8, N9	R2	AS1, AS2
N10, N11, N12	R2	AS1, AS3	N10, N11, N12	R6	AS1, AS3	N10, N11, N12	R2	AS1, AS3
N13, N14, N15	R4	AS1, AS4	N13, N14, N15	R1	AS1, AS4	N13, N14, N15	R4	AS1, AS4
Path table for R1			Path table for R2			Path table for R3		
Networks	Next	Path	Networks	Next	Path	Networks	Next	Path
N8, N9	R1	AS1, AS2	N1, N2, N3, N4	R1	AS2, AS1	N1, N2, N3, N4	R2	AS3, AS1
N10, N11, N12	R1	AS1, AS3	N10, N11, N12	R1	AS2, AS1, AS3	N8, N9	R2	AS3, AS1, AS2
N13, N14, N15	R9	AS1, AS4	N13, N14, N15	R1	AS2, AS1, AS4	N13, N14, N15	R2	AS3, AS1, AS4
Path table for R4			Path table for R5			Path table for R6		
Networks	Next	Path	Networks	Next	Path	Networks	Next	Path
N1, N2, N3, N4	R6	AS3, AS1	N1, N2, N3, N4	R6	AS3, AS1	N1, N2, N3, N4	R4	AS4, AS1
N8, N9	R6	AS3, AS1, AS2	N8, N9	R6	AS3, AS1, AS2	N8, N9	R4	AS4, AS1, AS2
N13, N14, N15	R6	AS3, AS1, AS4	N13, N14, N15	R6	AS3, AS1, AS4	N10, N11, N12	R4	AS4, AS1, AS3
Path table for R7			Path table for R8			Path table for R9		

Injection of Information into Intradomain Routing The role of an interdomain routing protocol such as BGP is to help the routers inside the AS to augment their routing information. In other words, the path tables collected and organized by BPG are not used, per se, for routing packets; they are injected into intradomain forwarding tables (RIP or OSPF) for routing packets. This can be done in several ways depending on the type of AS.

In the case of a stub AS, the only area border router adds a default entry at the end of its forwarding table and defines the next router to be the speaker router at the end of the eBGP connection. In Figure 4.79, R5 in AS2 defines R1 as the default router for all

networks other than N8 and N9. The situation is the same for router R9 in AS4 with the default router to be R4. In AS3, R6 set its default router to be R2, but R7 and R8 set their default routers to be R6. These settings are in accordance with the path tables we describe in Figure 4.82 for these routers. In other words, the path tables are injected into intradomain forwarding tables by adding only one default entry.

In the case of a transient AS, the situation is more complicated. R1 in AS1 needs to inject the whole contents of the path table for R1 in Figure 4.82 into its intradomain forwarding table. The situation is the same for R2, R3, and R4.

One issue to be resolved is the cost value. We know that RIP and OSPF use different metrics. One solution, which is very common, is to set the cost to the foreign networks at the same cost value as to reach the first AS in the path. For example, the cost for R5 to reach all networks in other ASs is the cost to reach N5. The cost for R1 to reach networks N10 to N12 is the cost to reach N6, and so on. The cost is taken from the intradomain forwarding tables (RIP or OSPF).

Figure 4.83 shows the interdomain forwarding tables. For simplicity, we assume that all ASs are using RIP as the intradomain routing protocol. The shaded areas are the augmentation injected by the BGP protocol; the default destinations are indicated as zero.

Figure 4.83 Forwarding tables after injection from BGP

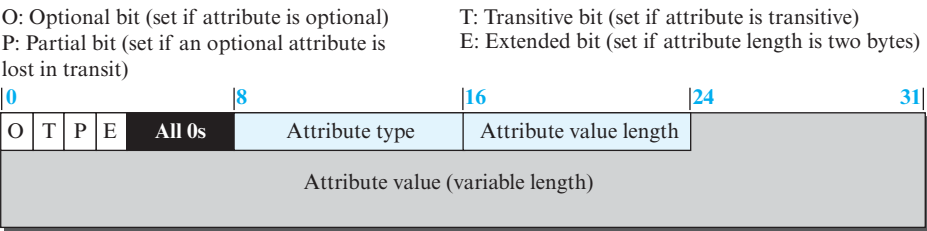
Des.	Next	Cost	Des.	Next	Cost	Des.	Next	Cost	Des.	Next	Cost
N1	—	1	N1	—	1	N1	R2	2	N1	R1	2
N4	R4	2	N4	R3	2	N4	—	1	N4	—	1
N8	R5	1	N8	R1	2	N8	R2	3	N8	R1	2
N9	R5	1	N9	R1	2	N9	R2	3	N9	R1	2
N10	R2	2	N10	R6	1	N10	R2	2	N10	R3	3
N11	R2	2	N11	R6	1	N11	R2	2	N11	R3	3
N12	R2	2	N12	R6	1	N12	R2	2	N12	R3	3
N13	R4	2	N13	R3	3	N13	R4	2	N13	R9	1
N14	R4	2	N14	R3	3	N14	R4	2	N14	R9	1
N15	R4	2	N15	R3	3	N15	R4	2	N15	R9	1
Table for R1			Table for R2			Table for R3			Table for R4		
Des.	Next	Cost	Des.	Next	Cost	Des.	Next	Cost	Des.	Next	Cost
N8	—	1	N10	—	1	N10	R6	2	N13	—	1
N9	—	1	N11	—	1	N11	R6	2	N14	—	1
0	R1	1	N12	R7	2	N12	—	1	N15	—	1
Table for R5			Table for R6			Table for R7			Table for R8		
N8	—	1	0	R2	1	0	R6	2	0	R6	2
N9	—	1									
0	R1	1									
Table for R9											
N10	—	1									
N11	—	1									
N12	—	1									
0	R4	1									

Address Aggregation The reader may have realized that intradomain forwarding tables obtained with the help of the BGP4 protocols may become huge in the case of the global Internet because many destination networks may be included in a forwarding table. Fortunately, BGP4 uses the prefixes as destination identifiers and allows the aggregation of these prefixes, as we discussed earlier in the chapter. For example, prefixes 14.18.20.0/26, 14.18.20.64/26, 14.18.20.128/26, and 14.18.20.192/26, can be combined into 14.18.20.0/24 if all four subnets can be reached through one path. Even if one or two of the aggregated prefixes need a separate path, the longest prefix principle we discussed earlier allows us to do so.

Path Attributes

In both intradomain routing protocols (RIP or OSPF), a destination is normally associated with two pieces of information: next hop and cost. The first one shows the address of the next router to deliver the packet; the second defines the cost to the final destination. Inter-domain routing is more involved and naturally needs more information about how to reach the final destination. In BGP these pieces are called **path attributes**. BGP allows a destination to be associated with up to seven path attributes. Path attributes are divided into two broad categories: *well-known* and *optional*. A well-known attribute must be recognized by all routers; an optional attribute does not. A well-known attribute can be mandatory, which means that it must be present in any BGP update message, or discretionary, which means it does not have to be. An optional attribute can be either transitive, which means it can pass to the next AS, or intransitive, which means it cannot. All attributes are inserted after the corresponding destination prefix in an update message (discussed later). The format for an attribute is shown in Figure 4.84.

Figure 4.84 *Format of path attribute*



The first byte in each attribute defines the four attribute flags (as shown in the figure). The next byte defines the type of attributes assigned by ICANN (only seven types have been assigned, as explained next). The attribute value length defines the length of the attribute value field (not the length of the whole attributes section). The following gives a brief description of each attribute.

- ❑ **ORIGIN (type 1).** This is a well-known mandatory attribute, which defines the source of the routing information. This attribute can be defined by one of the three values: 1, 2, and 3. Value 1 means that the information about the path has been taken from an intradomain protocol (RIP or OSPF). Value 2 means that the information comes from BGP. Value 3 means that it comes from an unknown source.
- ❑ **AS-PATH (type 2).** This is a well-known mandatory attribute, which defines the list of autonomous systems through which the destination can be reached. We have used this attribute in our examples. The AS-PATH attribute, as we discussed in path-vector routing in the last section, helps prevent a loop. Whenever an update message arrives at a router that lists the current AS as the path, the router drops that path. The AS-PATH can also be used in route selection.
- ❑ **NEXT-HOP (type 3).** This is a well-known mandatory attribute, which defines the next router to which the data packet should be forwarded. We have also used this attribute in our examples. As we have seen, this attribute helps to inject path

information collected through the operations of eBGP and iBGP into the intradomain routing protocols such as RIP or OSPF.

- ❑ **MULT-EXIT-DISC (type 4).** The multiple-exit discriminator is an optional non-transitive attribute, which discriminates among multiple exit paths to a destination. The value of this attribute is normally defined by the metric in the corresponding intradomain protocol (an attribute value of four-byte unsigned integer). For example, if a router has multiple paths to the destination with different values related to these attributes, the one with the lowest value is selected. Note that this attribute is non-transitive, which means that it is not propagated from one AS to another.
- ❑ **LOCAL-PREF (type 5).** The local preference attribute is a well-known discretionary attribute. It is normally set by the administrator, based on the organization policy. The routes the administrator prefers are given a higher local preference value (an attribute value of four-byte unsigned integer). For example, in an internet with five ASs, the administrator of AS1 can set the local preference value of 400 to the path AS1-AS2-AS5, the value of 300 to AS1-AS3, AS5, and the value of 50 to AS1-AS4-AS5. This means that the administrator prefers the first path to the second one and prefers the second one to the third one. This may be a case where AS2 is the most secured and AS4 is the least secured AS for the administration of AS1. The last route should be selected if the other two are not available.
- ❑ **ATOMIC-AGGREGATE (type 6).** This is a well-known discretionary attribute, which defines the destination prefix as not aggregate; it only defines a single destination network. This attribute has no value field, which means the value of the length field is zero.
- ❑ **AGGREGATOR (type 7).** This is an optional transitive attribute, which emphasizes that the destination prefix is an aggregate. The attribute value gives the number of the last AS that did the aggregation followed by the IP address of the router that did so.

Route Selection

So far in this section, we have been silent about how a route is selected by a BGP router mostly because our simple example has one route to a destination. In the case where multiple routes are received to a destination, BGP needs to select one among them. The route selection process in BGP is not as easy as the ones in the intradomain routing protocol that is based on the shortest-path tree. A route in BGP has some attributes attached to it and it may come from an eBGP session or an iBGP session. Figure 4.85 shows the flow diagram as used by common implementations.

The router extracts the routes which meet the criteria in each step. If only one route is extracted, it is selected and the process stops; otherwise, the process continues with the next step. Note that the first choice is related to the LOCAL-PREF attribute, which reflects the policy imposed by the administration on the route.

Messages

BGP uses four types of messages for communication between the BGP speakers across the ASs and inside an AS: *open*, *update*, *keepalive*, and *notification* (see Figure 4.86). All BGP packets share the same common header.

- ❑ **Open Message.** To create a neighborhood relationship, a router running BGP opens a TCP connection with a neighbor and sends an *open message*.

Figure 4.85 Flow diagram for route selection

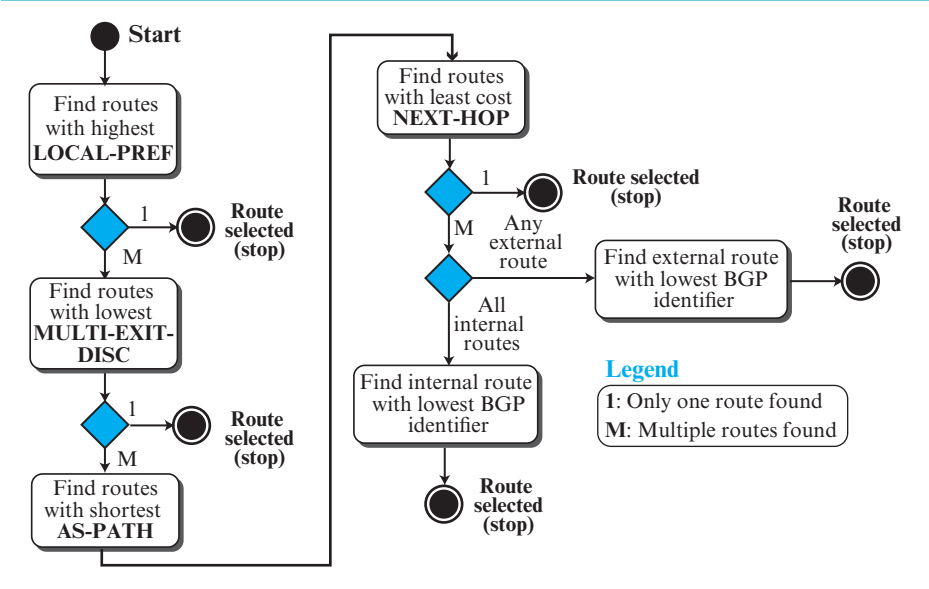
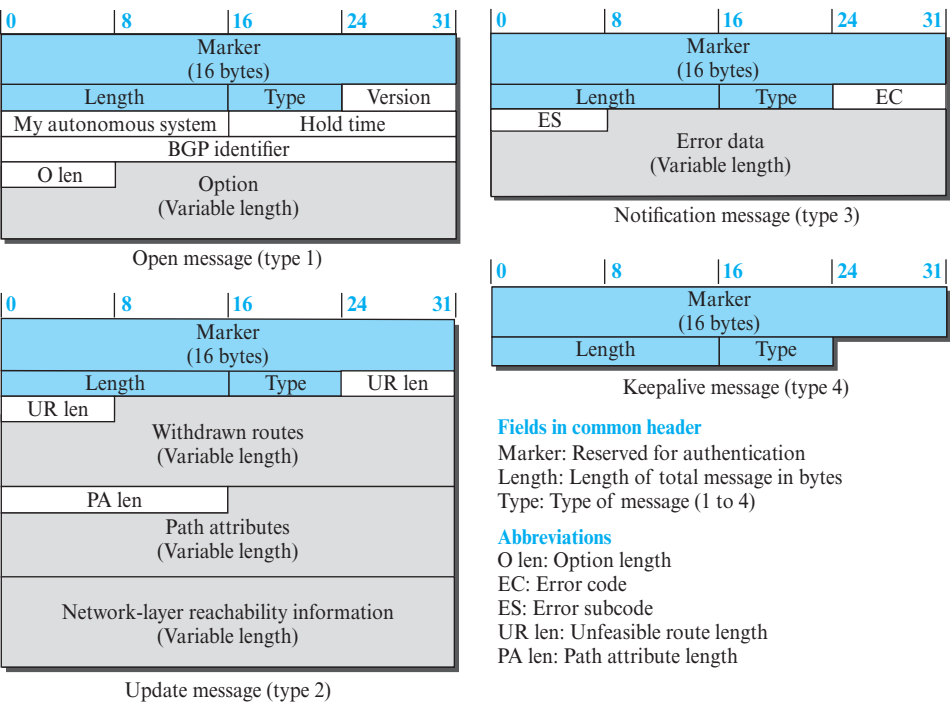


Figure 4.86 BGP messages



- ❑ **Update Message.** The *update message* is the heart of the BGP protocol. It is used by a router to withdraw destinations that have been advertised previously, to announce a route to a new destination, or both. Note that BGP can withdraw several destinations that were advertised before, but it can only advertise one new destination (or multiple destinations with the same path attributes) in a single update message.
- ❑ **Keepalive Message.** The BGP peers that are running exchange keepalive messages regularly (before their hold time expires) to tell each other that they are alive.
- ❑ **Notification.** A notification message is sent by a router whenever an error condition is detected or a router wants to close the session.

Performance

BGP performance can be compared with RIP. BGP speakers exchange a lot of messages to create forwarding tables, but BGP is free from loops and count-to-infinity. The same weakness we mention for RIP about propagation of failure and corruptness also exists in BGP.

4.4 MULTICAST ROUTING

Communication in the Internet today is not only unicasting; multicasting communication is growing fast. In this section, we first discuss the general ideas behind unicasting, multicasting, and broadcasting. We then talk about some basic issues in multicast routing. Finally, we discuss multicasting routing protocols in the Internet.

4.4.1 Introduction

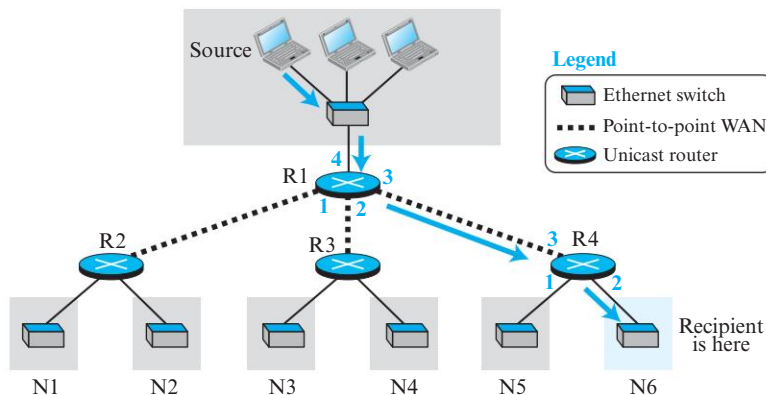
From the previous sections, we have learned that forwarding a datagram by a router is normally based on the prefix of the destination address in the datagram, which defines the network to which the destination host is connected. Understanding the above forwarding principle, we can now define unicasting, multicasting, and broadcasting. Let us clarify these terms as they relate to the Internet.

Unicasting

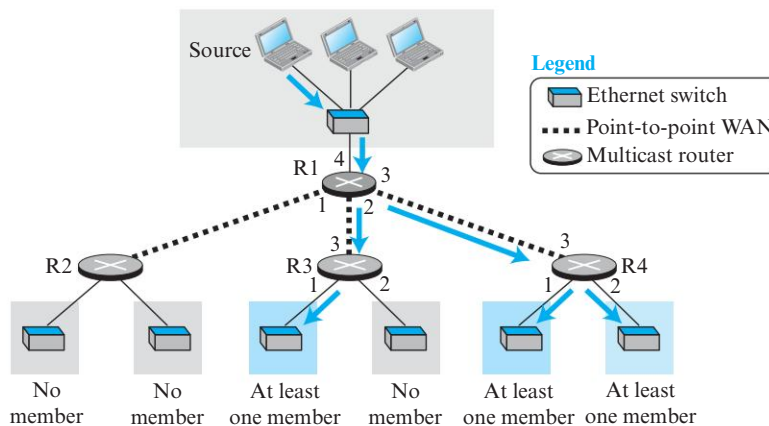
In unicasting, there is one source and one destination network. The relationship between the source and the destination network is one to one. Each router in the path of the datagram tries to forward the packet to one and only one of its interfaces. Figure 4.87 shows a small internet in which a unicast packet needs to be delivered from a source computer to a destination computer attached to N6. Router R1 is responsible for forwarding the packet only through interface 3; router R4 is responsible for forwarding the packet only through interface 2. When the packet arrives at N6, the delivery to the destination host is the responsibility of the network; it is either broadcast to all hosts or the Ethernet switch delivers it only to the destination host.

Multicasting

In multicasting, there is one source and a group of destinations. The relationship is one to many. In this type of communication, the source address is a unicast address, but the destination address is a group address, a group of one or more destination

Figure 4.87 *Unicasting*

networks in which there is at least one member of the group that is interested in receiving the multicast datagram. The group address defines the members of the group. Figure 4.88 shows the same small internet as in Figure 4.87, but the routers have been changed to multicast routers (or previous routers have been configured to do both types of jobs).

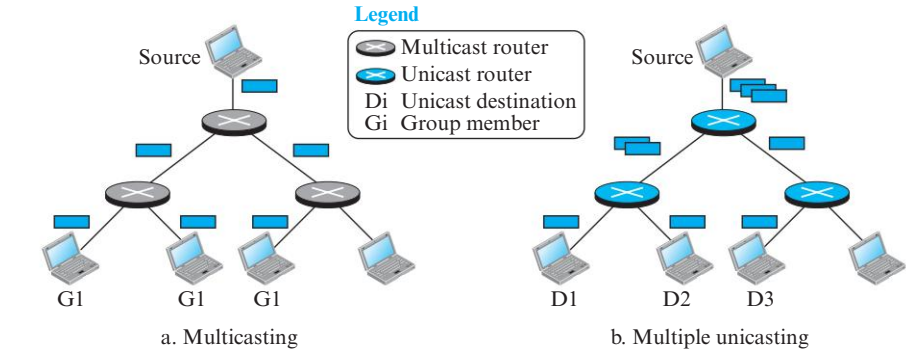
Figure 4.88 *Multicasting*

In multicasting, a multicast router may have to send out copies of the same datagram through more than one interface. In Figure 4.88, router R1 needs to send out the datagram through interfaces 2 and 3. Similarly, router R4 needs to send out the datagram through both its interfaces. Router R3, however, knows that there is no member belonging to this group in the area reached by interface 2; it only sends out the datagram through interface 1.

Multicasting versus Multiple Unicasting

We need to distinguish between multicasting and multiple unicasting. Figure 4.89 illustrates both concepts.

Figure 4.89 Multicasting versus multiple unicasting



Multicasting starts with a single packet from the source that is duplicated by the routers. The destination address in each packet is the same for all duplicates. Note that only a single copy of the packet travels between any two routers.

In multiple unicasting, several packets start from the source. If there are three destinations, for example, the source sends three packets, each with a different unicast destination address. Note that there may be multiple copies traveling between two routers. For example, when a person sends an e-mail message to a group of people, this is multiple unicasting. The e-mail application software creates replicas of the message, each with a different destination address, and sends them one by one.

Emulation of Multicasting with Unicasting

We may ask why we have a separate mechanism for multicasting, when it can be emulated with unicasting. There are at least two reasons:

1. Multicasting is more efficient than multiple unicasting. In Figure 4.89, we can see how multicasting requires less bandwidth than multiple unicasting. In multiple unicasting, some of the links must handle several copies.
2. In multiple unicasting, the packets are created by the source with a relative delay between packets. If there are 1,000 destinations, the delay between the first and the last packet may be unacceptable. In multicasting, there is no delay because only one packet is created by the source.

Multicast Applications

Multicasting has many applications today, such as access to distributed databases, information dissemination, teleconferencing, and distance learning.

- **Access to Distributed Databases.** Most of the large databases today are distributed. That is, the information is stored in more than one location, usually at the time of production. The user who needs to access the database does not know the location

of the information. A user's request is multicast to all the database locations, and the location that has the information responds.

- ❑ **Information Dissemination.** Businesses often need to send information to their customers. If the nature of the information is the same for each customer, it can be multicast. In this way a business can send one message that can reach many customers. For example, a software update can be sent to all purchasers of a particular software package. In a similar manner, news can be easily disseminated through multicasting.
- ❑ **Teleconferencing.** Teleconferencing involves multicasting. The individuals attending a teleconference all need to receive the same information at the same time. Temporary or permanent groups can be formed for this purpose.
- ❑ **Distance Learning.** One growing area in the use of multicasting is distance learning. Lessons taught by one professor can be received by a specific group of students. This is especially convenient for those students who find it difficult to attend classes on campus.

Broadcasting

Broadcasting means one-to-all communication: a host sends a packet to all hosts in an internet. Broadcasting in this sense is not provided at the Internet level for the obvious reason that it may create a huge volume of traffic and use a huge amount of bandwidth. Partial broadcasting, however, is done in the Internet. For example, some peer-to-peer applications may use broadcasting to access all peers. Controlled broadcasting may also be done in a domain (area or autonomous system) mostly as a step to achieve multicasting. We discuss these types of controlled broadcasting when we discuss multicasting protocols.

4.4.2 Multicasting Basics

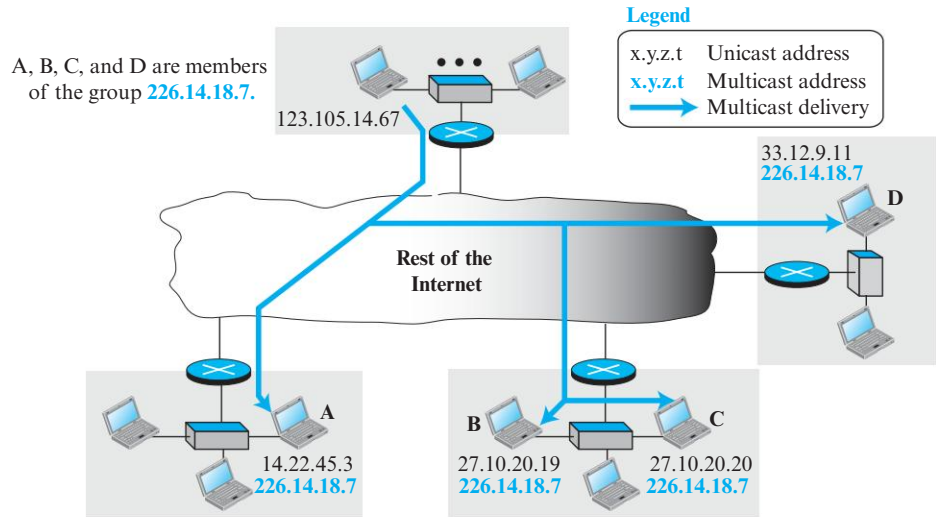
Before discussing multicast routing protocols in the Internet, we need to discuss some multicasting basics: multicast addressing, collecting information about multicast groups, and multicast optimal trees.

Multicast Addresses

When we send a unicast packet to a destination, the source address of the packet defines the sender and the destination address of the packet defines the receiver of the packet. In multicast communication, the sender is only one, but the receiver is many, sometimes thousands or millions spread all over the world. It should be clear that we cannot include the addresses of all recipients in the packet. The destination address of a packet, as described in the Internet Protocol (IP) should be only one. For this reason, we need multicast addresses. A multicast address defines a group of recipients, not a single one. In other words, a multicast address is an identifier for a group. If a new group is formed with some active members, an authority can assign an unused multicast address to this group to uniquely define it. This means that the source address of a packet in multicast communication can be a unicast address that uniquely defines the sender, but the destination address can be the multicast address that defines a group. In this way, a host, which is a member of n groups, actually has $(n + 1)$ addresses: one unicast address that is used for source or destination address in unicast communication and

n multicast addresses that are used only for destination addresses to receive messages sent to a group. Figure 4.90 shows the concept.

Figure 4.90 Needs for multicast addresses



Multicast Addresses in IPv4

A router or a destination host needs to distinguish between a unicast and a multicast datagram. IPv4 and IPv6 each assigns a block of addresses for this purpose. In this section, we discuss only IPv4 multicast addresses; IPv6 multicast addresses are discussed later in the chapter. Multicast addresses in IPv4 belong to a large block of addresses that are specially designed for this purpose. In classful addressing, all of class D was composed of these addresses; classless addressing used the same block, but it was referred to as the block 224.0.0.0/4 (from 224.0.0.0 to 239.255.255.255). Figure 4.91 shows the block in binary. Four bits define the block; the rest of the bits are used as the identifier for the group.

Figure 4.91 A multicast address in binary



The number of addresses in the multicast block is huge (2^{28}). We definitely cannot have that many individual groups. However, the block is divided into several subblocks, and each subblock is used in a particular multicast application. The following gives some of the common subblocks:

- ❑ **Local Network Control Block.** The subblock 224.0.0.0/24 is assigned to a multicast routing protocol to be used inside a network, which means that the packet with

a destination address in this range cannot be forwarded by a router. In this subblock, the address 224.0.0.0 is reserved, the address 224.0.0.1 is used to send datagrams to all hosts and routers inside a network, the address 224.0.0.2 is used to send datagrams to all routers inside a network. The rest of the addresses are assigned to some multicast protocols for communication, as we discuss later.

- ❑ **Internetwork Control Block.** The subblock 224.0.1.0/24 is assigned to a multicast routing protocol to be used in the whole Internet, which means that the packet with a destination address in this range can be forwarded by a router.
- ❑ **Source-Specific Multicast (SSM) Block.** The block 232.0.0.0/8 is used for source-specific multicast routing. We discuss SSM routing when we discuss IGMP protocol later in the chapter.
- ❑ **GLOP Block.** The block 233.0.0.0/8 is called the GLOP block (not an acronym nor an abbreviation). This block defines a range of addresses that can be used inside an autonomous system (AS). As we learned earlier, each autonomous system is assigned a 16-bit number. One can insert the AS number as the two middle octets in the block to create a range of 256 multicast addresses (233.x.y.0 to 233.x.y.255), in which x.y is the AS number.
- ❑ **Administratively Scoped Block.** The block 239.0.0.0/8 is called the Administratively Scoped Block. The addresses in this block are used in a particular area of the Internet. The packet whose destination address belongs to this range is not supposed to leave the area. In other words, an address in this block is restricted to an organization.

Selecting Multicast Address

To select a multicast address to be assigned to a group is not an easy task. The selection of address depends on the type of application. Let us discuss some cases.

Limited Group The administrator can use the AS number (x.y)₂₅₆ and choose an address between 239.x.y.0 and 239.x.y.255 (Administratively Scoped Block), that is not used by any other group, as the multicast address for that particular group. For example, assume college professors need to create group addresses to communicate with their students. If the AS number that the college belongs to is 23452, which can be written as (91.156)₂₅₆, this gives the college a range of 256 addresses: 233.91.156.0 to 233.91.156.255. The college administration can grant each professor one of the addresses in the range. This can then become the group address for the professor to use to send multicast communications to the students. However, the packets cannot go beyond the college AS territory.

Larger Group If the group is spread beyond an AS territory, the previous solution does not work. The group needs to choose an address from the SSM block (232.0.0.8). There is no need to get permission to use an address in this block, because the packets in source-specific multicasting are routed based on the group and the source address; they are unique.

Collecting Information about Groups

Creation of forwarding tables in both unicast and multicast routing involves two steps:

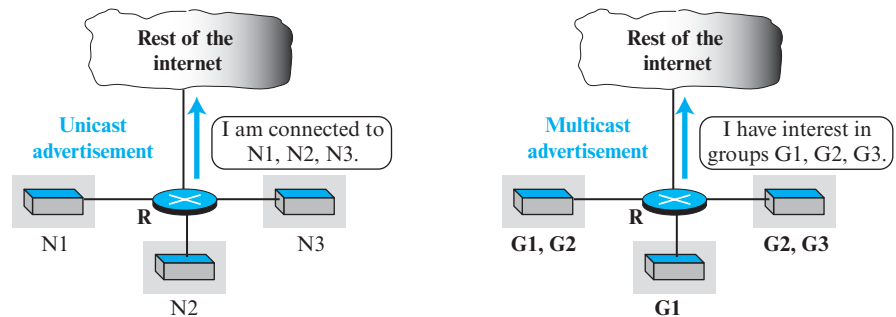
1. A router needs to know to which destinations it is connected.

2. Each router needs to propagate information obtained in the first step to all other routers so that each router knows to which destination each other router is connected.

In unicast routing, the collection of the information in the first step is automatic; each router knows to which network it is connected, and the prefix of the network (in CIDR) is what a router needs. The routing protocols we described in the previous sections (distance-vector or link-state) are responsible for propagating these automatically collected pieces of information to each other router in the internet.

In multicast routing, the collection of information in the first step is not automatic for two reasons. First, a router does not know which host in the attached network is a member of a particular group; membership in the group does not have any relation to the prefix associated with the network. We have shown that if a host is a member of a group, it has a separate multicast address related to that group. Second, the membership is not a fixed attribute of a host; a host may join some new groups and leave some others even in a short period of time. For this reason, a router needs help to find out which groups are active in each of its interfaces. After collecting these pieces of information, a router can propagate the membership to any other router using a multicast routing protocol, as we will discuss later. Figure 4.92 shows the difference between unicast and multicast advertisement in the first step. For unicasting, the router needs no help; for multicasting, it needs the help of another protocol, as we discuss next.

Figure 4.92 Unicast versus multicast advertisement



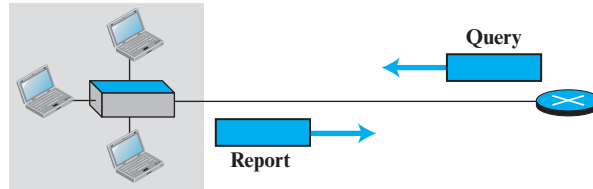
In the case of unicasting, router R knows hosts with prefixes N1, N2, and N3 are connected to its interfaces; it propagates this information to the rest of the internet. In the case of multicasting, router R needs to know that there are hosts with at least one loyal member in groups G1, G2, and G3 in networks connected to its interfaces. In other words, for unicast routing, we need only the routing protocol inside each domain to propagate the information about a router link; in multicasting we need two protocols: one to collect these pieces of information and the second to propagate them. Before we discuss the protocols involved in the second step, we need to discuss the protocol involved in the first step.

Internet Group Management Protocol (IGMP)

The protocol that is used today for collecting information about group membership is the **Internet Group Management Protocol (IGMP)**. IGMP is a protocol defined at

the network layer; it is one of the auxiliary protocols, like ICMP, which is considered part of the IP. IGMP messages, like ICMP messages, are encapsulated in an IP datagram. There are only two types of messages in IGMP version 3: query and report messages, as shown in Figure 4.93.

Figure 4.93 IGMP operation



- ❑ **Query Message.** A query message is periodically sent by a router to all hosts attached to it to ask them to report their interests about membership in groups. In IGMPv3, a query message can take one of three forms:
 - a. A *general* query message is sent about membership in any group. It is encapsulated in a datagram with the destination address 224.0.0.1 (all hosts and routers). Note that all routers attached to the same network receive this message to inform them that this message is already sent and that they should refrain from resending it.
 - b. A *group-specific* query message is sent from a router to ask about the membership related to a specific group. This is sent when a router does not receive a response about a specific group and wants to be sure that there is no active member of that group in the network. The group identifier (multicast address) is mentioned in the message. The message is encapsulated in a datagram with the destination address set to the corresponding multicast address. Although all hosts receive this message, those not interested drop it.
 - c. A *source-and-group-specific* query message is sent from a router to ask about the membership related to a specific group when the message comes from a specific source or sources. Again the message is sent when the router does not hear about a specific group related to a specific host or hosts. The message is encapsulated in a datagram with the destination address set to the corresponding multicast address. Although all hosts receive this message, those not interested drop it.
- ❑ **Report Message.** A report message is sent by a host as a response to a query message. The message contains a list of records in which each record gives the identifier of the corresponding group (multicast address) and the addresses of all sources that the host is interested in receiving messages from (inclusion). The record can also mention the source addresses from which the host does not desire to receive a group message (exclusion). The message is encapsulated in a datagram with the multicast address 224.0.0.22 (multicast address assigned to IGMPv3).

In IGMPv3, if a host needs to join a group, it waits until it receives a query message and then sends a report message. If a host needs to leave a group, it does not respond to

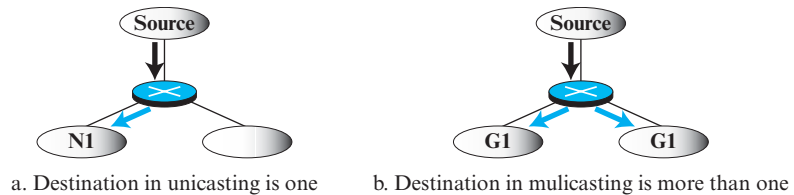
a query message. If no other host responds to the corresponding message, the group is purged from the router database.

Multicast Forwarding

Another important issue in multicasting is the decision a router needs to make to forward a multicast packet. Forwarding in unicast and multicast communication is different in two aspects:

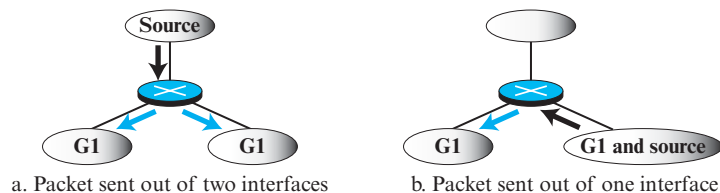
1. In unicast communication, the destination address of the packet defines one single destination. The packet needs to be sent only out of one of the interfaces, the interface which is the branch in the shortest-path tree reaching the destination with the minimum cost. In multicast communication, the destination of the packet defines one group, but that group may have more than one member in the internet. To reach all of the destinations, the router may have to send the packet out of more than one interface. Figure 4.94 shows the concept. In unicasting, the destination network N1 cannot be in more than one part of the internet; in multicasting, the group G1 may have members in more than one part of the internet.

Figure 4.94 Destination in unicasting and multicasting



2. Forwarding decisions in unicast communication depend only on the destination address of the packet. Forwarding decisions in multicast communication depend on both the destination and the source address of the packet. In other words, in unicasting, forwarding is based on where the packet should go; in multicasting, forwarding is based on where the packet should go and where the packet has come from. Figure 4.95 shows the concept. In part a of the figure, the source is in a section of the internet where there is no group member. In part b, the source is in a section where there is a group member. In part a, the router needs to send out the packet from two interfaces; in part b, the router should send the packet only from

Figure 4.95 Forwarding depends on the destination and the source



one interface to avoid sending a second copy of the packet from the interface it has arrived at. In other words, in part b of the figure, the member or members of the group G1 have already received a copy of the packet when it arrives at the router; sending out the packet in that direction does not help, but creates more traffic. This shows that the forwarding in multicast communication depends on both source and destination addresses.

Two Approaches to Multicasting

In multicast routing, as in unicast routing, we need to create routing trees to optimally route the packets from their source to their destination. However, as we discussed before, the multicast routing decision at each router depends not only on the destination of the packet, but also on the source of the packet. The involvement of the source in the routing process makes multicast routing much more difficult than unicast routing. For this reason, two different approaches in multicast routing have been developed: routing using source-based trees and routing using group-shared trees.

Source-Based Tree Approach

In the source-based tree approach to multicasting, each router needs to create a separate tree for each source-group combination. In other words, if there are m groups and n sources in the internet, a router needs to create $(m \times n)$ routing trees. In each tree, the corresponding source is the root, the members of the group are the leaves, and the router itself is somewhere on the tree. We can compare the situation with unicast routing in which a router needs only one tree with itself as the root and all networks in the internet as the leaves. Although it may appear that each router needs to create and store a huge amount of information about all of these trees, there are two protocols that use this approach in the Internet today, which we discuss later. These protocols use some strategies that alleviate the situation.

Group-Shared Tree Approach

In the group-shared tree approach, we designate a router to act as the phony source for each group. The designated router, which is called the *core* router or the *rendezvous-point* router, acts as the representative for the group. Any source that has a packet to send to a member of that group sends it to the core center (unicast communication) and the core center is responsible for multicasting. The core center creates one single routing tree with itself as the root and any routers with active members in the group as the leaves. In this approach, there are m core routers (one for each group) and each core router has a routing tree, for the total of m trees. This means that number of routing trees is reduced from $(m \times n)$ in the source-based tree approach to m in this approach. The reader may have noticed that we have divided a multicast delivery from the source to all group members into two deliveries. The first is a unicast delivery from the source to the core router; the second is the delivery from the core router to all group members. Note that the first part of the delivery needs to be done using tunneling. The multicast packet created by the source needs to be encapsulated in a unicast packet and sent to the core router. The core router decapsulates the unicast packet, extracts the multicast packet, and sends it to the group members. Although the reduction in number of trees in this approach looks very attractive, this approach has its own overhead: using an algorithm to select a router among all routers as the core router for a group.

4.4.3 Intradomain Routing Protocols

During the last few decades, several intradomain multicast routing protocols have emerged. In this section, we discuss three of these protocols. Two are extensions of unicast routing protocols (RIP and OSPF), using the source-based tree approach; the third is an independent protocol which is becoming more and more popular. It can be used in two modes, employing either the source-based tree approach or the shared-group tree approach.

Multicast Distance Vector (DVMRP)

The **Distance Vector Multicast Routing Protocol (DVMRP)** is the extension of the Routing Information Protocol (RIP) which is used in unicast routing. It uses the source-based tree approach to multicasting. It is worth mentioning that each router in this protocol that receives a multicast packet to be forwarded implicitly creates a source-based multicast tree in three steps:

1. The router uses an algorithm called *reverse path forwarding* (RPF) to simulate creating part of the optimal source-based tree between the source and itself.
2. The router uses an algorithm called *reverse path broadcasting* (RPB) to create a broadcast (spanning) tree whose root is the router itself and whose leaves are all networks in the internet.
3. The router uses an algorithm called *reverse path multicasting* (RPM) to create a multicast tree by cutting some branches of the tree that end in networks with no member in the group.

Reverse Path Forwarding (RPF)

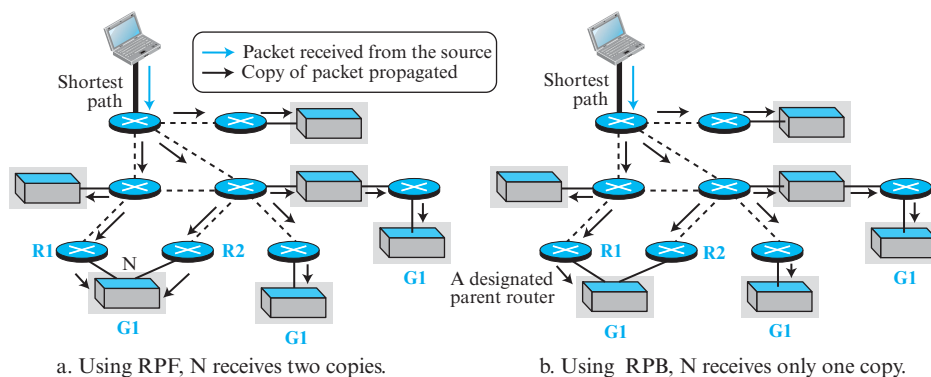
The first algorithm, **reverse path forwarding (RPF)**, forces the router to forward a multicast packet from one specific interface: the one which has come through the shortest path from the source to the router. How can a router know which interface is in this path if the router does not have a shortest-path tree rooted at the source? The router uses the first property of the shortest-path tree we discussed in unicast routing, which says that the shortest path from A to B is also the shortest path from B to A. The router does not know the shortest path from the source to itself, but it can find which is the next router in the shortest path from itself to the source (reverse path). The router simply consults its unicast forwarding table, pretending that it wants to send a packet to the source; the forwarding table gives the next router and the interface the message that the packet should be sent out from in this reverse direction. The router uses this information to accept a multicast packet only if it arrives from this interface. This is needed to prevent looping. In multicasting, a packet may arrive at the same router that has forwarded it. If the router does not drop all arrived packets except the one, multiple copies of the packet will be circulating in the internet. Of course, the router may add a tag to the packet when it arrives the first time and discard packets that arrive with the same tag, but the RPF strategy is simpler.

Reverse Path Broadcasting (RPB)

The RPF algorithm helps a router to forward only one copy received from a source and drop the rest. However, when we think about broadcasting in the second step, we need to remember that destinations are all the networks (LANs) in the internet. To be efficient, we need to prevent each network from receiving more than one copy of the packet. If a

network is connected to more than one router, it may receive a copy of the packet from each router. RPF cannot help here, because a network does not have the intelligence to apply the RPF algorithm; we need to allow only one of the routers attached to a network to pass the packet to the network. One way to do so is to designate only one router as the *parent* of a network related to a specific source. When a router that is not the parent of the attached network receives a multicast packet, it simply drops the packet. There are several ways that the parent of the network related to a network can be selected; one way is to select the router that has the shortest path to the source (using the unicast forwarding table, again in the reverse direction). If there is a tie in this case, the router with the smaller IP address can be selected. The reader may have noticed that RPB actually creates a broadcast tree from the graph that has been created by the RPF algorithm. RPB has cut those branches of the tree that cause cycles in the graph. If we use the shortest path criteria for choosing the parent router, we have actually created a shortest-path broadcast tree. In other words, after this step, we have a shortest-path tree with the source as the root and all networks (LANs) as the leaves. Every packet started from the source reaches all LANs in the internet travelling the shortest path. Figure 4.96 shows how RPB can avoid duplicate reception in a network by assigning a designated parent router, R1, for network N.

Figure 4.96 RPF versus RPB

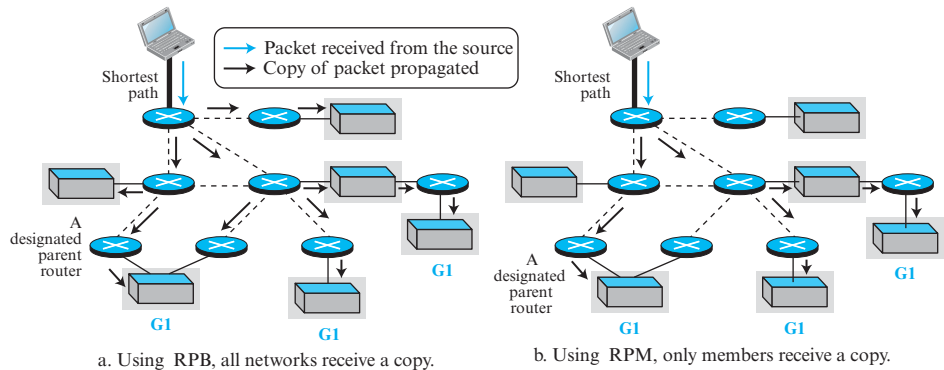


Reverse Path Multicasting (RPM)

As you may have noticed, RPB does not multicast the packet, it broadcasts it. This is not efficient. To increase efficiency, the multicast packet must reach only those networks that have active members for that particular group. This is called **reverse path multicasting (RPM)**. To change the broadcast shortest-path tree to a multicast shortest-path tree, each router needs to prune (make inactive) the interfaces that do not reach a network with active members corresponding to a particular source-group combination. This step can be done bottom-up, from the leaves to the root. At the leaf level, the routers connected to the network collect the membership information using the IGMP protocol discussed before. The parent router of the network can then disseminate this information upward using the reverse shortest-path tree from the router to the source, the same way as the distance vector messages are passed from one neighbor to another. When

a router receives all of these membership-related messages, it knows which interfaces need to be pruned. Of course, since these packets are disseminated periodically, if a new member is added to some networks, all routers are informed and can change the status of their interfaces accordingly. Joining and leaving is continuously applied. Figure 4.97 shows how pruning in RPM lets only networks with group members receive a copy of the packet unless they are in the path to a network with a member.

Figure 4.97 RPB versus RPM



Multicast Link State (MOSPF)

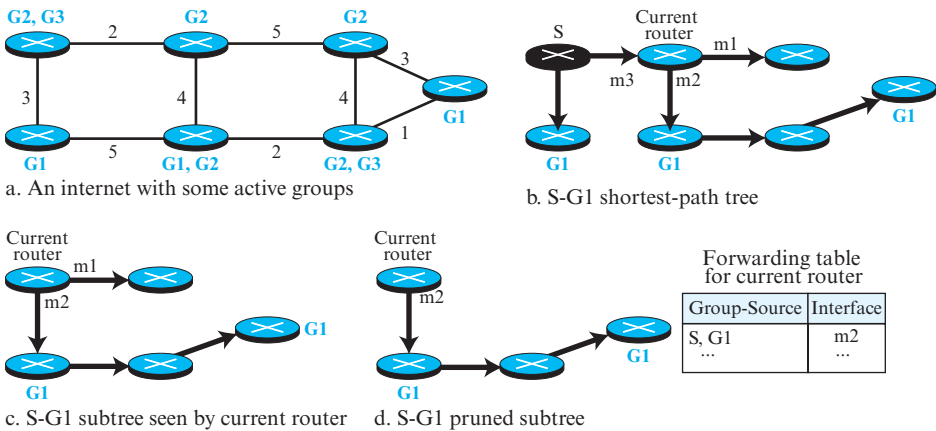
Multicast Open Shortest Path First (MOSPF) is the extension of the Open Shortest Path First (OSPF) protocol, which is used in unicast routing. It also uses the source-based tree approach to multicasting. If the internet is running a unicast link-state routing algorithm, the idea can be extended to provide a multicast link-state routing algorithm. Recall that in unicast link-state routing, each router in the internet has a link-state database (LSDB) that can be used to create a shortest-path tree. To extend unicasting to multicasting, each router needs to have another database, as with the case of unicast distance-vector routing, to show which interface has an active member in a particular group. Now a router goes through the following steps to forward a multicast packet received from source *S* and to be sent to destination *G* (a group of recipients):

1. The router uses the Dijkstra algorithm to create a shortest-path tree with *S* as the root and all destinations in the internet as the leaves. Note that this shortest-path tree is different from the one the router normally uses for unicast forwarding, in which the root of the tree is the router itself. In this case, the root of the tree is the source of the packet defined in the source address of the packet. The router is capable of creating this tree because it has the LSDB, the whole topology of the internet; the Dijkstra algorithm can be used to create a tree with any root, no matter which router is using it. The point we need to remember is that the shortest-path tree created this way depends on the specific source. For each source we need to create a different tree.
2. The router finds itself in the shortest-path tree created in the first step. In other words, the router creates a shortest-path subtree with itself as the root of the subtree.

3. The shortest-path subtree is actually a broadcast subtree with the router as the root and all networks as the leaves. The router now uses a strategy similar to the one we describe in the case of DVMRP to prune the broadcast tree and to change it to a multicast tree. The IGMP protocol is used to find the information at the leaf level. MOSPF has added a new type of link state update packet that floods the membership to all routers. The router can use the information it receives in this way and prune the broadcast tree to make the multicast tree.
4. The router can now forward the received packet out of only those interfaces that correspond to the branches of the multicast tree. We need to make certain that a copy of the multicast packet reaches all networks that have active members of the group and that it does not reach those networks that do not.

Figure 4.98 shows an example of using the steps to change a graph to a multicast tree. We have not shown the network for simplicity, but we added the groups to each router. The figure shows how a source-based tree is made with the source as the root and changed to a multicast subtree with the root at the current router.

Figure 4.98 Example of tree formation in MOSPF



Protocol Independent Multicast (PIM)

Protocol Independent Multicast (PIM) is the name given to a common protocol that needs a unicast routing protocol for its operation, but the unicast protocol can be either a distance-vector protocol or a link-state protocol. In other words, PIM needs to use the forwarding table of a unicast routing protocol to find the next router in a path to the destination, but it does not matter how the forwarding table is created. PIM has another interesting feature: it can work in two different modes: dense and sparse. The term *dense* here means that the number of active members of a group in the internet is large; the probability that a router has a member in a group is high. This may happen, for example, in a popular teleconference that has a lot of members. The term *sparse*, on the other hand, means that only a few routers in the internet have active members in the group; the probability that a router has a member of the group is low. This may happen, for example, in a very technical teleconference where a number of members are spread

somewhere in the internet. When the protocol is working in the dense mode, it is referred to as PIM-DM; when it is working in the sparse mode, it is referred to as PIM-SM. We explain both protocols next.

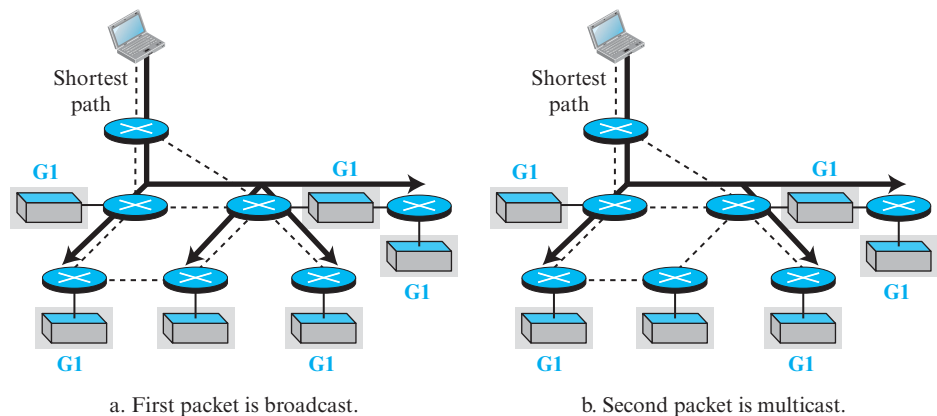
Protocol Independent Multicast-Dense Mode (PIM-DM)

When the number of routers with attached members is large relative to the number of routers in the internet, PIM works in the dense mode and is called *PIM-DM*. In this mode, the protocol uses a source-based tree approach and is similar to DVMRP, but simpler. PIM-DM uses only two strategies described in DVMRP: RPF and RPM. But unlike DVMRP, forwarding of a packet is not suspended awaiting pruning of the first subtree. Let us explain the two steps used in PIM-DM to clear the matter.

1. A router that has received a multicast packet from the source *S* destined for the group *G*, first uses the RPF strategy to avoid receiving a duplicate of the packet. It consults the forwarding table of the underlying unicast protocol to find the next router if it wants to send a message to the source *S* (in the reverse direction). If the packet has not arrived from the next router in the reverse direction, it drops the packet and sends a prune message in that direction to prevent receiving future packets related to (*S*, *G*).
2. If the packet in the first step has arrived from the next router in the reverse direction, the receiving router forwards the packet from all its interfaces except the one from which the packet has arrived and the interface from which it has already received a prune message related to (*S*, *G*). Note that this is actually a broadcasting instead of a multicasting if the packet is the first packet from the source *S* to group *G*. However, each router down the stream that receives an unwanted packet sends a prune message to the router up the stream, and eventually the broadcasting is changed to multicasting. Note that DVMRP behaves differently: it requires that the prune messages (which are part of DV packets) arrive and the tree is pruned before sending any message through unpruned interfaces. PIM-DM does not care about this precaution because it assumes that most routers have an interest in the group (the idea of the dense mode).

Figure 4.99 shows the idea behind PIM-DM. The first packet is broadcast to all networks, which have or do not have members. After a prune message arrives from a router with no member, the second packet is only multicast.

Figure 4.99 Idea behind PIM-DM



Protocol Independent Multicast-Sparse Mode (PIM-SM)

When the number of routers with attached members is small relative to the number of routers in the internet, PIM works in the sparse mode and is called *PIM-SM*. In this environment, the use of a protocol that broadcasts the packets until the tree is pruned is not justified; PIM-SM uses a group-shared tree approach to multicasting. The core router in PIM-SM is called the *rendezvous point* (RP). Multicast communication is achieved in two steps. Any router that has a multicast packet to send to a group of destinations first encapsulates the multicast packet in a unicast packet (tunneling) and sends it to the RP. The RP then decapsulates the unicast packet and sends the multicast packet to its destination.

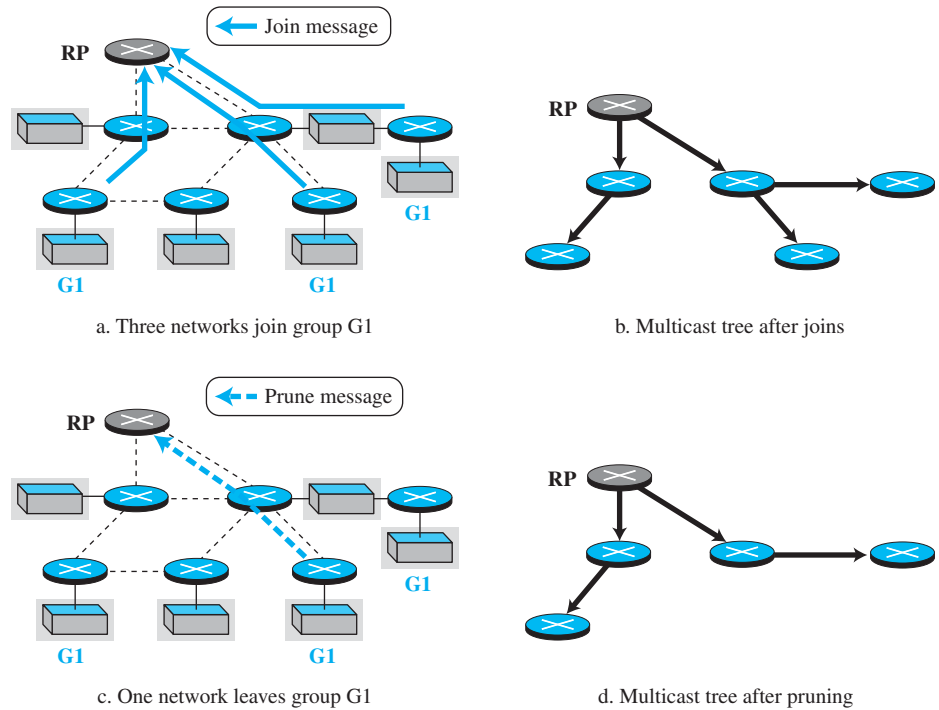
PIM-SM uses a complex algorithm to select one router among all routers in the internet as the RP for a specific group. This means that if we have m active groups, we need m RPs, although a router may serve more than one group. After the RP for each group is selected, each router creates a database and stores the group identifier and the IP address of the RP for tunneling multicast packets to it.

PIM-SM uses a spanning multicast tree rooted at the RP with leaves pointing to designated routers connected to each network with an active member. A very interesting point in PIM-SM is the formation of the multicast tree for a group. The idea is that each router helps to create the tree. The router should know the unique interface from which it should accept a multicast packet destined for a group (what was achieved by RPF in DVMRP). The router should also know the interface or interfaces from which it should send out a multicast packet destined for a group (what was achieved by RPM in DVMRP). To avoid delivering more than one copy of the same packet to a network through several routers (what was achieved by RPB in DVMRP), PIM-SM requires that designated routers only send PIM-SM messages, as we will see shortly.

To create a multicast tree rooted at the RP, PIM-SM uses *join* and *prune* messages. Figure 4.100 shows the operation of join and prune messages in PIM-SM. First, three networks join group G1 and form a multicast tree. Later, one of the networks leaves the group and the tree is pruned.

The join message is used to add possible new branches to the tree; the prune message is used to cut branches that are not needed. When a designated router finds out that a network has a new member in the corresponding group (via IGMP), it sends a join message in a unicast packet destined for the RP. The packet travels through the unicast shortest-path tree to reach the RP. Any router in the path receives and forwards the packet, but at the same time, the router adds two pieces of information to its multicast forwarding table. The number of the interface through which the join message has arrived is marked (if not already) as one of the interfaces through which the multicast packet destined for the group should be sent out in the future. The router also adds a count to the number of join messages received here, as we discuss shortly. The number of the interface through which the join message was sent to the RP is marked (if not already) as the only interface through which the multicast packet destined for the same group should be received. In this way, the first join message sent by a designated router creates a path from the RP to one of the networks with group members.

To avoid sending multicast packets to networks with no members, PIM-SM uses the prune message. Each designated router that finds out (via IGMP) that there is no active member in its network, sends a prune message to the RP. When a router receives

Figure 4.100 Idea behind PIM-SM

a prune message, it decrements the join count for the interface through which the message has arrived and forwards it to the next router. When the join count for an interface reaches zero, that interface is not part of the multicast tree anymore.

4.4.4 Interdomain Routing Protocols

The three protocols we discussed for multicast routing, DVMRP, MOSPF, and PIM, are designed to provide multicast communication inside an autonomous system. When the members of the groups are spread among different domains (ASs), we need an interdomain multicast routing protocol.

One common protocol for interdomain multicast routing is called *Multicast Border Gateway Protocol* (MBGP), which is the extension of BGP protocol we discussed for interdomain unicast routing. MBGP provides two paths between ASs: one for unicasting and one for multicasting. Information about multicasting is exchanged between border routers in different ASs. MBGP is a shared-group multicast routing protocol in which one router in each AS is chosen as the rendezvous point (RP).

The problem with MBGP protocol is that it is difficult to inform an RP about the sources of groups in other ASs. The Multicast Source Discovery Protocol (MSDP) is a new suggested protocol that assigns a source representative router in each AS to inform all RPs about the existence of sources in that AS.

Another protocol that is thought of as a possible replacement for the MBGP is Border Gateway Multicast Protocol (BGMP), which allows construction of shared-group trees with a single root in one of the ASs. In other words, for each group, there is only one shared tree, with leaves in different ASs, but the root is located in one of the ASs. The two problems, of course, are how to designate an AS as the root of the tree and how to inform all sources about the location of the root to tunnel their multicast packets to.

4.5 NEXT GENERATION IP

In the last section of this chapter, we discuss the new generation of IP, IPv6. The address depletion of IPv4 and other shortcomings of this protocol prompted a new version of IP protocol in the early 1990s. The new version, which is called **Internet Protocol version 6 (IPv6)** or **IP new generation (IPng)** was a proposal to augment the address space of IPv4 and at the same time redesign the format of the IP packet and revise some auxiliary protocols such as ICMP. It is interesting to know that IPv5 was a proposal, based on the OSI model, that never materialized. The following shows the main changes in the IPv6 protocol:

- ❑ **Larger address space.** An IPv6 address is 128 bits long. Compared with the 32-bit address of IPv4, this is a huge (2^{96} times) increase in the address space.
- ❑ **Better header format.** IPv6 uses a new header format in which options are separated from the base header and inserted, when needed, between the base header and the data. This simplifies and speeds up the routing process because most of the options do not need to be checked by routers.
- ❑ **New options.** IPv6 has new options to allow for additional functionalities.
- ❑ **Allowance for extension.** IPv6 is designed to allow the extension of the protocol if required by new technologies or applications.
- ❑ **Support for resource allocation.** In IPv6, the type-of-service field has been removed, but two new fields, traffic class and flow label, have been added to enable the source to request special handling of the packet. This mechanism can be used to support traffic such as real-time audio and video.
- ❑ **Support for more security.** The encryption and authentication options in IPv6 provide confidentiality and integrity of the packet.

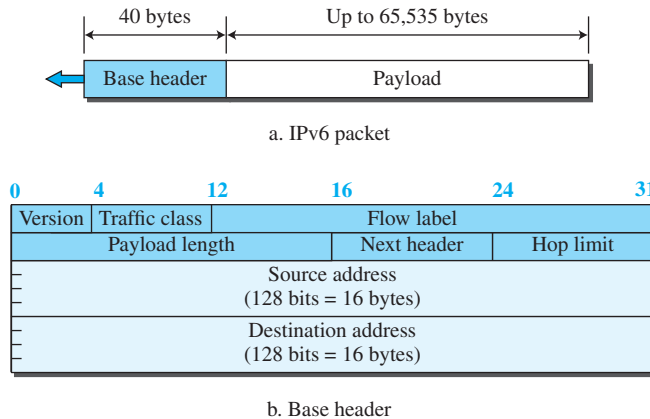
The adoption of IPv6 has been slow. The reason is that the original motivation for its development, depletion of IPv4 addresses, has been slowed down because of three short-term remedies: classless addressing, use of DHCP for dynamic address allocation, and NAT. However, the fast-spreading use of the Internet, and new services, such as mobile IP, IP telephony, and IP-capable mobile telephony, may require the total replacement of IPv4 with IPv6. It was predicted that all hosts in the world would be using IPv6 in 2010, but this did not happen. The recent prediction is 2020.

In this section, we first discuss the IPv6 packet format. We then discuss IPv6 addressing, which follows an architecture somewhat different from the architecture of version 4. We next show how the difficult task of transitioning from the old version to the new version can take place using some recommended strategies. Finally, we discuss ICMPv6, the only auxiliary protocol in the network layer, which replaces several protocols in version 4.

4.5.1 Packet Format

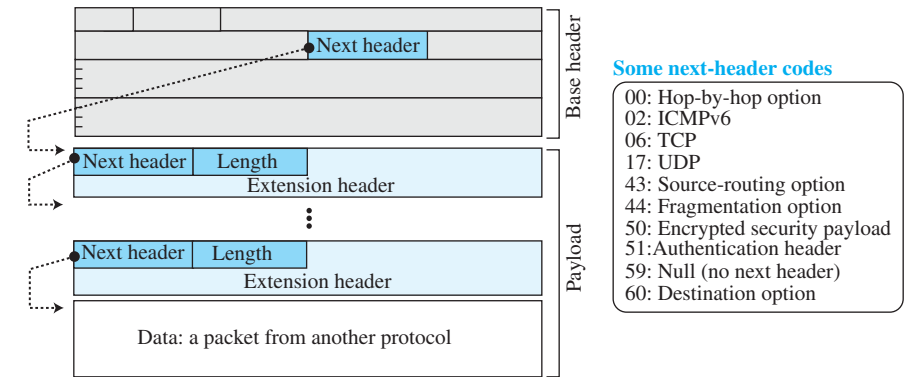
The IPv6 packet is shown in Figure 4.101. Each packet is composed of a base header followed by the payload. The base header occupies 40 bytes, whereas payload can be up to 65,535 bytes of information. The description of fields follows.

Figure 4.101 IPv6 datagram



- ❑ **Version.** The 4-bit version field defines the version number of the IP. For IPv6, the value is 6.
- ❑ **Traffic class.** The 8-bit traffic class field is used to distinguish different payloads with different delivery requirements. It replaces the *type-of-service* field in IPv4.
- ❑ **Flow label.** The flow label is a 20-bit field that is designed to provide special handling for a particular flow of data. We will discuss this field later.
- ❑ **Payload length.** The 2-byte payload length field defines the length of the IP datagram excluding the header. Note that IPv4 defines two fields related to the length: header length and total length. In IPv6, the length of the base header is fixed (40 bytes); only the length of the payload needs to be defined.
- ❑ **Next header.** The next header is an 8-bit field defining the type of first extension header (if present) or the type of the data that follows the base header in the datagram. This field is similar to the protocol field in IPv4, but we talk more about it when we discuss the payload.
- ❑ **Hop limit.** The 8-bit hop limit field serves the same purpose as the TTL field in IPv4.
- ❑ **Source and destination address.** The source address field is a 16-byte (128-bit) Internet address that identifies the original source of the datagram. The destination address field is a 16-byte (128-bit) Internet address that identifies the destination of the datagram.
- ❑ **Payload.** Compared to IPv4, the payload field in IPv6 has a different format and meaning, as shown in Figure 4.102.

Figure 4.102 Payload in an IPv6 datagram



The payload in IPv6 means a combination of zero or more extension headers (options) followed by the data from other protocols (UDP, TCP, and so on). In IPv6, options, which are part of the header in IPv4, are designed as extension headers. The payload can have as many extension headers as required by the situation. Each extension header has two mandatory fields, next header and the length, followed by information related to the particular option. Note that each next header field value (code) defines the type of the next header (hop-by-hop option, source-routing option, . . .); the last next header field defines the protocol (UDP, TCP, . . .) that is carried by the datagram.

Concept of Flow and Priority in IPv6

The IP protocol was originally designed as a connectionless protocol. However, the tendency is to use the IP protocol as a connection-oriented protocol. The MPLS technology described earlier allows us to encapsulate an IPv4 packet in an MPLS header using a label field. In version 6, the flow label has been directly added to the format of the IPv6 datagram to allow us to use IPv6 as a connection-oriented protocol.

To a router, a flow is a sequence of packets that share the same characteristics, such as traveling the same path, using the same resources, having the same kind of security, and so on. A router that supports the handling of flow labels has a flow label table. The table has an entry for each active flow label; each entry defines the services required by the corresponding flow label. When the router receives a packet, it consults its flow label table to find the corresponding entry for the flow label value defined in the packet. It then provides the packet with the services mentioned in the entry. However, note that the flow label itself does not provide the information for the entries of the flow label table; the information is provided by other means, such as the hop-by-hop options or other protocols.

In its simplest form, a flow label can be used to speed up the processing of a packet by a router. When a router receives a packet, instead of consulting the forwarding table and going through a routing algorithm to define the address of the next hop, it can easily look in a flow label table for the next hop.

In its more sophisticated form, a flow label can be used to support the transmission of real-time audio and video. Real-time audio or video, particularly in digital form, requires resources such as high bandwidth, large buffers, long processing time, and so on. A process can make a reservation for these resources beforehand to guarantee that real-time data will not be delayed due to a lack of resources. The use of real-time data and the reservation of these resources require other protocols such as Real-Time Transport Protocol (RTP) and Resource Reservation Protocol (RSVP) in addition to IPv6 (see Chapter 8).

Fragmentation and Reassembly

There is still fragmentation and reassembly of datagrams in the IPv6 protocol, but there is a major difference in this respect. IPv6 datagrams can be fragmented only by the source, not by the routers; the reassembly takes place at the destination. The fragmentation of packets at routers is not allowed to speed up the processing of packets in the router. The fragmentation of a packet in a router needs a lot of processing. The packet needs to be fragmented, all fields related to the fragmentation need to be recalculated. In IPv6, the source can check the size of the packet and make the decision to fragment the packet or not. When a router receives the packet, it can check the size of the packet and drop it if the size is larger than allowed by the MTU of the network ahead. The router then sends a packet-too-big ICMPv6 error message (discussed later) to inform the source.

Extension Headers

Extension headers play a necessary and important part in IPv6. In particular, three extension headers—fragmentation, authentication, and extended security payload—are present in some packets. To cut the length of the chapter, we have cut the discussion of extension headers, but they are included in the book website as part of extra materials for Chapter 4.

4.5.2 IPv6 Addressing

The main reason for migration from IPv4 to IPv6 is the small size of the address space in IPv4. In this section, we show how the huge address space of IPv6 prevents address depletion in the future. We also discuss how the new addressing responds to some problems in the IPv4 addressing mechanism. An IPv6 address is 128 bits or 16 bytes (octets) long, four times the address length in IPv4.

A computer normally stores the address in binary, but it is clear that 128 bits cannot easily be handled by humans. Several notations have been proposed to represent IPv6 addresses when they are handled by humans. The following shows two of these notations: binary and colon hexadecimal.

Binary	1111111011110110 ... 1111111100000000
Colon Hexadecimal	FEF6:BA98:7654:3210:ADEF:BBFF:2922:FF00

Binary notation is used when the addresses are stored in a computer. The colon hexadecimal notation (or colon hex for short) divides the address into eight sections, each made of four hexadecimal digits separated by colons.

Although an IPv6 address, even in hexadecimal format, is very long, many of the digits are zeros. In this case, we can abbreviate the address. The leading zeros of a section can be omitted. Using this form of abbreviation, 0074 can be written as 74, 000F as

F, and 0000 as 0. Note that 3210 cannot be abbreviated. Further abbreviation, often called zero compression, can be applied to colon hex notation if there are consecutive sections consisting of zeros only. We can remove all the zeros altogether and replace them with a double semicolon.

FDEC:0:0:0:BBFF:0:FFFF → **FDEC::BBFF:0:FFFF**

Note that this type of abbreviation is allowed only once per address. If there is more than one run of zero sections, only one of them can be compressed.

Sometimes we see a mixed representation of an IPv6 address: colon hex and dotted-decimal notation. This is appropriate during the transition period in which an IPv4 address is embedded in an IPv6 address (as the rightmost 32 bits). We can use the colon hex notation for the leftmost six sections and four-byte dotted-decimal notation instead of the rightmost two sections. However, this happens when all or most of the leftmost sections of the IPv6 address are 0s. For example, the address (**::130.24.24.18**) is a legitimate address in IPv6, in which the zero compression shows that all 96 leftmost bits of the address are zeros.

As we see will shortly, IPv6 uses hierarchical addressing. For this reason, IPv6 allows slash or CIDR notation. For example, the following shows how we can define a prefix of 60 bits using CIDR. We will later show how an IPv6 address is divided into a prefix and a suffix.

FDEC::BBFF:0:FFFF/60

Address Space

The address space of IPv6 contains 2^{128} addresses. This address space is 2^{96} times the IPv4 address—definitely no address depletion—as shown, the size of the space is

340, 282, 366, 920, 938, 463, 374, 607, 431, 768, 211, 456.

To give some idea about the number of addresses, we assume that only 1/64 (almost 2 percent) of the addresses in the space can be assigned to the people on the planet Earth and the rest are reserved for special purposes. We also assume that the number of people on the earth is soon to be 2^{34} (more than 16 billion). Each person can have 2^{88} addresses to use. Address depletion in this version is impossible.

Three Address Types

In IPv6, a destination address can belong to one of three categories: unicast, anycast, and multicast. A unicast address defines a single interface (computer or router). The packet sent to a unicast address will be routed to the intended recipient.

An anycast address defines a group of computers that all share a single address. A packet with an anycast address is delivered to only one member of the group, the most reachable one. An anycast communication is used, for example, when there are several servers that can respond to an inquiry. The request is sent to the one that is most reachable. The hardware and software generate only one copy of the request; the copy reaches only one of the servers. IPv6 does not designate a block for anycasting; the addresses are assigned from the unicast block.

A multicast address also defines a group of computers. However, there is a difference between anycasting and multicasting. In anycasting, only one copy of the packet is

sent to one of the members of the group; in multicasting each member of the group receives a copy. As we will see shortly, IPv6 has designated a block for multicasting from which the same address is assigned to the members of the group.

It is interesting that IPv6 does not define broadcasting, even in a limited version. As we will see, IPv6 considers broadcasting as a special case of multicasting.

Address Space Allocation

Like the address space of IPv4, the address space of IPv6 is divided into several blocks of varying size and each block is allocated for a special purpose. Most of the blocks are still unassigned and have been set aside for future use. Table 4.7 shows only the assigned blocks. In this table, the last column shows the fraction each block occupies in the whole address space.

Table 4.7 Prefixes for assigned IPv6 addresses

Block prefix	CIDR	Block assignment	Fraction
0000 0000	0000::/8	Special addresses	1/256
001	2000::/3	Global unicast	1/8
1111 110	FC00::/7	Unique local unicast	1/128
1111 1110 10	FE80::/10	Link local addresses	1/1024
1111 1111	FF00::/8	Multicast addresses	1/256

Global Unicast Addresses

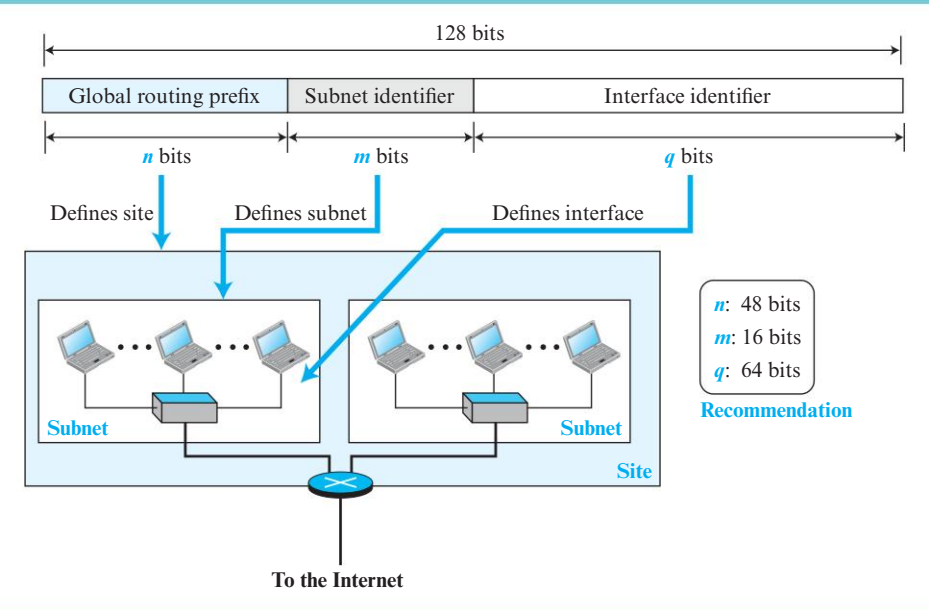
The block in the address space that is used for unicast (one-to-one) communication between two hosts in the Internet is called the *global unicast address block*. CIDR for the block is 2000::/3, which means that the three leftmost bits are the same for all addresses in this block (001). The size of this block is 2^{125} bits, which is more than enough for Internet expansion for many years to come. An address in this block is divided into three parts: *global routing prefix* (n bits), *subnet identifier* (m bits), and *interface identifier* (q bits), as shown in Figure 4.103. The figure also shows the recommended length for each part.

The global routing prefix is used to route the packet through the Internet to the organization site, such as the ISP that owns the block. Since the first three bits in this part are fixed (001), the rest of the 45 bits can be defined for up to 2^{45} sites (a private organization or an ISP). The global routers in the Internet route a packet to its destination site based on the value of n . The next m bits (16 bits based on recommendation) define a subnet in an organization. This means that an organization can have up to $2^{16} = 65,536$ subnets, which is more than enough.

The last q bits (64 bits based on recommendation) define the interface identifier. The interface identifier is similar to hostid in IPv4 addressing although the term interface identifier is a better choice because, as we discussed earlier, the host identifier actually defines the interface, not the host. If the host is moved from one interface to another, its IP address needs to be changed.

In IPv4 addressing, there is not a specific relation between the hostid (at the IP level) and link-layer address (at the data-link layer) because the link-layer address is normally much longer than the hostid. The IPv6 addressing allows this relationship. A link-layer address whose length is less than 64 bits can be embedded as the whole or

Figure 4.103 Global unicast address

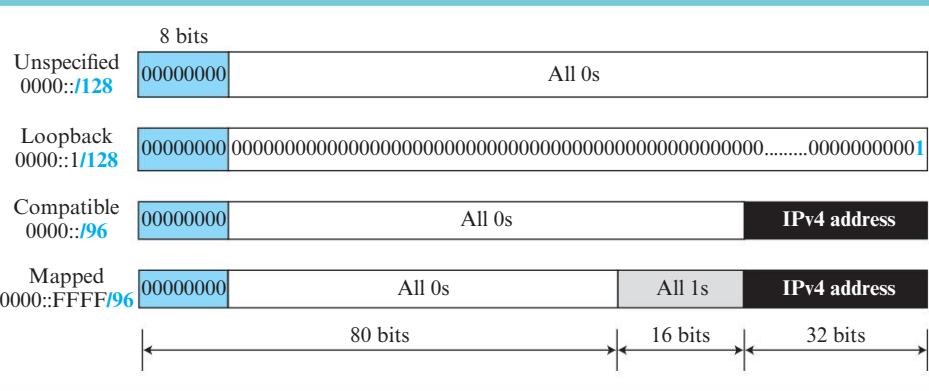


part of the interface identifier, eliminating the mapping process. Two common link-layer addressing schemes can be considered for this purpose: the 64-bit extended unique identifier (EUI-64) defined by IEEE and the 48-bit link-layer address defined by Ethernet.

Special Addresses

After discussing the global unicast block, let us discuss the characteristics and purposes of assigned and reserved blocks in the first row of Table 4.7. Addresses that use the prefix (0000::/8) are reserved, but part of this block is used to define some special addresses. Figure 4.104 shows the assigned addresses in this block.

Figure 4.104 Special addresses



The unspecified address is a subblock containing only one single address, which is used during bootstrap when a host does not know its own address and wants to send an inquiry to find it (see DHCP section).

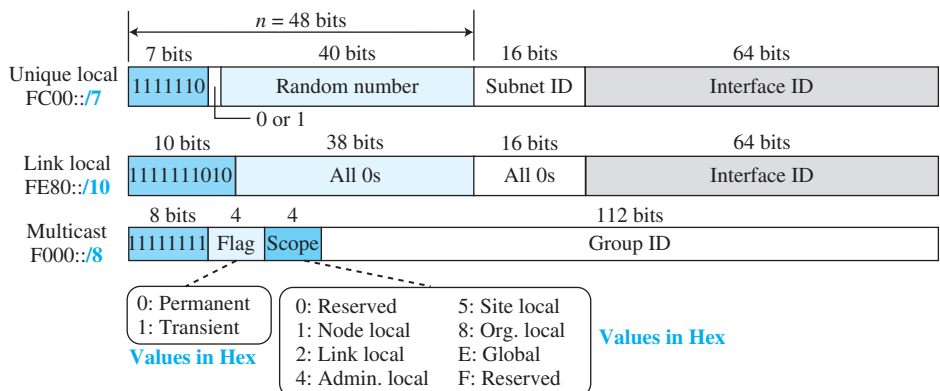
The loopback address also consists of one single address. We discussed loopback addresses for IPv4 before. In IPv4 the block is made of a range of addresses; in IPv6, the block has only a single address in it.

As we will see later, during the transition from IPv4 to IPv6, hosts can use their IPv4 addresses embedded in IPv6 addresses. Two formats have been designed for this purpose: compatible and mapped. A **compatible address** is an address of 96 bits of zero followed by 32 bits of IPv4 address. It is used when a computer using IPv6 wants to send a message to another computer using IPv6. A **mapped address** is used when a computer already migrated to version 6 wants to send an address to a computer still using version 4. A very interesting point about mapped and compatible addresses is that they are designed such that, when calculating the checksum, one can use either the embedded address or the total address because extra 0s or 1s in multiples of 16 do not have any effect in checksum calculation. This is important for UDP and TCP, which use a pseudoheader to calculate the checksum because the checksum calculation is not affected if the address of the packet is changed from IPv6 to IPv4 by a router.

Other Assigned Blocks

IPv6 uses two large blocks for private addressing and one large block for multicasting, as shown in Figure 4.105. A subblock in a unique local unicast block can be privately created and used by a site. The packet carrying this type of address as the destination address is not expected to be routed. This type of address has the identifier 1111 110, the next bit can be 0 or 1 to define how the address is selected (locally or by an authority). The next 40 bits are selected by the site using a randomly generated number of length 40 bits. This means that the total of 48 bits defines a subblock that looks like a global unicast address. The 40-bit random number makes the probability of duplication of the address extremely small. Note the similarity between the format of these addresses and the global unicast. The second block, designed for private addresses, is the link local block. A subblock in this block can be used as a

Figure 4.105 Unique local unicast block



private address in a network. This type of address has the block identifier 1111111010. The next 54 bits are set to zero. The last 64 bits can be changed to define the interface for each computer. Note the similarity between the format of these addresses and the global unicast address.

We discussed multicast addresses of IPv4 earlier in the chapter. Multicast addresses are used to define a group of hosts instead of just one. In IPv6 a large block of addresses are assigned for multicasting. All these addresses use the prefix 11111111. The second field is a flag that defines the group address as either permanent or transient. A permanent group address is defined by the Internet authorities and can be accessed at all times. A transient group address, on the other hand, is used only temporarily. Systems engaged in a teleconference, for example, can use a transient group address. The third field defines the scope of the group address. Many different scopes have been defined, as shown in the figure.

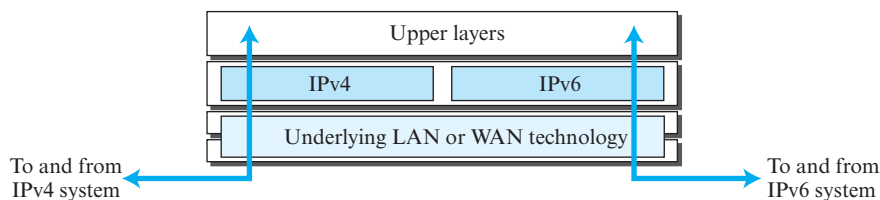
4.5.3 Transition from IPv4 to IPv6

Although we have a new version of the IP protocol, how can we make the transition to stop using IPv4 and start using IPv6? The first solution that comes to mind is to define a transition day on which every host or router should stop using the old version and start using the new version. However, this is not practical; because of the huge number of systems on the Internet, the transition from IPv4 to IPv6 cannot happen suddenly. It will take a considerable amount of time before every system in the Internet can move from IPv4 to IPv6. The transition must be smooth to prevent any problems between IPv4 and IPv6 systems. Three strategies have been devised by the IETF to help the transition: dual stack, tunneling, and header translation.

Dual Stack

It is recommended that all hosts, before migrating completely to version 6, have a dual stack of protocols during the transition. In other words, a station must run IPv4 and IPv6 simultaneously until all the Internet uses IPv6. See Figure 4.106 for the layout of a dual-stack configuration.

Figure 4.106 *Dual stack*



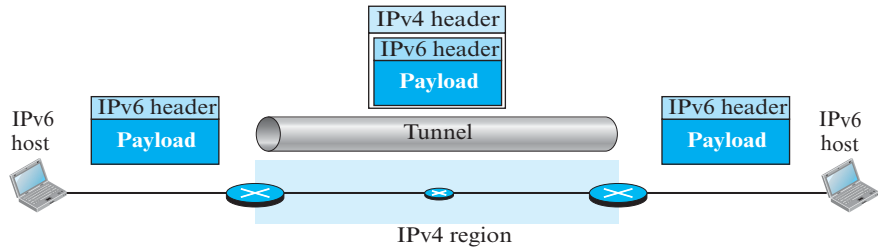
To determine which version to use when sending a packet to a destination, the source host queries the DNS. If the DNS returns an IPv4 address, the source host sends an IPv4 packet. If the DNS returns an IPv6 address, the source host sends an IPv6 packet.

Tunneling

Tunneling is a strategy used when two computers using IPv6 want to communicate with each other and the packet must pass through a region that uses IPv4. To pass

through this region, the packet must have an IPv4 address. So the IPv6 packet is encapsulated in an IPv4 packet when it enters the region, and it leaves its capsule when it exits the region. It seems as if the IPv6 packet goes through a tunnel at one end and emerges at the other end. To make it clear that the IPv4 packet is carrying an IPv6 packet as data, the protocol value is set to 41. Tunneling is shown in Figure 4.107.

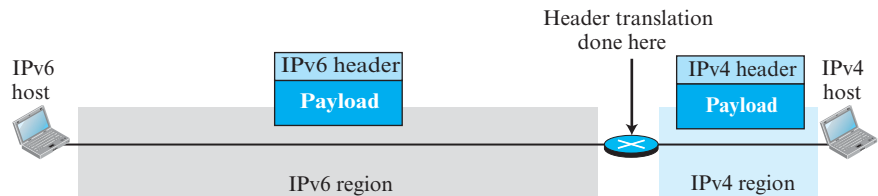
Figure 4.107 Tunneling strategy



Header Translation

Header translation is necessary when the majority of the Internet has moved to IPv6 but some systems still use IPv4. The sender wants to use IPv6, but the receiver does not understand IPv6. Tunneling does not work in this situation because the packet must be in the IPv4 format to be understood by the receiver. In this case, the header format must be totally changed through header translation. The header of the IPv6 packet is converted to an IPv4 header (see Figure 4.108).

Figure 4.108 Header translation strategy

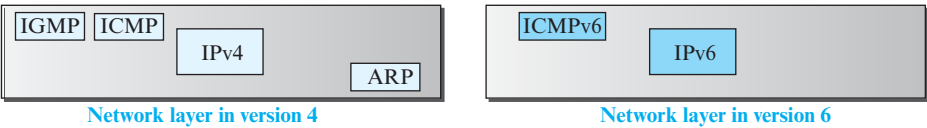


4.5.4 ICMPv6

Another protocol that has been modified in version 6 of the TCP/IP protocol suite is ICMP. This new version, Internet Control Message Protocol version 6 (ICMPv6), follows the same strategy and purposes of version 4. ICMPv6, however, is more complicated than ICMPv4: some protocols that were independent in version 4 are now part of ICMPv6 and some new messages have been added to make it more useful. Figure 4.109 compares the network layer of version 4 to that of version 6. The ICMP, ARP (discussed in Chapter 5), and IGMP protocols in version 4 are combined into one single protocol, ICMPv6.

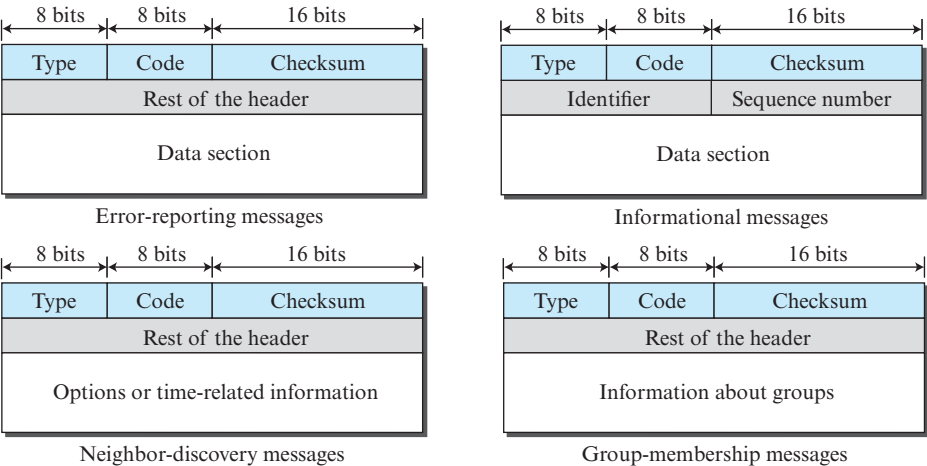
We can divide the messages in ICMPv6 into four groups: error-reporting messages, informational messages, neighbor-discovery messages, and group-membership messages.

Figure 4.109 Comparison of network layer in version 4 and version 6



The error-reporting messages follow the same policy as in version 4. There are four message types in this group: *destination-unreachable*, *packet-too-big*, *time-exceeded*, and *parameter-problem*. The source-quench message in version 4 was dropped because it was rarely used. The redirection message was moved to another category. A new message, *packet-too-big*, was added because, in version 6, a router is not allowed to do fragmentation. If a message is too big, the router will drop it and send a *packet-too-big* error message to the source (see Figure 4.110).

Figure 4.110 ICMPv6 messages



Type and code values

- | | |
|--|--|
| Error-reporting messages <ul style="list-style-type: none">01: Destination unreachable (codes 0 to 5)02: Packet too big (code 0)03: Time exceeded (codes 0 and 1)04: Parameter problem (codes 0 to 2) | Informational messages <ul style="list-style-type: none">128 and 129: Echo request and reply (code 0) |
| Neighbor-discovery messages <ul style="list-style-type: none">133 and 134: Router solicitation and advertisement (code 0)135 and 136: Neighbor solicitation and advertisement (code 0)137: Redirection (codes 0 to 3)141 and 142: Inverse neighbor solicitation and advertisement (code 0) | Group-membership messages <ul style="list-style-type: none">130: Membership query (code 0)131: Membership report |

Note: See the book website for more explanation about messages.

The informational messages in version 6 play the same role as the query messages in version 4. There is only one pair of messages in this group: the *echo-request* and *echo-reply* messages. Comparing this group with the one in version 4, we see that the two messages, timestamp request and timestamp reply, have been dropped because they were rarely used.

A new group, called neighbor-discovery messages, has been added in version 6. The messages in this group can find the routers in the neighborhood, find the link-layer addresses of other hosts in the neighborhood, find the IPv6 addresses of the hosts in the neighborhood, and redirect messages. In other words, this group can be divided into four subgroups. The *router-solicitation* and *router-advertisement* messages can help a host to find a default router to forward its packet to. The DHCP protocol in version 4 was responsible for this task. The *neighbor-solicitation* and *neighbor-advertisement* messages try to find the link-layer address of a host or router when the IPv6 address is given. As we discuss in Chapter 5, this task is done by the Address Resolution Protocol (ARP) in version 4. The *redirection* message is doing the same task as the one in version 4. The *inverse-neighbor-solicitation* and *inverse-neighbor-advertisement* messages are used to find the IPv6 address of a host or router when the link-layer address is given.

Another new group, the group-membership messages, has two messages: the *membership-query* and the *membership-report* messages that are used in multicasting. They do the same task as the IGMP messages described earlier in the chapter.

More discussion related to the ICMPv6 can be found in the book website as part of extra materials for Chapter 4.

4.6 END-CHAPTER MATERIALS

4.6.1 Further Reading

Books

Several books give thorough coverage of materials discussed in this chapter. We recommend [Com 06], [Tan 03], [Koz 05], [Ste 95], [G & W 04], [Per 00], [Kes 02], [Moy 98], [W & Z 01], and [Los 04].

RFCs

IPv4 addressing is discussed in RFCs 917, 927, 930, 932, 940, 950, 1122, and 1519. Forwarding is discussed in RFCs 1812, 1971, and 1980. MPLS is discussed in RFCs 3031, 3032, 3036, and 3212. IPv4 protocol is discussed in RFCs 791, 815, 894, 1122, 2474, and 2475. ICMP is discussed in RFCs 792, 950, 956, 957, 1016, 1122, 1256, 1305, and 1987. RIP is discussed in RFC 1058 and 2453. OSPF is discussed in RFCs 1583 and 2328. BGP is discussed in RFCs 1654, 1771, 1773, 1997, 2439, 2918, and 3392. Multicasting is discussed in RFCs 1075, 1585, 2189, 2362, and 3376. IPv6 addressing is discussed in RFCs 2375, 2526, 3513, 3587, 3789, RFC 4291. IPv6 protocol is discussed in RFC 2460, 2461, and 2462. ICMPv6 is discussed in RFC 2461, 2894, 3122, 3810, 4443, and 4620.

4.6.2 Key Terms

address aggregation
address space

anycast address
autonomous system (AS)

banyan switch	link-state routing
Bellman-Ford	longest mask matching
Border Gateway Protocol (BGP)	maximum transfer unit (MTU)
classful addressing	Multicast Open Shortest Path First (MOSPF)
classless addressing	network address
Classless Interdomain Routing (CIDR)	Network Address Translation (NAT)
compatible address	Open Shortest Path First (OSPF)
crossbar switch	path vector routing
Dijkstra's algorithm	poison reverse
direct broadcast address	Protocol Independent Multicast (PIM)
Distance Vector Multicast Routing Protocol (DVMRP)	reverse path broadcasting (RPB)
distance vector routing	reverse path forwarding (RPF)
Internet Control Message Protocol (ICMP)	reverse path multicasting (RPM)
Internet Group Management Protocol (IGMP)	Routing Information Protocol (RIP)
	split horizon

4.6.3 Summary

The Internet is made of many networks (or links) connected through connecting devices, each of which acts as a router or as a switch. Two types of switching are traditionally used in networking: circuit switching and packet switching. The network layer is designed as a packet-switched network.

The network layer supervises the handling of packets by the underlying physical networks. The delivery of a packet can be direct or indirect. Two categories of forwarding are defined: forwarding based on the destination address of the IP datagram and forwarding based on the label attached to an IP datagram.

IPv4 is an unreliable connectionless protocol responsible for source-to-destination delivery. Packets in the IP layer are called datagrams. The identifiers used in the IP layer of the TCP/IP protocol suite are called the IP addresses. An IPv4 address is 32 bits long, divided into two parts: the prefix and the suffix. All addresses in the block have the same prefix; each address has a different suffix.

The Internet Control Message Protocol (ICMP) supports the unreliable and connectionless Internet Protocol (IP).

To be able to route a packet, a router needs a forwarding table. Routing protocols are specific application programs that have the duty of updating forwarding tables.

Multicasting is the sending of the same message to more than one receiver simultaneously. The Internet Group Management Protocol (IGMP) is involved in collecting local membership group information.

IPv6, the latest version of the Internet Protocol, has a 128-bit address space. IPv6 uses hexadecimal colon notation with abbreviation methods available.

4.7 PRACTICE SET

4.7.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book website. It is strongly recommended that students take the quizzes to check his/her understanding of the materials before continuing with the practice set.

4.7.2 Questions

- Q4-1.** If a label in a connection-oriented service is 8-bits, how many virtual circuits can be established at the same time?
- Q4-2.** List the three phases in the virtual-circuit approach to switching.
- Q4-3.** Why does the network-layer protocol need to provide packetizing service to the transport layer? Why cannot the transport layer send out the segments without encapsulating them in datagrams?
- Q4-4.** Why is the routing the responsibility of the network layer? In other words, why cannot the routing be done at the transport layer or the data-link layer?
- Q4-5.** Distinguish between the process of routing a packet from the source to the destination and the process of forwarding a packet at each router.
- Q4-6.** What is the piece of information in a packet upon which the forwarding decision is made in each of the following approaches to switching?
a. datagram approach **b.** virtual-circuit approach
- Q4-7.** In Figure 4.10, assume that the link between R1 and R2 is upgraded to 170 Kbps and the link between the source host and R1 is now downgraded to 140 Kbps. What is the throughput between the source and destination after these changes? Which link is the bottleneck now?
- Q4-8.** Can the value of the header length field in an IPv4 packet be less than 5? When is it exactly 5?
- Q4-9.** A host is sending 100 datagrams to another host. If the identification number of the first datagram is 1024, what is the identification number of the last?
- Q4-10.** An IP fragment has arrived with an offset value of 100. How many bytes of data were originally sent by the source before the data in this fragment?
- Q4-11.** Do we have any of the following services at the network layer of TCP/IP? If not, why?
a. flow control **b.** error control **c.** congestion control
- Q4-12.** List four types of delays in a packet-switch network.
- Q4-13.** In classless addressing, can two different blocks have the same prefix length? Explain.
- Q4-14.** In classless addressing, we know the first address and one of the addresses in the block (not necessarily the last address). Can we find the prefix length? Explain.
- Q4-15.** In classless addressing, we know the first and the last address in the block. Can we find the prefix length? If yes, show the process.
- Q4-16.** In classless addressing, we know the first address and the number of addresses in the block. Can we find the prefix length? If yes, show the process.
- Q4-17.** Compare and contrast the protocol field at the network layer with the port numbers at the transport layer. What is their common purpose? Why do we need two port-number fields but only one protocol field? Why is the size of the protocol field only half the size of each port number?
- Q4-18.** Which field(s) in the datagram is(are) responsible for gluing together all fragments belonging to an original datagram?

- Q4-19.** Assume that a destination computer receives several packets from a source. How can it be sure that the fragments belonging to a datagram are not mixed with the fragments belonging to another datagram?
- Q4-20.** Explain why, using MPLS means adding a new layer to the TCP/IP protocol suite. Where is this layer located?
- Q4-21.** An ISP has a block of 1024 addresses. It needs to divide the addresses among 1024 customers. Does it need subnetting? Explain your answer.
- Q4-22.** Mention the three auxiliary protocols at the network layer of the TCP/IP suite that are designed to help the IPv4 protocol.
- Q4-23.** In an IPv4 datagram, the value of the header-length (HLEN) field is $(6)_{16}$. How many bytes of options have been added to the packet?
- Q4-24.** Can the value of the TTL in a datagram be any of the following? Explain your answer.
- a. 23 b. 0 c. 1 d. 301
- Q4-25.** In a graph, if we know that the shortest path from node A to node G is $(A \rightarrow B \rightarrow E \rightarrow G)$, what is the shortest path from node G to node A?
- Q4-26.** Assume that the shortest path in a graph from node A to node H is $A \rightarrow B \rightarrow H$. Also assume that the shortest path from node H to node N is $H \rightarrow G \rightarrow N$. What is the shortest path from node A to node N?
- Q4-27.** Explain why the Internet does not create a report message to report the error in an IP datagram that carries an ICMP message.
- Q4-28.** What are the source and destination IP addresses in a datagram that carries the ICMP message reported by a router?
- Q4-29.** Assume that we have an isolated AS running RIP. We can say that we have at least two different kinds of datagram traffic in this AS. The first kind carries the messages exchanged between hosts; the second carries messages belonging to RIP. What is the difference between the two kinds of traffic when we think about source and destination IP addresses? Does this show that routers also need IP addresses?
- Q4-30.** Router A sends two RIP messages to two immediate neighboring routers, B and C. Do the two datagrams carrying the messages have the same source IP addresses? Do the two datagrams have the same destination IP addresses?
- Q4-31.** At any moment, a RIP message may arrive at a router that runs RIP as the routing protocol. Does it mean that the RIP process should be running all the time?
- Q4-32.** Why do you think RIP uses UDP instead of TCP? Give reasons.
- Q4-33.** Explain why a router using link-state routing needs to receive the whole LSDB before creating and using its forwarding table. In other words, why cannot the router create its forwarding table with a partially received LSDB?
- Q4-34.** Is the path-vector routing algorithm closer to the distance-vector routing algorithm or to the link-state routing algorithm? Explain.
- Q4-35.** List three types of autonomous systems (ASs) described in the text, and make a comparison between them.
- Q4-36.** Explain the concept of hop count in RIP. Can you explain why no hop is counted between N1 and R1 in Figure 4.70?

- Q4-37.** Why do you think we need only one update RIP message, but several OSPF update messages?
- Q4-38.** OSPF messages are exchanged between routers. Does this mean that we need to have OSPF processes run all the time to be able to receive an OSPF message when it arrives?
- Q4-39.** OSPF messages and ICMP messages are directly encapsulated in an IP datagram. If we intercept an IP datagram, how can we tell whether the payload belongs to OSPF or ICMP?
- Q4-40.** Explain what type of OSPF link state is advertised in each of the following cases:
- a.** A router needs to advertise the existence of another router at the end of a point-to-point link.
 - b.** A router needs to advertise the existence of two stub networks and one transient network.
 - c.** A designated router advertises a network as a node.
- Q4-41.** We say that OSPF is a hierarchical intradomain protocol, but RIP is not. What is the reason behind this statement?
- Q4-42.** In a very small AS using OSPF, is it more efficient to use only one single area (backbone) or several areas?
- Q4-43.** Can you explain why BGP uses the services of TCP instead of UDP?
- Q4-44.** Explain why policy routing can be implemented on an interdomain routing, but it cannot be implemented on an intradomain routing.
- Q4-45.** Explain when each of the following attributes can be used in BGP:
- a.** LOCAL-PREF **b.** AS-PATH **c.** NEXT-HOP
- Q4-46.** Distinguish between multicasting and multiple-unicasting.
- Q4-47.** Can a router combine the advertisement of a link and a network in one single link state update?
- Q4-48.** Explain why we can have different intradomain routing protocols in different ASs, but we need only one interdomain routing protocol in the whole Internet?
- Q4-49.** Define the group of each of the following multicast addresses (local network control block, internetwork control block, SSM block, Glop block, or administratively scoped block):
- a.** 224.0.1.7 **b.** 232.7.14.8 **c.** 239.14.10.12
- Q4-50.** Can a host have more than one multicast address? Explain.
- Q4-51.** When we send an e-mail to multiple recipients, are we using multicasting or multiple unicasting? Give reason(s) for your answer.
- Q4-52.** Define which of the following addresses are multicast addresses:
- a.** 224.8.70.14 **b.** 226.17.3.53 **c.** 240.3.6.25
- Q4-53.** Assume that we have 20 hosts in a small AS. There are only four groups in this AS. Find the number of spanning trees in each of the following approaches:
- a.** source-based tree **b.** group-shared tree

- Q4-54.** We say that a router in DVMRP creates a shortest-path tree *on demand*. What is the meaning of this statement? What is the advantage of creating shortest-path trees only on demand?
- Q4-55.** List three steps that a DVMRP router uses to create a source-based tree. Which phase is responsible for creating the part of the tree from the source to the current router? Which phase is responsible for creating a broadcast tree with the router as the root? Which phase is responsible for changing the broadcast tree to a multicast tree?
- Q4-56.** Explain why an MOSPF router can create the shortest path with the source as the root in one step, but DVMRP needs three steps to do so.
- Q4-57.** Explain why PIM is called *Protocol Independent Multicast*.
- Q4-58.** Which version of PIM uses the first and the third steps of DVMRP? What are these two steps?
- Q4-59.** Explain why broadcasting the first or the first few messages is not important in PIM-DM, but it is important in PIM-SM?
- Q4-60.** Explain the advantages of IPv6 when compared to IPv4.
- Q4-61.** A multicast router is connected to four networks. The interest of each network is shown below, what is the group list that should be advertised by the router?
a. N1: {G1, G2, G3} **b.** N2: {G1, G3} **c.** N3: {G1, G4} **d.** N4: {G1, G3}
- Q4-62.** It is obvious that we need to have spanning trees for both unicasting and multicasting. How many leaves of the tree are involved in a transmission in each case?
a. a unicast transmission **b.** a multicast transmission
- Q4-63.** List three protocols in the IPv4 network layer that are combined into a single protocol in IPv6.
- Q4-64.** What is the purpose of including the IP header and the first 8 bytes of datagram data in the error-reporting ICMP messages?
- Q4-65.** Explain the use of the flow field in IPv6. What is the potential application of this field?
- Q4-66.** Distinguish between compatible and mapped addresses and explain their applications.

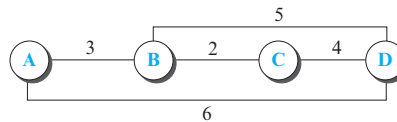
4.7.3 Problems

- P4-1.** Which fields of the IPv4 main header may change from router to router?
- P4-2.** Determine if a datagram with the following information is a first fragment, a middle fragment, a last fragment, or the only fragment (no fragmentation):
a. *M*-bit is set to 1 and the value of the offset field is zero.
b. *M*-bit is set to 1 and the value of the offset field is nonzero.
- P4-3.** Briefly describe how we can defeat the following security attacks:
a. packet sniffing **b.** packet modification **c.** IP spoofing
- P4-4.** What is the size of the address space in each of the following systems?
a. A system in which each address is only 16-bits.

- b. A system in which each address is made of six hexadecimal digits.
 c. A system in which each address is made of four octal digits.
- P4-5.** In an IPv4 datagram, the value of total-length field is $(00A0)_{16}$ and the value of the header-length (HLEN) is $(5)_{16}$. How many bytes of payload are being carried by the datagram? What is the efficiency (ratio of the payload length to the total length) of this datagram?
- P4-6.** An IP datagram has arrived with the following partial information in the header (in hexadecimal):
- 45000054 00030000 2006...**
- a. What is the header size?
 b. Is there any options in the packet?
 c. What is the size of the data?
 d. Is the packet fragmented?
 e. How many more routers can the packet travel to?
 f. What is the protocol number of the payload being carried by the packet?
- P4-7.** Find the class of the following classful IP addresses:
 a. 130.34.54.12 b. 200.34.2.1 c. 245.34.2.8
- P4-8.** Find the class of the following classful IP addresses:
 a. 01110111 11110011 10000111 11011101
 b. 11101111 11000000 11110000 00011101
 c. 11011111 10110000 00011111 01011101
- P4-9.** Rewrite the following IP addresses using binary notation:
 a. 110.11.5.88 b. 12.74.16.18 c. 201.24.44.32
- P4-10.** Rewrite the following IP addresses using dotted-decimal notation:
 a. 01011110 10110000 01110101 00010101
 b. 10001001 10001110 11010000 00110001
 c. 01010111 10000100 00110111 00001111
- P4-11.** Change each of the following masks to a prefix length:
 a. 255.224.0.0 b. 255.240.0.0 c. 255.255.255.128
- P4-12.** Which of the following cannot be a mask in CIDR?
 a. 255.225.0.0 b. 255.192.0.0 c. 255.255.255.6
- P4-13.** Each of the following addresses belongs to a block. Find the first and the last address in each block.
 a. 14.12.72.8/24 b. 200.107.16.17/18 c. 70.110.19.17/16
- P4-14.** Show the n leftmost bits of the following network-addresses/masks that can be used in a forwarding table (see Figure 4.45).
 a. 170.40.11.0/24 b. 110.40.240.0/22 c. 70.14.0.0/18
- P4-15.** In classless addressing, show the whole address space as a single block using the CIDR notation.
- P4-16.** In classless addressing, what is the size of the block (N) if the value of the prefix length (n) is one of the following?
 a. $n = 0$ b. $n = 14$ c. $n = 32$

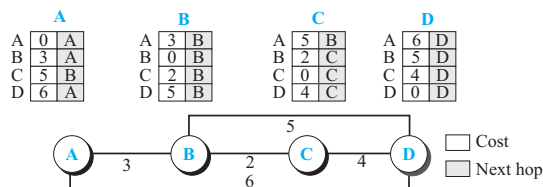
- P4-17.** In classless addressing, what is the value of prefix length (n) if the size of the block (N) is one of the following?
- a. $N = 1$ b. $N = 1024$ c. $N = 2^{32}$
- P4-18.** Change each of the following prefix lengths to a mask in dotted-decimal notation:
- a. $n = 0$ b. $n = 14$ c. $n = 30$
- P4-19.** Assume that we have an internet with an 8-bit address space. The addresses are equally divided between four networks (N_0 to N_3). The internetwork communication is done through a router with four interfaces (m_0 to m_3). Show the internet outline and the forwarding table (with two columns: prefix in binary and the interface number) for the only router that connects the networks. Assign a network address to each network.
- P4-20.** Assume that we have an internet with a 12-bit address space. The addresses are equally divided between eight networks (N_0 to N_7). The internetwork communication is done through a router with eight interfaces (m_0 to m_7). Show the internet outline and the forwarding table (with two columns: prefix in binary and the interface number) for the only router that connects the networks. Assign a network address to each network.
- P4-21.** Assume that we have an internet with a 9-bit address space. The addresses are divided between three networks (N_0 to N_2), with 64, 192, and 256 addresses, respectively. The internetwork communication is done through a router with three interfaces (m_0 to m_2). Show the internet outline and the forwarding table (with two columns: prefix in binary and the interface number) for the only router that connects the networks. Assign a network address to each network.
- P4-22.** Combine the following three blocks of addresses into a single block:
- a. 16.27.24.0/26 b. 16.27.24.64/26 c. 16.27.24.128/25
- P4-23.** A large organization with a large block address (12.44.184.0/21) is split into one medium-size company using the block address (12.44.184.0/22) and two small organizations. If the first small company uses the block (12.44.188.0/23), what is the remaining block that can be used by the second small company? Explain how the datagrams destined for the two small companies can be correctly routed to these companies if their address blocks are still part of the original company.
- P4-24.** An ISP is granted the block 16.12.64.0/20. The ISP needs to allocate addresses for eight organizations, each with 256 addresses.
- a. Find the number and range of addresses in the ISP block.
- b. Find the range of addresses for each organization and the range of unallocated addresses.
- c. Show the outline of the address distribution and the forwarding table.
- P4-25.** An ISP is granted the block 80.70.56.0/21. The ISP needs to allocate addresses for two organizations each with 500 addresses, two organizations each with 250 addresses, and three organizations each with 50 addresses.
- a. Find the number and range of addresses in the ISP block.

- b. Find the range of addresses for each organization and the range of unallocated addresses.
- c. Show the outline of the address distribution and the forwarding table.
- P4-26.** An organization is granted the block 130.56.0.0/16. The administrator wants to create 1024 subnets.
- a. Find the number of addresses in each subnet.
- b. Find the subnet prefix.
- c. Find the first and the last address in the first subnet.
- d. Find the first and the last address in the last subnet.
- P4-27.** Explain how DHCP can be used when the size of the block assigned to an organization is less than the number of hosts in the organization.
- P4-28.** Compare NAT and DHCP. Both can solve the problem of a shortage of addresses in an organization, but by using different strategies.
- P4-29.** When does an OSPF router send each of the following messages?
- a. hello b. data description c. link-state request
- d. link-state update e. link-state acknowledge
- P4-30.** To understand how the distance vector algorithm in Table 4.4 works, let us apply it to a four-node internet as shown in Figure 4.111.

Figure 4.111 Problem 4-30

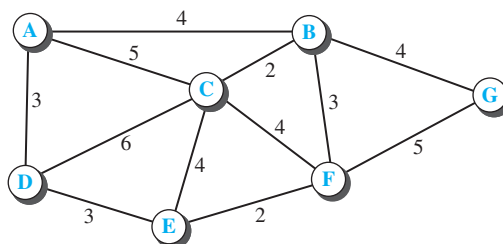
Assume that all nodes are initialized first. Also assume that the algorithm is applied, one at a time, to each node respectively (A, B, C, D). Show that the process converges and all nodes will have their stable distance vectors.

- P4-31.** In distance-vector routing, good news (decrease in a link metric) will propagate fast. In other words, if a link distance decreases, all nodes quickly learn about it and update their vectors. In Figure 4.112, we assume that a four-node internet is stable, but suddenly the distance between nodes A and D, which is currently 6, is decreased to 1 (probably due to some improvement in the link quality). Show how this good news is propagated, and find the new distance vector for each node after stabilization.

Figure 4.112 Problem 4-31

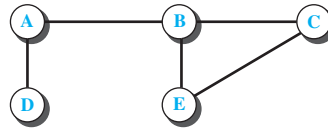
- P4-32.** In distance-vector routing, bad news (increase in a link metric) will propagate slowly. In other words, if a link distance increases, sometimes it takes a long time for all nodes to know the bad news. In Figure 4.112 (see the previous problem), we assume that a four-node internet is stable, but suddenly the distance between nodes B and C, which is currently 2, is increased to infinity (link fails). Show how this bad news is propagated, and find the new distance vector for each node after stabilization. Assume that the implementation uses a periodic timer to trigger updates to neighbors (no more updates are triggered when there is change). Also assume that if a node receives a higher cost from the same previous neighbor, it uses the new cost because this means that the old advertisement is not valid anymore. To make the stabilization faster, the implementation also suspends a route when the next hop is not accessible.
- P4-33.** Can router R1 in Figure 4.48 receive a packet with destination address 140.24.7.194? What will happen to the packet if this occurs?
- P4-34.** Assume that router R2 in Figure 4.48 receives a packet with destination address 140.24.7.42. How is the packet routed to its final destination?
- P4-35.** Assume that the shortest distance between nodes *a*, *b*, *c*, and *d* to node *y* and the costs from node *x* to nodes *a*, *b*, *c*, and *d* are given below:
- | | | | |
|--------------|--------------|--------------|--------------|
| $D_{ay} = 5$ | $D_{by} = 6$ | $D_{cy} = 4$ | $D_{dy} = 3$ |
| $c_{xa} = 2$ | $c_{xb} = 1$ | $c_{xc} = 3$ | $c_{xd} = 1$ |
- What is the shortest distance between node *x* and node *y*, D_{xy} , according to the Bellman-Ford equation?
- P4-36.** Assume that a router using RIP has 10 entries in its forwarding table at time t_1 . Six of these entries are still valid at time t_2 . Four of these entries have been expired 70, 90, 110, and 210 seconds before time t_2 . Find the number of periodic timers, expiration timers, and garbage collection timers running at time t_1 and time t_2 .
- P4-37.** Create the forwarding table for node A in Figure 4.65.
- P4-38.** Create the shortest-path tree and the forwarding table for node G in Figure 4.63.
- P4-39.** Create the shortest-path tree and the forwarding table for node B in Figure 4.63.
- P4-40.** Use Dijkstra's algorithm (Table 4.4) to find the shortest path tree and the forwarding table for node A in the Figure 4.113.

Figure 4.113 Problem 4-40



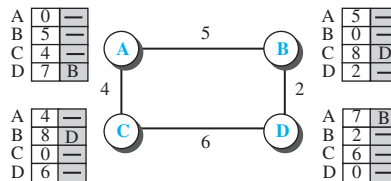
- P4-41.** In computer science, when we encounter an algorithm, we often need to ask about the complexity of that algorithm (how many computations we need to do). To find the complexity of Dijkstra's algorithm, find the number of searches we have to do in order to find the shortest path for a single node when the number of nodes is n .
- P4-42.** Assume that A, B, C, D, and E in Figure 4.114 are autonomous systems (ASs). Find the path vector for each AS using the algorithm in Table 4.6. Assume that the best path in this case is the path which passes through the shorter list of ASs. Also assume that the algorithm first initializes each AS and then is applied, one at a time, to each node respectively (A, B, C, D, E). Show that the process converges and all ASs will have their stable path vectors.

Figure 4.114 Problem 4-42



- P4-43.** In computer science, when we encounter an algorithm, we often need to ask about the complexity of that algorithm (how many computations we need to do). To find the complexity of the distance vector's algorithm, find the number of operations a node needs to do when it receives a vector from a neighbor.
- P4-44.** Assume that the network in Figure 4.115 uses distance-vector routing with the forwarding table as shown for each node. If each node periodically announces

Figure 4.115 Problem 4-44

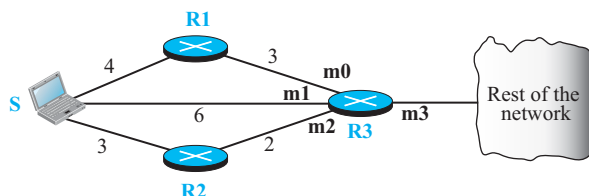


their vectors to the neighbor using the poisoned-reverse strategy, what is the distance vector advertised in the appropriate period:

- a.** from A to B? **b.** from C to D? **c.** from D to B? **d.** from C to A?
- P4-45.** Assume that the network in Figure 4.115 (previous problem) uses distance-vector routing with the forwarding table as shown for each node. If each node periodically announces their vectors to the neighbor using the split-horizon strategy, what is the distance vector advertised in the appropriate period:
- a.** from A to B? **b.** from C to D? **c.** from D to B? **d.** from C to A?

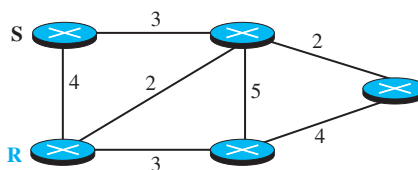
- P4-46.** Assume that the network in Figure 4.115 (Problem 4-34) uses distance vector routing with the forwarding table as shown for each node. If node E is added to the network with a link of cost 1 to node D, can you find the new forwarding tables for each node without using the distance-vector algorithm?
- P4-47.** In Figure 4.116, assume that router R3 has received three copies of the same packet from source S through three interfaces, m0, m1, and m2. Which packet should be forwarded through interface m3 to the rest of the network if the router R3 uses RPF strategy?

Figure 4.116 Problem 4-47



- P4-48.** Assume that m is much less than n and that router R is connected to n networks in which only m of these networks are interested in receiving packets related to group G. How can router R manage to send a copy of the packet to only those network interested in group G?
- P4-49.** In the network of Figure 4.117, find the shortest path trees for router R in two cases: first, if the network is using OSPF; second, if the network is using MOSPF with the source connected to the router marked as S. Assume that all routers have interest in the corresponding multicast group.

Figure 4.117 Problem 4-49



- P4-50.** What is the size of a RIP message that advertises only one network? What is the size of a RIP message that advertises n networks?
- P4-51.** In Figure 4.79, assume that the intra-AS routing protocol used by AS1 is OSPF, but the one used by AS2 is RIP. Explain how R5 can find how to route a packet to N4.
- P4-52.** In Figure 4.79, assume that the intra-AS routing protocol used by AS4 and AS3 is RIP. Explain how R8 can find how to route a packet to N13.

- P4-53.** Show the unabbreviated colon hex notation for the following IPv6 addresses:
- An address with 64 0s followed by 32 two-bit (01)s.
 - An address with 64 0s followed by 32 two-bit (10)s.
 - An address with 64 two-bit (01)s.
 - An address with 32 four-bit (0111)s.
- P4-54.** Show abbreviations for the following addresses:
- 0000 : FFFF : FFFF : 0000 : 0000 : 0000 : 0000 : 0000
 - 1234 : 2346 : 3456 : 0000 : 0000 : 0000 : 0000 : FFFF
 - 0000 : 0001 : 0000 : 0000 : 0000 : FFFF : 1200 : 1000
 - 0000 : 0000 : 0000 : 0000 : FFFF : FFFF : 24 . 123 . 12 . 6
- P4-55.** Decompress the following addresses and show the complete unabbreviated IPv6 address:
- ::2222
 - 1111::
 - B:A:CC::1234:A
- P4-56.** Show the original (unabbreviated) form of the following IPv6 addresses:
- ::2
 - 0:23::0
 - 0:A::3
- P4-57.** What is the corresponding block or subblock associated with each of the following IPv6 addresses based on Table 4.7:
- FE80::12/10
 - FD23::/7
 - 32::/3
- P4-58.** An organization is assigned the block 2000:1234:1423/48. What is the CIDR for the blocks in the first and second subnets in this organization?
- P4-59.** A router using RIP has the following forwarding table. Show the RIP update message sent by this router.

<i>Destination</i>	<i>Cost</i>	<i>Next Router</i>
Net1	4	B
Net2	2	C
Net3	1	F
Net4	5	G

- P4-60.** Compare and contrast the IPv4 header with the IPv6 header. Create a table to compare each field.

4.8 SIMULATION EXPERIMENTS

4.8.1 Applets

We have created some Java applets to show some of the main concepts discussed in this chapter. It is strongly recommended that students activate these applets on the book website and carefully examine the protocols in action.

4.8.2 Lab Assignments

In this chapter, we use Wireshark to capture and investigate some packets exchanged at the network layer. We use Wireshark and some other computer network administration utilities. See the book website for complete details of lab assignments.

Lab4-1. In the first lab, we investigate IP protocol by capturing and studying IP datagrams.

Lab4-2. In the second lab, we capture and study ICMP packets generated by other utility programs such as ping and traceroute programs.

4.9 PROGRAMMING ASSIGNMENT

Write the source code, compile, and test the following programs in one of the programming languages of your choice:

Prg4-1. Write a program to simulate the distance vector algorithm (Table 4.4).

Prg4-2. Write a program to simulate the link state algorithm (Table 4.5).

Prg4-3. Write a program to simulate the path vector algorithm (Table 4.6).